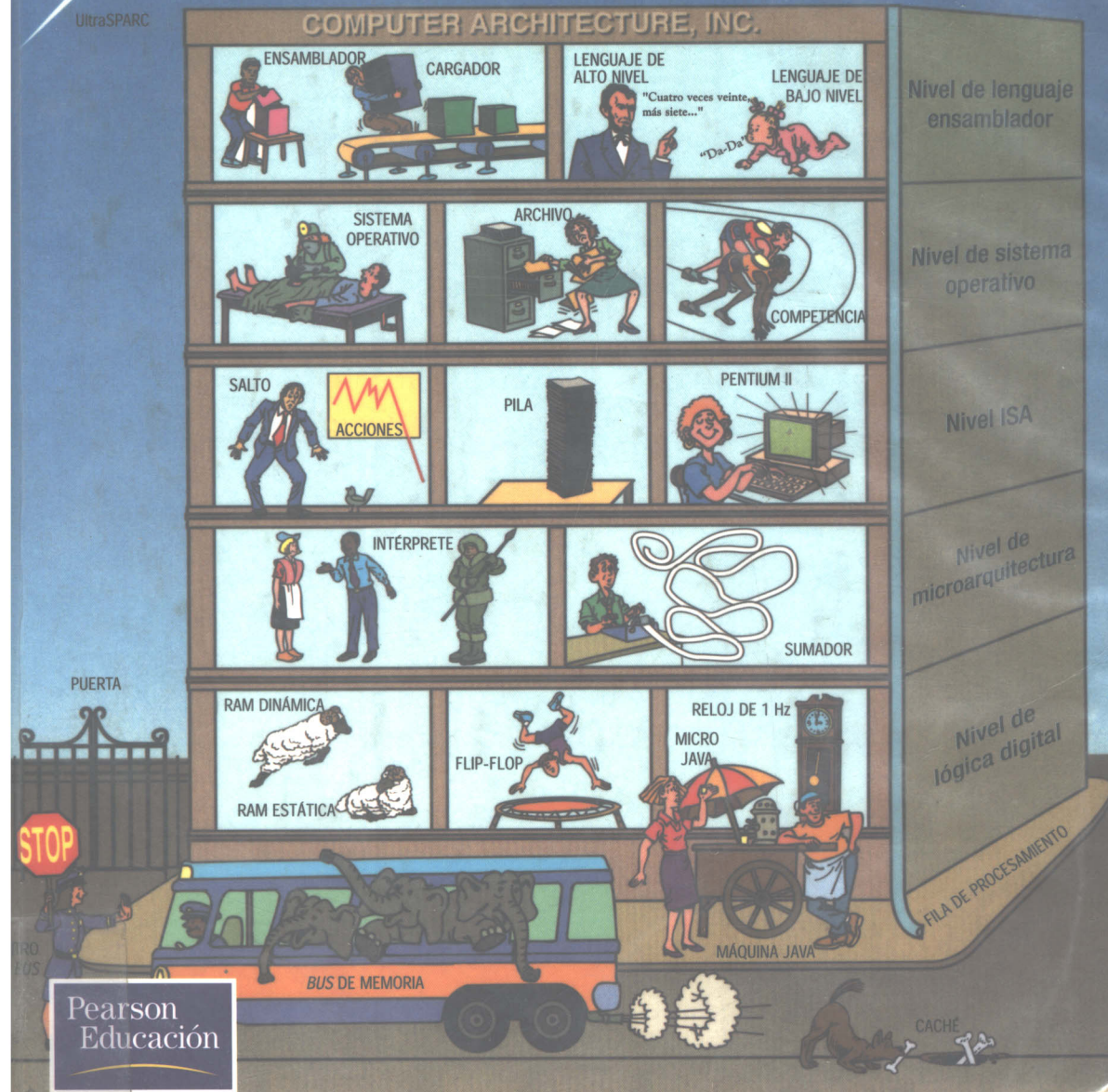
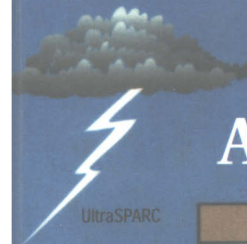


Cuarta edición

ORGANIZACIÓN DE COMPUTADORAS

UN ENFOQUE ESTRUCTURADO

Andrew S. Tanenbaum



Pearson Educación

ORGANIZACIÓN DE COMPUTADORAS

un enfoque estructurado

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

LOSA LIBROS
Colonia 1651/53
Tel/Fax: 401 - 2905 - 401 - 8587
[http:// www.losa.com.uy](http://www.losa.com.uy)
libros@losa.com.uy
Montevideo - Uruguay

Otros títulos exitosos de Andrew S. Tanenbaum

Redes de computadoras, tercera edición

Este clásico tan leído proporciona la introducción ideal a las redes de hoy y del mañana. Explica en detalle cómo se estructuran las redes modernas. Partiendo de un nivel físico y avanzando hasta el nivel de aplicación, el libro abarca un vasto número de temas importantes, incluyendo comunicación inalámbrica, fibra óptica, protocolos de cadenas de datos, Ethernet, algoritmos de enrutamiento, comportamiento de redes, seguridad, DNS, correo electrónico, novedades de USENET, la World Wide Web y multimedia. El libro presenta un estudio completo de TCP/IP e Internet.

Sistemas operativos: diseño e implementación, segunda edición

Este famoso texto sobre sistemas operativos es el único que abarca tanto los principios de los sistemas operativos como sus aplicaciones en un sistema real. En él se analizan en detalle todos los temas tradicionales de sistemas operativos. Se ilustran los principios con MINIX, un sistema operativo libre del tipo UNIX basado en POSIX, para computadoras personales. Cada libro incluye un disco compacto gratis con el sistema MINIX completo, incluyendo todo el código fuente. Éste se lista en un apéndice del libro y se explica en detalle dentro del texto.

Sistemas operativos modernos

Este exitoso texto presenta las bases de un procesador simple y de un sistema de computadora distribuido. Trata todos los temas tradicionales, incluyendo procesos, administración de memoria y sistemas de archivos, así como temas clave sobre sistemas distribuidos, entre ellos, el modelo cliente-servidor, las llamadas de procedimiento remoto, las redes y los servidores de archivo distribuidos. Los principios empleados en el libro se ilustran con dos procesadores simples extensivos y dos sistemas distribuidos.

Sistemas operativos distribuidos

Este texto incluye los conceptos fundamentales de los sistemas operativos distribuidos. Entre los temas clave están la comunicación y la sincronización, los procesos y los procesadores, la memoria compartida distribuida, los sistemas de archivos distribuidos y los sistemas distribuidos de tiempo real. Los principios se ilustran con cuatro ejemplos de capítulo extensos.

ORGANIZACIÓN DE
COMPUTADORAS
un enfoque estructurado

CUARTA EDICIÓN

ANDREW S. TANENBAUM
*Vrije Universiteit
Amsterdam, Países Bajos*

Con la colaboración de
JAMES R. GOODMAN
*University of Wisconsin
Madison, WI*

TRADUCCIÓN:
Luis Roberto Escalona García
Traductor profesional

REVISIÓN TÉCNICA:
Alma Corral López
*Ingeniería en Electrónica
Instituto Tecnológico y de Estudios Superiores de
Monterrey, Campus Estado de México
Maestría en Sistemas de Manufactura
University of Texas at Austin*

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DPTO. DE DOCUMENTACION Y BIBLIOTECA
BIBLIOTECA CENTRAL
Ing. Edm. Garcia de Zúñiga
MONTEVIDEO - URUGUAY

No. de Entrada 053845
1.8.01



MÉXICO • ARGENTINA • BRASIL • COLOMBIA • COSTA RICA • CHILE
ESPAÑA • GUATEMALA • PERÚ • PUERTO RICO • VENEZUELA

Datos de catalogación bibliográfica

Tanenbaum, S. Andrew
Organización de computadoras:
un enfoque estructurado
PRENTICE HALL
México, 2000

ISBN: 970-17-0399-5
Área: Universitarios

Formato: 17 × 23 cm

Páginas: 688

Versión en español de la obra titulada *Structured computer organization, Fourth Edition*, de Andrew S. Tanenbaum, publicada originalmente en inglés por Prentice Hall Inc., Upper Saddle River, New Jersey, U.S.A.

Esta edición en español es la única autorizada.

Original English language title by
Prentice Hall Inc.
Copyright © 1999
All rights reserved
ISBN 0-13-095990-1

Edición en español:

Editor: Pablo Eduardo Roig Vázquez
Supervisor de traducción: Rocío Cabañas Chávez
Supervisor de producción: Alejandro A. Gómez Ruiz

Edición en inglés:
Publisher: Alan Apt
Development editor: Sondra Chavez
Editor-in-chief: Marcia Horton
Production editor: Irwin Zucker
Managing editor: Eileen Clark
Composition and interior design: Andrew S. Tanenbaum
Cover concept: Andrew S. Tanenbaum
Cover illustrator: Don Martinetti, DM Graphics, Inc.

SÉPTIMA EDICIÓN, 2000

D.R. © 2000 por Prentice Hall Hispanoamericana, S.A.
Calle 4 Núm. 25-2do. piso
Fracc. Industrial Alce Blanco
53370 Naucalpan de Juárez, Edo. de México

Cámara Nacional de la Industria Editorial Mexicana. Reg. Núm. 1524.

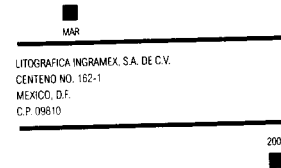
Reservados todos los derechos. Ni la totalidad ni parte de esta publicación pueden reproducirse, registrarse o transmitirse, por un sistema de recuperación de información, en ninguna forma ni por ningún medio, sea electrónico, mecánico, fotoquímico, magnético o electroóptico, por fotocopia, grabación o cualquier otro, sin permiso previo por escrito del editor.

El préstamo, alquiler o cualquier otra forma de cesión de uso de este ejemplar requerirá también la autorización del editor o de sus representantes.

ISBN 970-17-0399-5

Impreso en México. *Printed in Mexico*

1 2 3 4 5 6 7 8 9 0 03 02 01 00



A Suzanne, Barbara, Marvin, Bram, y a la memoria de Sweetie π

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

CONTENIDO

PREFACIO

xvi

1 INTRODUCCIÓN

1

1.1 ORGANIZACIÓN ESTRUCTURADA DE COMPUTADORAS 2

1.1.1 Lenguajes, niveles y máquinas virtuales 2

1.1.2 Máquinas multinivel contemporáneas 4

1.1.3 Evolución de las máquinas multinivel 8

1.2 ACONTECIMIENTOS IMPORTANTES EN ARQUITECTURA DE COMPUTADORAS 13

1.2.1 La generación cero — computadoras mecánicas (1642-1945) 13

1.2.2 La primera generación — bulbos (1945-1955) 16

1.2.3 La segunda generación — transistores (1955-1965) 19

1.2.4 La tercera generación — circuitos integrados (1965-1980) 21

1.2.5 La cuarta generación — integración a muy grande escala (1980-?) 23

1.3 EL ZOOLÓGICO DE LAS COMPUTADORAS 24

1.3.1 Fuerzas tecnológicas y económicas 25

1.3.2 La gama de las computadoras 26

1.4 EJEMPLOS DE FAMILIAS DE COMPUTADORAS 29

1.4.1 Introducción al Pentium II 29

1.4.2 Introducción al UltraSPARC II 31

1.4.3 Introducción al picoJava II 34

1.5 BOSQUEJO DEL LIBRO 36

2 ORGANIZACIÓN DE LOS SISTEMAS DE COMPUTADORA 39

- 2.1 PROCESADORES 39
 - 2.1.1 Organización de la CPU 40
 - 2.1.2 Ejecución de instrucciones 42
 - 2.1.3 RISC *versus* CISC 46
 - 2.1.4 Principios de diseño de las computadoras modernas 47
 - 2.1.5 Paralelismo en el nivel de instrucciones 49
 - 2.1.6 Paralelismo en el nivel de procesador 53
- 2.2 MEMORIA PRIMARIA 56
 - 2.2.1 Bits 56
 - 2.2.2 Direcciones de memoria 57
 - 2.2.3 Ordenamiento de bytes 58
 - 2.2.4 Códigos para corrección de errores 61
 - 2.2.5 Memoria caché 65
 - 2.2.6 Empaquetamiento y tipos de memoria 67
- 2.3 MEMORIA SECUNDARIA 68
 - 2.3.1 Jerarquías de memoria 69
 - 2.3.2 Discos magnéticos 70
 - 2.3.3 Discos flexibles 73
 - 2.3.4 Discos IDE 73
 - 2.3.5 Discos SCSI 75
 - 2.3.6 RAID 76
 - 2.3.7 CD-ROM 80
 - 2.3.8 CD grabables 84
 - 2.3.9 CD reescribibles 86
 - 2.3.10 DVD 86
- 2.4 ENTRADA/SALIDA 89
 - 2.4.1 Buses 89
 - 2.4.2 Terminales 91
 - 2.4.3 Ratones 99
 - 2.4.4 Impresoras 101
 - 2.4.5 Módems 106
 - 2.4.6 Códigos de caracteres 109
- 2.5 RESUMEN 113

3 EL NIVEL DE LÓGICA DIGITAL 117

- 3.1 COMPUERTAS Y ÁLGEBRA BOOLEANA 117
 - 3.1.1 Compuertas 118
 - 3.1.2 Álgebra booleana 120
 - 3.1.3 Implementación de funciones booleanas 122
 - 3.1.4 Equivalencia de circuitos 123
- 3.2 CIRCUITOS LÓGICOS DIGITALES BÁSICOS 128
 - 3.2.1 Circuitos integrados 128
 - 3.2.2 Circuitos combinacionales 129
 - 3.2.3 Circuitos aritméticos 134
 - 3.2.4 Relojes 139
- 3.3 MEMORIA 141
 - 3.3.1 Latches 141
 - 3.3.2 Flip-flops 143
 - 3.3.3 Registros 145
 - 3.3.4 Organización de la memoria 146
 - 3.3.5 Chips de memoria 150
 - 3.3.6 Las memorias RAM y las ROM 152
- 3.4 CHIPS DE CPU Y BUSES 154
 - 3.4.1 Chips de CPU 154
 - 3.4.2 Buses de computadora 156
 - 3.4.3 Ancho de bus 159
 - 3.4.4 Temporización del bus 160
 - 3.4.5 Arbitraje del bus 165
 - 3.4.6 Operaciones de bus 167
- 3.5 EJEMPLOS DE CHIPS DE CPU 170
 - 3.5.1 El Pentium II 170
 - 3.5.2 El UltraSPARC II 176
 - 3.5.3 El picoJava II 179
- 3.6 EJEMPLOS DE BUSES 181
 - 3.6.1 El bus ISA 181
 - 3.6.2 El bus PCI 183
 - 3.6.3 El bus serial universal 189

3.7	INTERFACES	193
3.7.1	Chips de E/S	193
3.7.2	Decodificación de direcciones	195
3.8	RESUMEN	198
4	EL NIVEL DE MICROARQUITECTURA	203
4.1	UN EJEMPLO DE MICROARQUITECTURA	203
4.1.1	La trayectoria de datos	204
4.1.2	Microinstrucciones	211
4.1.3	Control de microinstrucciones: el Mic-1	213
4.2	UN EJEMPLO DE ISA: IJVM	218
4.2.1	Pilas	218
4.2.2	El modelo de memoria IJVM	220
4.2.3	El conjunto de instrucciones IJVM	222
4.2.4	Compilación de Java a IJVM	226
4.3	UN EJEMPLO DE IMPLEMENTACIÓN	227
4.3.1	Microinstrucciones y notación	227
4.3.2	Implementación de IJVM utilizando el Mic-1	232
4.4	DISEÑO DEL NIVEL DE MICROARQUITECTURA	243
4.4.1	Rapidez <i>versus</i> costo	243
4.4.2	Reducción de la longitud de la trayectoria de ejecución	245
4.4.3	Un diseño con prebúsqueda: el Mic-2	253
4.4.4	Un diseño con filas de procesamiento: el Mic-3	253
4.4.5	Una fila de procesamiento de siete etapas: el Mic-4	260
4.5	MEJORAMIENTO DEL DESEMPEÑO	264
4.5.1	Memoria caché	265
4.5.2	Predicción de ramificaciones	270
4.5.3	Ejecución fuera de orden y cambio de nombres de registros	276
4.5.4	Ejecución especulativa	281
4.6	EJEMPLOS DEL NIVEL DE MICROARQUITECTURA	283
4.6.1	La microarquitectura de la CPU Pentium II	283
4.6.2	La microarquitectura de la CPU UltraSPARC II	288
4.6.3	La microarquitectura de la CPU picoJava II	291
4.6.4	Comparación del Pentium, UltraSPARC y picoJava	296
4.7	RESUMEN	298

5 EL NIVEL DE ARQUITECTURA DEL CONJUNTO DE INSTRUCCIONES

303

5.1	GENERALIDADES DEL NIVEL ISA	305
5.1.1	Propiedades del nivel ISA	305
5.1.2	Modelos de memoria	307
5.1.3	Registros	309
5.1.4	Instrucciones	311
5.1.5	Generalidades del nivel ISA del Pentium II	311
5.1.6	Generalidades del nivel ISA del UltraSPARC II	313
5.1.7	Generalidades del nivel ISA de la Máquina Virtual Java	317
5.2	TIPOS DE DATOS	318
5.2.1	Tipos de datos numéricos	319
5.2.2	Tipos de datos no numéricos	319
5.2.3	Tipos de datos en el Pentium II	320
5.2.4	Tipos de datos en el UltraSPARC II	321
5.2.5	Tipos de datos en la Máquina Virtual Java	321
5.3	FORMATOS DE INSTRUCCIONES	322
5.3.1	Criterios de diseño para los formatos de instrucciones	322
5.3.2	Expansión de códigos de operación	325
5.3.3	Los formatos de instrucciones del Pentium II	327
5.3.4	Los formatos de instrucciones del UltraSPARC II	328
5.3.5	Los formatos de instrucciones de la JVM	330
5.4	DIRECCIONAMIENTO	332
5.4.1	Modos de direccionamiento	333
5.4.2	Direccionamiento inmediato	334
5.4.3	Direccionamiento directo	334
5.4.4	Direccionamiento por registro	334
5.4.5	Direccionamiento indirecto por registro	335
5.4.6	Direccionamiento indexado	336
5.4.7	Direccionamiento basado indexado	338
5.4.8	Direccionamiento de pila	338
5.4.9	Modos de direccionamiento para instrucciones de ramificación	341
5.4.10	Ortogonalidad de códigos de operación y modos de direccionamiento	342
5.4.11	Modos de direccionamiento del Pentium II	344
5.4.12	Modos de direccionamiento del UltraSPARC II	346
5.4.13	Modos de direccionamiento de la JVM	346
5.4.14	Análisis de los modos de direccionamiento	347

5.5	TIPOS DE INSTRUCCIONES	348
5.5.1	Instrucciones de movimiento de datos	348
5.5.2	Operaciones diádicas	349
5.5.3	Operaciones monádicas	350
5.5.4	Comparaciones y ramificaciones condicionales	352
5.5.5	Instrucciones de llamada a procedimientos	353
5.5.6	Control de ciclos	354
5.5.7	Entrada/Salida	356
5.5.8	Las instrucciones del Pentium II	359
5.5.9	Las instrucciones del UltraSPARC II	362
5.5.10	Las instrucciones del picoJava II	364
5.5.11	Comparación de conjuntos de instrucciones	369
5.6	FLUJO DE CONTROL	370
5.6.1	Flujo de control secuencial y ramificaciones	371
5.6.2	Procedimientos	372
5.6.3	Corrutinas	376
5.6.4	Trampas	379
5.6.5	Interrupciones	379
5.7	UN EJEMPLO DETALLADO: LAS TORRES DE HANOI	383
5.7.1	Las Torres de Hanoi en lenguaje ensamblador del Pentium II	384
5.7.2	Las Torres de Hanoi en lenguaje ensamblador del UltraSPARC II	384
5.7.3	Las Torres de Hanoi en lenguaje ensamblador de la JVM	386
5.8	EL IA-64 DE INTEL	388
5.8.1	El problema del Pentium II	390
5.8.2	El modelo IA-64: Computación con instrucciones explícitamente paralelas	391
5.8.3	Predicación	393
5.8.4	Cargas especulativas	395
5.8.5	Los pies en la tierra	396
5.9	RESUMEN	397
6	EL NIVEL DE MÁQUINA DE SISTEMA OPERATIVO	403
6.1	MEMORIA VIRTUAL	404
6.1.1	Paginación	405
6.1.2	Implementación de la paginación	407
6.1.3	Paginación por demanda y modelo de conjunto de trabajo	409
6.1.4	Política de reemplazo de páginas	412

6.1.5	Tamaño de página y fragmentación	414
6.1.6	Segmentación	415
6.1.7	Implementación de la segmentación	418
6.1.8	Memoria virtual en el Pentium II	421
6.1.9	Memoria virtual en el UltraSPARC II	426
6.1.10	Memoria virtual y uso de cachés	428
6.2	INSTRUCCIONES DE E/S VIRTUALES	429
6.2.1	Archivos	430
6.2.2	Implementación de instrucciones de E/S virtuales	431
6.2.3	Instrucciones para gestión de directorios	435
6.3	INSTRUCCIONES VIRTUALES PARA PROCESAMIENTO EN PARALELO	436
6.3.1	Creación de procesos	437
6.3.2	Condiciones de competencia	438
6.3.3	Sincronización de procesos con semáforos	442
6.4	EJEMPLOS DE SISTEMAS OPERATIVOS	446
6.4.1	Introducción	446
6.4.2	Ejemplos de memoria virtual	455
6.4.3	Ejemplos de E/S virtual	459
6.4.4	Ejemplos de gestión de procesos	470
6.5	RESUMEN	476
7	EL NIVEL DE LENGUAJE ENSAMBLADOR	483
7.1	INTRODUCCIÓN AL LENGUAJE ENSAMBLADOR	484
7.1.1	¿Qué es un lenguaje ensamblador?	484
7.1.2	¿Por qué usar lenguaje ensamblador?	485
7.1.3	Formato de un enunciado en lenguaje ensamblador	488
7.1.4	Seudoinstrucciones	491
7.2	MACROS	494
7.2.1	Definición, llamada y expansión de macros	494
7.2.2	Macros con parámetros	496
7.2.3	Características avanzadas	497
7.2.4	Implementación de un recurso de macros en un ensamblador	498
7.3	EL PROCESO DE ENSAMBLADO	498
7.3.1	Ensambladores de dos pasadas	498
7.3.2	Primera pasada	499

7.3.3 Segunda pasada	502
7.3.4 La tabla de símbolos	505
7.4 ENLAZADO Y CARGA	506
7.4.1 Tareas que realiza el enlazador	508
7.4.2 Estructura de un módulo objeto	511
7.4.3 Tiempo de ligado y reubicación dinámica	512
7.4.4 Enlazado dinámico	515
7.5 RESUMEN	519
8 ARQUITECTURAS DE COMPUTADORAS PARALELAS	523
8.1 ASPECTOS DEL DISEÑO DE COMPUTADORAS PARALELAS	524
8.1.1 Modelos de comunicación	526
8.1.2 Redes de interconexión	530
8.1.3 Desempeño	539
8.1.4 Software	545
8.1.5 Taxonomía de computadoras paralelas	551
8.2 COMPUTADORAS SIMD	554
8.2.1 Arreglos de procesadores	554
8.2.2 Procesadores vectoriales	555
8.3 MULTIPROCESADORES CON MEMORIA COMPARTIDA	559
8.3.1 Semántica de la memoria	559
8.3.2 Arquitecturas SMP basadas en el bus UMA	564
8.3.3 Multiprocesadores UMA que usan conmutadores de barras cruzadas	569
8.3.4 Multiprocesadores UMA que usan redes de conmutación multietapas	571
8.3.5 Multiprocesadores NUMA	573
8.3.6 Multiprocesadores NUMA con coherencia de caché	575
8.3.7 Multiprocesadores COMA	585
8.4 MULTICOMPUTADORAS DE TRANSFERENCIA DE MENSAJES	586
8.4.1 Procesadores masivamente paralelos (MPP)	587
8.4.2 Cúmulos de estaciones de trabajo (COW)	592
8.4.3 Planificación	593
8.4.4 Software de comunicación para multicomputadoras	598
8.4.5 Memoria compartida en el nivel de aplicaciones	601
8.5 RESUMEN	609

9 LISTA DE LECTURAS Y BIBLIOGRAFÍA	613
9.1 SUGERENCIAS PARA LECTURAS ADICIONALES	613
9.1.1 Introducción y obras generales	613
9.1.2 Organización de sistemas de cómputo	614
9.1.3 El nivel de lógica digital	615
9.1.4 El nivel de microarquitectura	616
9.1.5 El nivel de arquitectura del conjunto de instrucciones	617
9.1.6 El nivel de máquina del sistema operativo	617
9.1.7 El nivel de lenguaje ensamblador	618
9.1.8 Arquitecturas de computadoras paralelas	618
9.1.9 Números binarios y de punto flotante	620
9.2 BIBLIOGRAFÍA EN ORDEN ALFABÉTICO	620
A NÚMEROS BINARIOS	631
A.1 NÚMEROS DE PRECISIÓN FINITA	631
A.2 SISTEMAS NUMÉRICOS CON BASE	633
A.3 CONVERSIÓN DE UNA BASE A OTRA	635
A.4 NÚMEROS BINARIOS NEGATIVOS	637
A.5 ARITMÉTICA BINARIA	640
B NÚMEROS DE PUNTO FLOTANTE	643
B.1 PRINCIPIOS DE PUNTO FLOTANTE	644
B.2 ESTÁNDAR DE PUNTO FLOTANTE IEEE 754	646
ÍNDICE	653

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

PREFACIO

Las primeras tres ediciones de este libro se basaron en la idea de que una computadora puede considerarse como una jerarquía de niveles, cada uno de los cuales desempeña alguna función bien definida. Este concepto fundamental es tan válido hoy como cuando apareció la primera edición, así que se ha conservado como base para la cuarta edición. Al igual que en las tres primeras ediciones, se analizan detalladamente el nivel de lógica digital, el nivel de microarquitectura, el nivel de arquitectura del conjunto de instrucciones, el nivel de máquina del sistema operativo y el nivel de lenguaje ensamblador (aunque hemos modificado algunos de los nombres de modo que reflejen la práctica moderna).

Aunque se ha mantenido la estructura básica, esta cuarta edición contiene muchos cambios, pequeños y grandes, que lo ponen al día en la industria de las computadoras con su vertiginoso ritmo de cambio. Por ejemplo, todos los ejemplos de código, que estaban en Pascal, se han reescrito en Java con objeto de reflejar la popularidad de este lenguaje en el mundo de la computación. Además, se han actualizado los ejemplos de máquinas empleados. Los ejemplos actuales son el Intel Pentium II, el Sun UltraSPARC II y el Sun picoJava II, un chip Java incorporado de bajo costo.

El uso de los multiprocesadores y las computadoras paralelas se ha extendido mucho desde la tercera edición, por lo que se reescribió totalmente el material sobre arquitecturas paralelas, expandiéndolo considerablemente; ahora cubre una amplia gama de temas, desde multiprocesadores hasta los COW.

El libro se ha vuelto más extenso con el paso de los años (aunque todavía no es tan largo como otros libros populares sobre el tema). Tal expansión es inevitable a medida que avanzan los conocimientos en el área. Por ello, si el libro se usa para un curso, tal vez no siempre sea posible terminar el libro en un solo curso (por ejemplo, en un sistema de trimestres). Una

posible estrategia sería cubrir los capítulos 1, 2 y 3 en su totalidad, la primera parte del capítulo 4 (hasta la sección 4.4 inclusive) y el capítulo 5 como mínimo indispensable. Si sobra tiempo, podría dedicarse al resto del capítulo 4 y partes de los capítulos 6, 7 y 8, dependiendo del interés del profesor.

A continuación describimos los principales cambios hechos a cada capítulo desde la tercera edición. El capítulo 1 todavía contiene una reseña histórica de la arquitectura de las computadoras; nos dice cómo llegamos a dónde estamos ahora y qué hitos pasamos en el camino. Se describe la gama más amplia de computadoras que existen ahora y se presentan nuestros tres ejemplos principales (Pentium II, UltraSPARC II y picoJava II).

En el capítulo 2 se actualizó el material sobre dispositivos de entrada/salida, con hincapié en la tecnología de los dispositivos modernos, que incluyen los discos RAID, los CD grabables, los DVD y las impresoras a color, entre muchos otros.

El capítulo 3 (nivel de lógica digital) sufrió algunas modificaciones y ahora trata los buses de computadora y los modernos chips de E/S. El principal cambio aquí es material adicional sobre buses, sobre todo el bus PCI y el bus USB. Los tres nuevos ejemplos se describen aquí en el nivel de chip.

El capítulo 4 (que ahora se llama nivel de microarquitectura) se reescribió totalmente. Se conservó la idea de usar un ejemplo detallado de máquina microprogramada para ilustrar las ideas de control del camino de datos, pero la máquina de ejemplo ahora es un subconjunto de la Máquina Virtual Java. La microarquitectura subyacente se modificó de manera acorde. Se dan varias iteraciones del diseño, mostrando los posibles equilibrios en términos de costo y desempeño. El último ejemplo, el Mic-4, usa un conducto de siete etapas y ofrece una introducción accesible al funcionamiento de las computadoras modernas importantes como el Pentium II. Se añadió una sección nueva sobre cómo mejorar el desempeño, que se concentra en las técnicas más recientes como uso de cachés, predicción de ramas, ejecución en desorden (superescalar) y predicación. Las nuevas máquinas de ejemplo se describen en el nivel de microarquitectura.

El capítulo 5 (que ahora se llama nivel de arquitectura del conjunto de instrucciones) se ocupa de lo que mucha gente llama el “lenguaje de máquina”. Aquí se usan como ejemplos primarios el Pentium II, el UltraSPARC II y la Máquina Virtual Java.

El capítulo 6 (nivel de máquina del sistema operativo) tiene ejemplos para el Pentium II (Windows NT) y UltraSPARC II (UNIX). El primero es nuevo y tiene muchas características que vale la pena examinar, pero UNIX sigue siendo un caballito de batalla confiable en muchas universidades y compañías, y además resulta provechoso examinarlo con detalle por su sencillo y elegante diseño.

El capítulo 7 (nivel de lenguaje ensamblador) se actualizó usando ejemplos de las máquinas que hemos estado estudiando. También se añadió material nuevo sobre enlazado dinámico.

El capítulo 8 (arquitecturas de computadoras paralelas) se reescribió totalmente. Ahora cubre con detalle tanto multiprocesadores (UMA, NUMA y COM) como multicomputadoras (MPP y COW).

La bibliografía se sometió a una revisión y actualización exhaustivas. Más de las dos terceras partes de las referencias corresponden a obras que se publicaron después de que lo

hiciera la tercera edición de este libro. Los números binarios y los de punto flotante no han sufrido muchos cambios a últimas fechas, por lo que los apéndices son prácticamente los mismos de la edición anterior.

Por último, algunos problemas se modificaron y se añadieron muchos problemas nuevos respecto a la tercera edición.

Existe un sitio Web para este libro. Es posible obtener electrónicamente archivos PostScript para todas las ilustraciones del libro. Estos archivos pueden imprimirse, por ejemplo, para crear transparencias. Además, el sitio incluye un simulador y otras herramientas de software. El URL para este sitio es

<http://www.cs.vu.nl/~ast/sco4/>

El simulador y las herramientas de software son producto de Ray Ontko. El autor desea expresar su gratitud a Ray por haber producido tan útiles programas.

Varias personas leyeron (parcialmente) el manuscrito y ofrecieron sugerencias útiles o ayudaron de otras maneras. En particular, quiero agradecer a Henri Bal, Alan Charlesworth, Kourosh Gharachorloo, Marcus Goncalves, Karen Panetta Lentz, Timothy Mattson, Harlan McGhan, Miles Murdocca, Kevin Normoyle, Mike O'Connor, Mitsunori Ogihara, Ray Ontko, Aske Plaat, William Potvin II, Nagarajan Prabhakaran, James H. Pugsley, Ronald N. Schroeder, Ryan Shoemaker, Charles Silio, Jr., y Dale Skrien su ayuda, que ha sido inapreciable. Mis estudiantes, sobre todo Adriaan Bon, Laura de Vries, Dolf Loth y Guido van't Noordende, también ayudaron a depurar el texto. Muchas gracias.

Me gustaría expresar un agradecimiento especial a Jim Goodman por sus muchas contribuciones a este libro, sobre todo a los capítulos 4 y 5. La idea de usar la Máquina Virtual Java fue suya, lo mismo que las microarquitecturas para implementarla. Muchas de las ideas avanzadas se deben a él. Este libro es mucho mejor de lo que habría sido sin su esfuerzo.

Por último, quiero agradecer a Suzanne su paciencia ante mis largas horas absorto en mi Pentium. Desde mi punto de vista, el Pentium es mucho mejor que mi antiguo 386, pero a ella no le parece que haya mucha diferencia. También quiero expresar mi gratitud a Barbara y Marvin por ser unos chicos excelentes, y a Bram por permanecer callado cuando yo trataba de escribir.

Andrew S. Tanenbaum

UNIVERSIDAD DE LA REPUBLICA
FACULTAD DE INGENIERIA
DEPARTAMENTO DE
DOCUMENTACION Y BIBLIOTECA
MONTEVIDEO - URUGUAY

1

INTRODUCCIÓN

Una computadora digital es una máquina que puede resolver problemas ejecutando las instrucciones que recibe de las personas. Una secuencia de instrucciones que describe cómo realizar cierta tarea se llama **programa**. Los circuitos electrónicos de una computadora pueden reconocer y ejecutar directamente un conjunto limitado de instrucciones sencillas, y todos los programas tienen que convertirse en una serie de esas instrucciones para que la computadora pueda ejecutarlos. Dichas instrucciones básicas casi nunca son más complicadas que

Sumar dos números.

Verificar si un número es cero.

Copiar un dato de una parte de la memoria de la computadora a otra.

Juntas, las instrucciones primitivas de una computadora constituyen un lenguaje que permite a las personas comunicarse con la computadora. Dicho lenguaje se llama **lenguaje de máquina**. Las personas que diseñan una computadora nueva deben decidir qué instrucciones incluirán en su lenguaje de máquina. Por lo regular, se trata de hacer las instrucciones primitivas lo más simples posible, en congruencia con el uso que se piensa dar a la computadora y sus requisitos de desempeño, a fin de reducir la complejidad y el costo de los circuitos requeridos. Casi todos los lenguajes de máquina son tan simples que para las personas resulta difícil y tedioso usarlos.

Con el paso de los años, esta sencilla observación ha dado pie a que las computadoras se estructuren como una serie de abstracciones, donde cada una de éstas se apoya en la que está

abajo de ella. De este modo es posible controlar la complejidad y diseñar sistemas de cómputo de manera sistemática y organizada. Llamamos a este enfoque **organización estructurada de computadoras** y éste es también el nombre que hemos dado al presente libro. En la sección que sigue describiremos el significado de este término. Después veremos un poco de historia, hablaremos de lo último en organización de computadoras, y daremos algunos ejemplos importantes.

1.1 ORGANIZACIÓN ESTRUCTURADA DE COMPUTADORAS

Como ya se mencionó, hay una gran diferencia entre lo que es cómodo para las personas y lo que es cómodo para las computadoras. Las personas quieren hacer *X*, pero las computadoras sólo pueden hacer *Y*. Esto crea un problema. El objetivo de este libro es explicar cómo puede resolverse ese problema.

1.1.1 Lenguajes, niveles y máquinas virtuales

Se puede atacar el problema de dos maneras; ambas implican diseñar un nuevo conjunto de instrucciones que para las personas sea más fácil de usar que el conjunto de instrucciones de máquina original. Juntas, estas nuevas instrucciones también forman un lenguaje, que llamaremos *L1*, así como las instrucciones de máquina originales forman un lenguaje, que nombraremos *L0*. Las dos estrategias difieren en la forma en que la computadora ejecuta los programas escritos en *L1* ya que, como dijimos, la computadora sólo puede ejecutar programas escritos en su lenguaje de máquina, *L0*.

Un método de ejecutar un programa escrito en *L1* es sustituir primero cada instrucción escrita en *L1* por una sucesión equivalente de instrucciones en *L0*. El programa resultante consiste exclusivamente en instrucciones de *L0*. Luego, la computadora ejecuta el nuevo programa en *L0* en lugar del antiguo programa en *L1*. Esta técnica se llama **traducción**.

La otra técnica consiste en escribir un programa en *L0* que tome programas en *L1* como datos de entrada y los ejecute examinando sus instrucciones una por una y ejecutando directamente la sucesión de instrucciones en *L0* que equivale a cada una. Con esta técnica, no es necesario generar primero un nuevo programa en *L0*. La técnica se conoce con el nombre de **interpretación** y el programa que la implementa se denomina **intérprete**.

La traducción y la interpretación son similares. Con ambos métodos las instrucciones en *L1* se ejecutan finalmente realizando sucesiones equivalentes de instrucciones en *L0*. La diferencia es que, con la traducción, todo el programa en *L1* se convierte primero en un programa en *L0*, el programa en *L1* se desecha, el nuevo programa en *L0* se carga en la memoria de la computadora, y se ejecuta. Durante la ejecución, el programa en *L0* recién generado es el que se ejecuta y controla la computadora.

Con la interpretación, después de que cada instrucción en *L1* se examina y se decodifica, se ejecuta inmediatamente. No se genera ningún programa traducido. Aquí es el intérprete el que controla la computadora. Para él, el programa en *L1* no es más que datos. Ambos métodos se usan ampliamente, y cada vez se usa más una combinación de los dos.

En lugar de pensar en términos de traducción o interpretación, a menudo es más fácil imaginar la existencia de una computadora hipotética o **máquina virtual** cuyo lenguaje de máquina es *L1*. Llamemos a esta máquina virtual *M1* (y sea *M0* la máquina virtual que corresponde a *L0*). Si fuera posible construir tal máquina a un costo razonable, no habría necesidad de tener *L1* ni una máquina que ejecutara programas en *L1*. La gente simplemente escribiría sus programas en *L1* y la computadora los ejecutaría directamente. Incluso si es demasiado costoso o complicado construir con circuitos electrónicos la máquina virtual cuyo lenguaje es *L1*, es posible escribir programas para ella. Tales programas podrían interpretarse o traducirse con un programa escrito en *L1* que la computadora existente puede ejecutar directamente. En otras palabras, las personas pueden escribir programas para las máquinas virtuales como si realmente existieran.

Para que la traducción o interpretación sea práctica, los lenguajes *L0* y *L1* no deben ser “demasiado” diferentes. Esta restricción a menudo implica que *L1*, aunque mejor que *L0*, todavía dista mucho de ser ideal para la generalidad de las aplicaciones. Este resultado podría ser decepcionante si pensamos en que el propósito original de crear *L1* era evitar que el programador tuviera que expresar algoritmos en un lenguaje más apropiado para las máquinas que para las personas. No obstante, la situación no es desesperada.

La estrategia obvia es inventar un tercer conjunto de instrucciones que esté más orientado hacia las personas y menos orientado hacia la máquina que *L1*. Esas instrucciones también constituyen un lenguaje, al que llamaremos *L2* (con una máquina virtual *M2*). Las personas pueden escribir programas en *L2* como si en realidad existiera una máquina virtual *M2* cuyo lenguaje de máquina es *L2*. Esos programas podrían traducirse a *L1* o ser ejecutados por un intérprete escrito en *L1*.

La invención de una serie de lenguajes, cada uno más cómodo que sus predecesores, puede continuar indefinidamente hasta llegar a uno adecuado. Cada lenguaje se basa en su predecesor, por lo que podemos pensar en una computadora que emplea esta técnica como una serie de **capas** o **niveles**, uno encima del otro, como se muestra en la figura 1-1. El lenguaje o nivel más bajo es el más simple, y el lenguaje o nivel más alto es el más sofisticado.

Existe una relación importante entre un lenguaje y una máquina virtual. Cada máquina tiene cierto lenguaje de máquina, que consiste en todas las instrucciones que la máquina puede ejecutar. Efectivamente, una máquina define un lenguaje. De igual modo, un lenguaje define una máquina, la máquina que puede ejecutar todos los programas escritos en ese lenguaje. Desde luego, la máquina definida por un lenguaje dado podría ser enormemente complicada y demasiado costosa para construirse directamente con circuitos electrónicos, pero eso no quiere decir que no podamos imaginarla. Una máquina cuyo lenguaje de máquina es C++ o COBOL sería en verdad compleja, pero con la tecnología actual sería fácil construirla. No obstante, hay razones de peso para no construir semejante computadora: no sería económica en comparación con otras técnicas.

Una computadora con *n* niveles puede verse como *n* máquinas virtuales distintas, cada una con diferente lenguaje de máquina. Usaremos los términos “nivel” y “máquina virtual” indistintamente. Los circuitos electrónicos sólo pueden ejecutar directamente programas escritos en el lenguaje *L0* sin necesidad de traducción ni interpretación. Los programas escritos

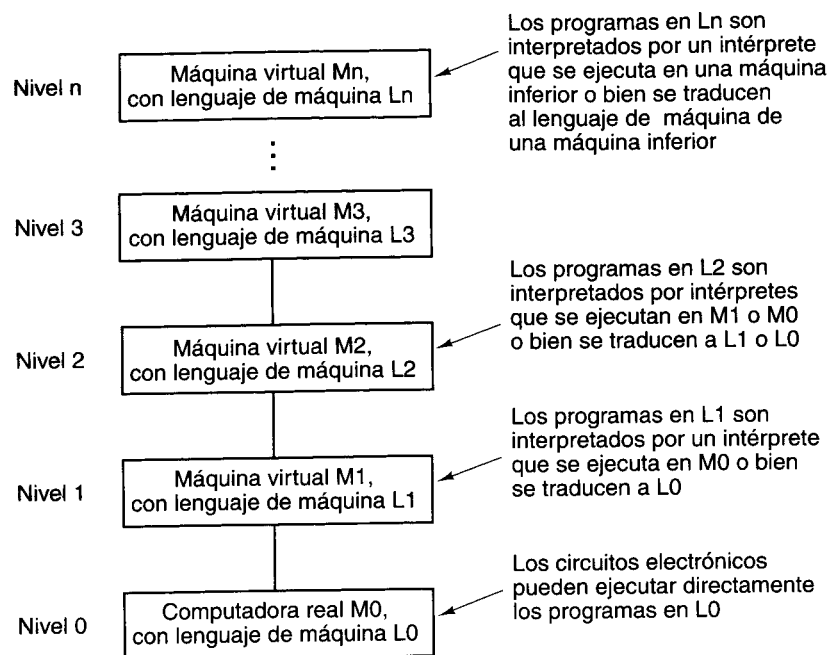


Figura 1-1. Una máquina multinivel.

en $L_1, L_2, \dots, L_n$ deberán ser interpretados por un intérprete que se ejecute en un nivel más bajo, o traducirse a otro lenguaje correspondiente a un nivel más bajo.

La persona encargada de escribir programas para la máquina virtual del nivel n no tiene que estar consciente de los intérpretes y traductores subyacentes. La estructura de la máquina asegura que de un modo u otro esos programas se ejecutarán. No interesa si un intérprete los ejecuta paso por paso, o si ese intérprete es a su vez ejecutado por otro intérprete o por los circuitos electrónicos. El resultado es el mismo en ambos casos: los programas se ejecutan.

En general, a los programadores que usan una máquina de nivel n sólo les interesa el nivel más alto, el que menos se parece al lenguaje de máquina que está hasta abajo. En cambio, las personas interesadas en entender cómo funciona realmente una computadora deben estudiarla en todos los niveles. Quienes se interesen en diseñar nuevas computadoras o nuevos niveles (o sea, nuevas máquinas virtuales) también deberán familiarizarse con niveles distintos del más alto. Los conceptos y técnicas para construir máquinas como una serie de niveles, y los detalles de los niveles mismos, constituyen el tema principal de este libro.

1.1.2 Máquinas multinivel contemporáneas

Casi todas las computadoras modernas constan de dos o más niveles, y pueden llegar a existir máquinas con hasta seis niveles, como se muestra en la figura 1-2. El nivel 0, en la base, es el

verdadero hardware de la máquina. Sus circuitos ejecutan los programas en lenguaje de máquina de nivel 1. Con ánimo totalizador, deberíamos mencionar la existencia de un nivel más bajo de nuestro nivel 0. Este nivel, que no se muestra en la figura 1-2 porque queda dentro del ámbito de la ingeniería eléctrica (y rebasa el alcance de este libro), se llama **nivel de dispositivos**. En este nivel, el diseñador ve transistores individuales, que son las primitivas de más bajo nivel para los diseñadores de computadoras. Si nos preguntamos cómo funcionan internamente los transistores, entramos en la física del estado sólido.

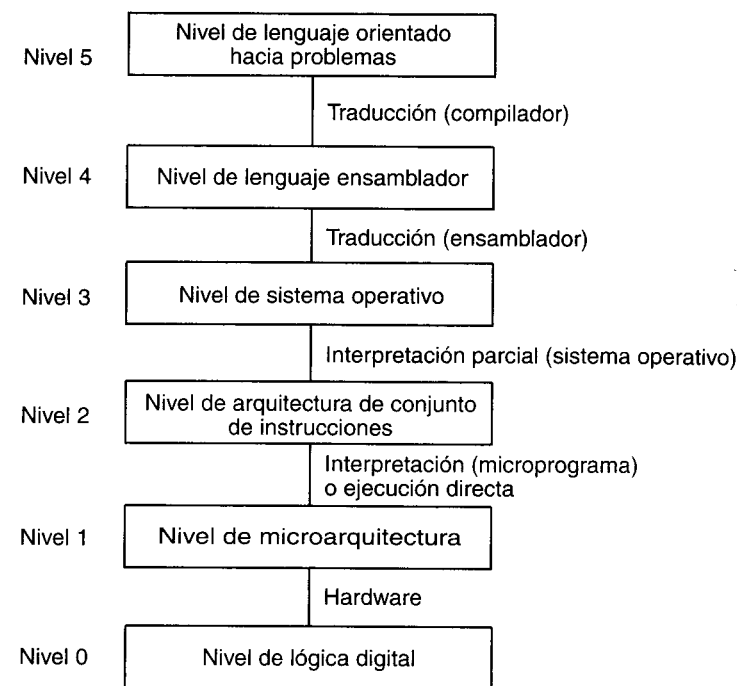


Figura 1-2. Computadora de seis niveles. El método de apoyo para cada nivel se indica inmediatamente abajo de él (junto con el nombre del programa de apoyo).

En el nivel más bajo que estudiaremos, el **nivel de lógica digital**, los objetos integrantes se llaman **compuertas**. Aunque se construyen con componentes analógicos, como los transistores, las compuertas pueden modelarse con exactitud como dispositivos digitales. Cada compuerta tiene una o más entradas digitales (señales que representan 0 o 1) y para generar su salida calcula alguna función sencilla de dichas entradas, como AND u OR. Cada compuerta contiene unos cuantos transistores como máximo. Podemos combinar pocas compuertas para formar una memoria de un bit, capaz de almacenar un 0 o un 1. Las memorias de un bit pueden combinarse en grupos de (por ejemplo) 16, 32 o 64 para formar registros. Cada **registro** puede contener un solo número binario menor que cierto valor límite. Las compuertas también pueden combinarse para formar la máquina calculadora principal misma. Examinaremos las compuertas y el nivel de lógica digital con detalle en el capítulo 3.

El siguiente nivel hacia arriba es el **nivel de microarquitectura**. En este nivel vemos una colección de (típicamente) 8 a 32 registros que forman una memoria local y un circuito llamado **ALU (unidad de aritmética lógica, Arithmetic Logic Unit)** que puede efectuar operaciones aritméticas sencillas. Los registros se conectan a la ALU para formar una **trayectoria de datos** por donde fluyen los datos. La operación básica de la trayectoria de datos consiste en seleccionar uno o dos registros, hacer que la ALU opere con ellos (sumándolos, por ejemplo), y almacenar después el resultado en algún registro.

En algunas máquinas un programa llamado **microprograma** controla la operación de la trayectoria de datos. En otras máquinas la trayectoria de datos está bajo el control directo del hardware. En ediciones anteriores de este libro llamamos a este nivel el “nivel de microprogramación” porque en el pasado casi siempre se trataba de un intérprete en software. Puesto que ahora es común que el hardware controle directamente la trayectoria de datos, hemos cambiado el nombre para que refleje este hecho.

En las máquinas con control por software de la trayectoria de datos, el microprograma es un intérprete de las instrucciones en el nivel 2: obtiene, examina y ejecuta las instrucciones una por una, utilizando la trayectoria de datos para hacerlo. Por ejemplo, para una instrucción **ADD** (sumar), se obtendría la instrucción, se localizarían sus operandos y se colocarían en registros, la ALU calcularía la suma, y por último el resultado se enviaría al lugar correcto. En una máquina con control por hardware de la trayectoria de datos, se llevarían a cabo pasos similares, pero sin un programa almacenado explícito que controle la interpretación de las instrucciones del nivel 2.

En el nivel 2 tenemos un nivel que llamaremos **nivel de arquitectura del conjunto de instrucciones (Instruction Set Architecture)** o **nivel ISA**. Cada fabricante de computadoras publica un manual para cada una de las computadoras que vende, intitulado “Manual de referencia del lenguaje de máquina” o “Principios de operación de la computadora Western Wombat Model 100X” o algo parecido. Estos manuales se ocupan realmente del nivel ISA, no de los niveles subyacentes. Cuando describen el conjunto de instrucciones de la máquina, estos manuales están describiendo realmente las instrucciones que el microprograma o los circuitos de ejecución en hardware ejecutan de forma interpretativa. Si un fabricante de computadoras incluye dos intérpretes en una de sus máquinas, para interpretar dos niveles ISA distintos, tendrá que proporcionar dos manuales de referencia del “lenguaje de máquina”, uno para cada intérprete.

El siguiente nivel suele ser un nivel híbrido. Casi todas las instrucciones de su lenguaje están también en el nivel ISA. (No hay razón para que una instrucción que aparece en un nivel no pueda estar presente también en otros niveles.) Además, hay un nuevo conjunto de instrucciones, una diferente organización de memoria, la capacidad para ejecutar dos o más programas al mismo tiempo, y varias características más. Entre los distintos diseños de nivel 3 hay más variación que entre los de los niveles 1 o 2.

Las nuevas funciones que se añaden en el nivel 3 son desempeñadas por un intérprete que se ejecuta en el nivel 2, que históricamente se conoce como sistema operativo. El microprograma (o el control por hardware), no el sistema operativo, ejecuta directamente las instrucciones del nivel 3 que son idénticas a las del nivel 2. En otras palabras, algunas de las instrucciones del nivel 3 son interpretadas por el sistema operativo y otras son interpretadas directamente por el microprograma. A esto es a lo que nos referimos con el término “híbrido”. Llamamos a este nivel el **nivel de máquina del sistema operativo**.

Existe una discontinuidad fundamental entre los niveles 3 y 4. Los tres niveles más bajos no están diseñados para que sean usados por un programador ordinario. Su propósito primordial es la ejecución de los intérpretes y traductores que se necesitan para apoyar a los niveles superiores. Estos intérpretes y traductores son escritos por personas llamadas **programadores de sistemas** que se especializan en el diseño e implementación de máquinas virtuales nuevas. Los niveles 4 y superiores pertenecen a los programadores de aplicaciones que tienen un problema que resolver.

Otro cambio que ocurre en el nivel 4 tiene que ver con el método de apoyo de los niveles superiores. Los niveles 2 y 3 siempre se interpretan. Los niveles del 4 en adelante por lo regular se traducen, aunque no siempre es así.

Una diferencia más entre los niveles 1, 2 y 3, por un lado, y los niveles del 4 en adelante, por el otro, es la naturaleza del lenguaje empleado. Los lenguajes de máquina de los niveles 1, 2 y 3 son numéricos. Los programas escritos en ellos constan de largas series de números, lo cual es magnífico para las máquinas pero malo para las personas. A partir del nivel 4, los lenguajes contienen palabras y abreviaturas que tienen un significado para las personas.

El nivel 4, el nivel de lenguaje ensamblador, es en realidad una forma simbólica de uno de los lenguajes subyacentes. Este nivel ofrece a las personas un método de escribir programas para los niveles 1, 2 y 3 en una forma no tan incomprensible como los lenguajes de máquina virtuales. Los programas en lenguaje ensamblador primero se traducen a un lenguaje de nivel 1, 2 o 3 y luego se interpretan por la máquina virtual o real apropiada. El programa que realiza la traducción se llama **ensamblador**.

El nivel 5 por lo regular consta de lenguajes diseñados para ser usados por programadores de aplicaciones que quieren resolver problemas. Tales lenguajes suelen llamarse **lenguajes de alto nivel**, y hay literalmente cientos de ellos. Entre los más conocidos están BASIC, C, C++, Java, LISP y Prolog. Los programas escritos en estos lenguajes generalmente se traducen a lenguajes de nivel 3 o 4 con traductores llamados **compiladores**, aunque ocasionalmente se interpretan en vez de traducirse. Los programas en Java, por ejemplo, a menudo se interpretan.

En algunos casos, el nivel 5 consiste en un intérprete para un dominio de aplicación específico, como las matemáticas simbólicas. El nivel proporciona datos y operaciones para resolver problemas de este dominio en términos que la gente que conoce el dominio puede entender fácilmente.

En síntesis, lo más importante que debemos recordar es que las computadoras se diseñan como una serie de niveles, cada uno construido sobre sus predecesores. Cada nivel representa una abstracción distinta, y contiene diferentes objetos y operaciones. Al diseñar y analizar las computadoras de esta manera, podemos suprimir temporalmente los detalles que no son pertinentes y así reducir un tema complejo a algo más fácil de entender.

El conjunto de tipos de datos, operaciones y características de cada nivel es su **arquitectura**. La arquitectura se ocupa de los aspectos que el usuario de ese nivel puede ver. Las características que el programador ve, como la cantidad de memoria disponible, forman parte de la arquitectura. Los aspectos de implementación, como el tipo de tecnología de chips empleado para implementar la memoria, no forman parte de la arquitectura. El estudio del diseño de las partes de un sistema de cómputo que los programadores pueden ver se llama **arquitectura de computadoras**. En la práctica común, la arquitectura de las computadoras y la organización de las computadoras significan prácticamente lo mismo.

1.1.3 Evolución de las máquinas multinivel

Para entender mejor las máquinas multinivel, examinaremos brevemente su desarrollo histórico, mostrando cómo han evolucionado al paso de los años el número y la naturaleza de los niveles. Los programas escritos en el verdadero lenguaje de máquina de una computadora (nivel 1) pueden ser ejecutados directamente por los circuitos electrónicos de la máquina (nivel 0), sin intervención de intérpretes ni traductores. Estos circuitos electrónicos, junto con la memoria y los dispositivos de entrada/salida, constituyen el **hardware** de la computadora. El hardware consta de objetos tangibles —circuitos integrados, tarjetas de circuitos impresos, cables, fuentes de alimentación, memorias e impresoras— no de ideas abstractas, algoritmos o instrucciones.

El **software**, en cambio, consta de **algoritmos** (instrucciones detalladas que indican cómo hacer algo) y sus representaciones en la computadora: los programas. Los programas se pueden almacenar en un disco duro, disco flexible, CD-ROM u otro medio, pero la esencia del software es la serie de instrucciones que constituye los programas, no los medios físicos en los que se registran.

En las primeras computadoras, la frontera entre hardware y software era muy nítida, pero con el tiempo se ha vuelto mucho más borrosa, principalmente por la adición, eliminación y fusión de niveles a medida que las computadoras evolucionaban. Hoy día, a veces es difícil distinguirlos. De hecho, un tema central de este libro es

El hardware y el software son lógicamente equivalentes.

Cualquier operación realizada por software también puede incorporarse directamente en hardware, de preferencia una vez que se ha entendido muy bien. Como dijo Karen Panetta Lenz: “El hardware no es más que software petrificado.” Desde luego, lo contrario también es cierto: cualquier ejecución realizada por el hardware también puede simularse en software. La decisión de colocar ciertas funciones en hardware y otras en software se basa en factores como costo, rapidez, confiabilidad y frecuencia de cambios previstos. Existen pocas reglas rígidas para determinar si X debe incluirse en el hardware y Y debe programarse explícitamente. Tales decisiones cambian con las tendencias en la tecnología y el uso de las computadoras.

La invención de la microprogramación

Las primeras computadoras digitales, en los años cuarenta, sólo tenían dos niveles: el ISA, en el que se efectuaba toda la programación, y el de lógica digital, que ejecutaba esos programas. Los circuitos del nivel de lógica digital eran complicados, difíciles de entender y construir, y poco confiables.

En 1951 Maurice Wilkes, investigador de la Universidad de Cambridge, sugirió la idea de diseñar una computadora de tres niveles a fin de simplificar drásticamente el hardware (Wilkes, 1951). Esta máquina tendría un intérprete inmutable integrado (el microprograma) cuya función sería ejecutar programas del nivel ISA por interpretación. Puesto que el hardware ahora sólo tendría que ejecutar microprogramas, que tienen un conjunto limitado de instruc-

ciones, en lugar de programas del nivel ISA, que tienen un conjunto de instrucciones mucho más grande, se requerirían menos circuitos electrónicos. Dado que en esa época los circuitos se construían con bulbos, tal simplificación prometía reducir el número de bulbos y por tanto mejorar la confiabilidad.

Durante los cincuenta se construyeron unas cuantas máquinas de tres niveles. En los años sesenta se construyeron más, y para 1970 la idea de interpretar el nivel ISA con un microprograma, y no directamente con los circuitos, era preponderante.

La invención del sistema operativo

En estos primeros años, la mayor parte de las computadoras eran de “taller abierto”, lo que significa que el programador tenía que operar la máquina personalmente. Junto a cada máquina había una hoja para reservar. Un programador que quería ejecutar un programa reservaba cierto tiempo, digamos el miércoles de las 3 A.M. a las 5 A.M. (a muchos programadores les gustaba trabajar cuando el cuarto de máquinas estaba tranquilo). Al llegar la hora, el programador se dirigía al cuarto de máquinas con un paquete de tarjetas perforadas de 80 columnas (un medio de entrada primitivo) en una mano y un lápiz afilado en la otra. Al llegar al cuarto de la computadora, conducía amablemente al programador anterior hacia la puerta y se hacía cargo de la computadora.

Si el programador quería ejecutar un programa en FORTRAN, era necesario realizar los pasos siguientes:

1. Él† se dirigía al mueble en el que se guardaba la biblioteca de programas, sacaba el gran paquete verde rotulado “compilador FORTRAN”, lo colocaba en el lector de tarjetas, y oprimía el botón de inicio.
2. El programador colocaba el programa en FORTRAN en el lector de tarjetas y oprimía el botón de continuar. Se leía el programa.
3. Una vez que la computadora se detenía, él hacía que se leyera otra vez el programa en FORTRAN. Aunque algunos compiladores sólo requerían leer una vez sus entradas, muchos requerían varias pasadas. En cada pasada era necesario leer un paquete grande de tarjetas.
4. Por último, se llegaba a la etapa final de la traducción. El programador a menudo se ponía nervioso a estas alturas porque si el compilador encontraba un error en el programa, el programador tenía que corregirlo e iniciar otra vez el proceso. Si no había errores, el compilador perforaba el programa traducido a lenguaje de máquina en otras tarjetas.
5. Luego, el programador colocaba el programa en lenguaje de máquina en el lector de tarjetas junto con el paquete de la biblioteca de subrutinas para que se leyeran.

†“Él” debe leerse como “él o ella” en todo el libro.

6. El programa comenzaba a ejecutarse. Las más de las veces, no funcionaba y se detenía inesperadamente antes de terminar. En general, el programador movía los interruptores de la consola y miraba las luces de la consola durante un rato. Si tenía suerte, encontraba el problema, corregía el error, y regresaba al mueble que contenía el gran paquete verde del compilador FORTRAN para comenzar otra vez desde el principio. Si no tenía suerte, preparaba un listado del contenido de la memoria, llamado **vaciado de núcleo** (*core dump*) y se lo llevaba a casa para estudiarlo.

Este procedimiento, con variaciones menores, fue normal en muchos centros de cómputo durante años; obligaba a los programadores a aprender a operar la máquina y a saber qué hacer cuando tenía un desperfecto, que era muy seguido. La máquina pasaba mucho tiempo ociosa mientras la gente llevaba tarjetas de un lado a otro del recinto o se rascaba la cabeza tratando de averiguar por qué sus programas no estaban funcionando correctamente.

Alrededor de 1960 la gente trató de reducir el desperdicio de tiempo automatizando la tarea del operador. Un programa llamado **sistema operativo** estaba cargado en la computadora todo el tiempo. El programador incluía ciertas tarjetas de control junto con el programa, y el sistema operativo las leía y ejecutaba sus instrucciones. En la figura 1-3 se muestra un ejemplo de paquete de tarjetas para uno de los primeros sistemas operativos de mayor uso, FMS (FORTRAN Monitor System), en la IBM 709.

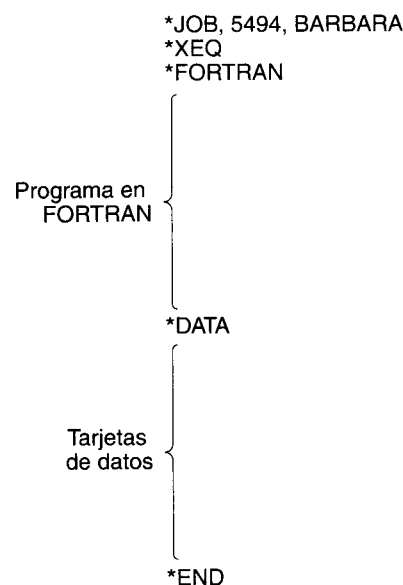


Figura 1-3. Ejemplo de trabajo para el sistema operativo FMS.

El sistema operativo leía la tarjeta *JOB y utilizaba la información que contenía para fines de contabilidad. (El asterisco identificaba las tarjetas de control, para que la computado-

ra no las confundiera con las tarjetas del programa y de datos.) Después, el sistema operativo leía la tarjeta *FORTRAN, que era una instrucción para cargar el compilador FORTRAN de una cinta magnética. Luego, el compilador leía y compilaba el programa en FORTRAN. Cuando el compilador terminaba, devolvía el control al sistema operativo, que entonces leía la tarjeta *DATA. Ésta era una instrucción para ejecutar el programa traducido, utilizando las tarjetas que seguían a la tarjeta *DATA como datos.

Aunque el sistema operativo se diseñó para automatizar la tarea del operador (de ahí el nombre), también fue el primer paso en la creación de una nueva máquina virtual. La tarjeta *FORTRAN podía verse como una instrucción “compilar programa” virtual. Así mismo, la tarjeta *DATA podía considerarse como una instrucción “ejecutar programa” virtual. Un nivel con sólo dos instrucciones no era mucho, pero era un paso en esa dirección.

En años subsecuentes, los sistemas operativos se volvieron más y más sofisticados. Se añadieron nuevas instrucciones, recursos y características al nivel ISA hasta que comenzó a parecer un nuevo nivel. Algunas de las instrucciones de este nuevo nivel eran idénticas a las del nivel ISA, pero otras, sobre todo las de entrada/salida, eran totalmente distintas. Las nuevas instrucciones se conocían como **macros del sistema operativo** o **llamadas al supervisor**. El término usual ahora es **llamada al sistema**.

Los sistemas operativos evolucionaron también en otros sentidos. Los primeros leían paquetes de tarjetas e imprimían sus salidas en la impresora en línea. Esta organización se llamaba **sistema por lotes**. Por lo regular, había una espera de varias horas entre el momento en que se presentaba un programa y el momento en que los resultados estaban listos. Desarrollar software era difícil en esas circunstancias.

A principios de los años sesenta investigadores del Dartmouth College, MIT, y otros sitios crearon sistemas operativos que permitían a (varios) programadores comunicarse directamente con la computadora. En esos sistemas, terminales remotas se conectaban a la computadora central a través de líneas telefónicas. Muchos usuarios compartían el mismo CPU. Un programador podía introducir un programa con el teclado y obtener los resultados impresos casi de inmediato en su oficina, en la cochera de su casa o en donde fuera que se instalara la terminal. Esos sistemas se llamaban, y se siguen llamando, **sistemas de tiempo compartido**.

Nuestro interés en los sistemas operativos se centra en las partes que interpretan las instrucciones y en las características que están presentes en el nivel 3 pero no en el nivel ISA, más que en los aspectos de tiempo compartido. Aunque no haremos hincapié en ello, el lector debe tener presente que los sistemas operativos hacen más que sólo interpretar funciones añadidas al nivel ISA.

Migración de la funcionalidad al microcódigo

Una vez que se hizo común la microprogramación (hacia 1970), los diseñadores se dieron cuenta de que podían añadir nuevas instrucciones con sólo extender el microprograma. En otras palabras, podían agregar “hardware” (nuevas instrucciones de máquina) programando. Esta revelación dio pie a una verdadera explosión en los conjuntos de instrucciones de máquina, a medida que los diseñadores trataban de producir conjuntos de instrucciones más grandes y

mejores que los de otros diseñadores. Muchas de esas instrucciones no eran indispensables en el sentido de que su efecto podía lograrse fácilmente con las instrucciones existentes, pero a menudo eran un poco más rápidas que una sucesión de instrucciones existentes. Por ejemplo, muchas instrucciones tenían una instrucción **INC** (INCRementar) que sumaba 1 a un número. Puesto que esas máquinas también tenían una instrucción general **ADD** para sumar, tener una instrucción especial para sumar 1 (o para sumar 270, para el caso) no era necesario. Sin embargo, **INC** era por lo regular un poco más rápida que **ADD**, así que se incluyó.

Se añadieron muchas otras instrucciones al microprograma siguiendo el mismo razonamiento. Entre ellas estaban

1. Instrucciones para multiplicación y división de enteros.
2. Instrucciones para aritmética de punto flotante.
3. Instrucciones para invocar procedimientos y regresar de ellos.
4. Instrucciones para acelerar los ciclos.
5. Instrucciones para manejar cadenas de caracteres.

Además, una vez que los diseñadores de máquinas vieron lo fácil que era añadir nuevas instrucciones, comenzaron a buscar otras funciones que añadir a sus microprogramas. Ejemplos de tales adiciones son

1. Funciones para acelerar cálculos con matrices (direccionamiento indexado e indirecto).
2. Funciones para trasladar programas de un lugar a otro de la memoria después de que han iniciado su ejecución (recursos de reubicación).
3. Sistemas de interrupciones que avisan a la computadora cuando se termina una operación de entrada o salida.
4. La capacidad para suspender un programa e iniciar otro en un número reducido de instrucciones (conmutación de procesos).

Con los años, se han añadido muchas otras funciones y recursos, casi siempre para acelerar alguna actividad específica.

La eliminación de la microprogramación

Los microprogramas crecieron durante los años dorados de la microprogramación (años sesenta y setenta). El único problema fue que también tendían a volverse más y más lentos a medida que se hacían más voluminosos. Por fin algunos investigadores se dieron cuenta de que si eliminaban el microprograma, reducían considerablemente el conjunto de instrucciones y hacían que las instrucciones restantes se ejecutaran en forma directa (es decir, control por hardware de la trayectoria de datos), las máquinas podían acelerarse. En cierto sentido, el

diseño de computadoras había descrito un círculo completo, volviendo a la forma que tenía antes de que Wilkes inventara la microprogramación.

El punto importante de esta explicación es mostrar que la frontera entre hardware y software es arbitraria y cambia constantemente. El software de hoy podría ser el hardware de mañana, y viceversa. Además, las fronteras entre los diversos niveles también son variables. Desde el punto de vista del programador, la forma en que se implementa una instrucción dada no es importante (excepto tal vez por su velocidad). Una persona que programa en el nivel ISA puede utilizar su instrucción de multiplicar como si fuera una instrucción de hardware sin preocuparse por ello, o sin siquiera saber si en efecto es una instrucción de hardware. El hardware de una persona es el software de otra. Regresaremos a estos temas más adelante.

1.2 ACONTECIMIENTOS IMPORTANTES EN ARQUITECTURA DE COMPUTADORAS

Se han diseñado y construido cientos de tipos distintos de computadoras durante la evolución de la computadora digital moderna. Casi todos se han perdido en el olvido, pero unos cuantos han tenido un impacto importante sobre las ideas modernas. En esta sección bosquejaremos brevemente algunos de los sucesos históricos clave para entender mejor cómo llegamos a donde estamos ahora. No es necesario decir que esta sección sólo trata los puntos principales y deja muchas cosas sin mencionar. En la figura 1-4 se lee una lista de algunas de las máquinas que trataremos en esta sección. Slater (1987) presenta material histórico adicional acerca de las personas que inauguraron la era de las computadoras. En la obra de Morgan (1997), se encuentran biografías breves, ilustradas con bellas fotografías a color de esos fundadores, tomadas por Louis Fabian Bachrach. Otro libro de biografías es (Slater, 1987).

1.2.1 La generación cero – computadoras mecánicas (1642-1945)

La primera persona que construyó una máquina calculadora funcional fue el científico francés Blaise Pascal (1623-1662), en cuyo honor se nombró el lenguaje de programación Pascal. Este dispositivo, construido en 1642, cuando Pascal tenía apenas 19 años, fue diseñado para ayudar a su padre, un cobrador de impuestos del gobierno francés. La calculadora era totalmente mecánica, con engranes, y se impulsaba con una manivela operada a mano.

La máquina de Pascal sólo podía sumar y restar, pero 30 años después el gran matemático alemán Barón Gottfried Wilhelm von Leibniz (1646-1716) construyó otra máquina mecánica que también podía multiplicar y dividir. Efectivamente, Leibniz había construido el equivalente de una calculadora de bolsillo de cuatro funciones, hace tres siglos.

No sucedió algo relevante durante 150 años hasta que un profesor de matemáticas de la University of Cambridge, Charles Babbage (1792-1871), inventor del velocímetro, diseñó y construyó su **máquina de diferencias**. Este aparato mecánico, que al igual que el de Pascal sólo podía sumar y restar, fue diseñado para calcular tablas de números útiles para la navegación marítima. La máquina entera estaba diseñada para ejecutar un solo algoritmo, el método

Año	Nombre	Hecho por	Comentario
1834	Máquina analítica	Babbage	Primer intento de construir una computadora digital
1936	Z1	Zuse	Primera máquina calculadora de relevadores funcional
1943	COLOSSUS	Gobierno inglés	Primera computadora electrónica
1944	Mark I	Aiken	Primera computadora estadounidense de propósito general
1946	ENIAC I	Eckert/Mauchley	Inicia la historia moderna de la computación
1949	EDSAC	Wilkes	Primera computadora de programa almacenado
1951	Whirlwind I	M.I.T.	Primera computadora de tiempo lineal
1952	IAS	Von Neumann	Casi todas las máquinas actuales emplean este diseño
1960	PDP-1	DEC	Primera minicomputadora (50 vendidas)
1961	1401	IBM	Máquina pequeña para negocios de enorme popularidad
1962	7094	IBM	Dominó la computación científica a principios de los años sesenta
1963	B5000	Burroughs	Primera máquina diseñada para un lenguaje de alto nivel
1964	360	IBM	Primera línea de productos diseñada como familia
1964	6600	CDC	Primera supercomputadora científica
1965	PDP-8	DEC	Primera minicomputadora con mercado masivo (50,000 vendidas)
1970	PDP-11	DEC	Dominó las minicomputadoras en los años setenta
1974	8080	Intel	Primera computadora de propósito general de 8 bits en un chip
1974	CRAY-1	Cray	Primera supercomputadora vectorial
1978	VAX	DEC	Primera superminicomputadora de 32 bits
1981	IBM PC	IBM	Inició la era de la computadora personal moderna
1985	MIPS	MIPS	Primera máquina RISC comercial
1987	SPARC	Sun	Primera estación de trabajo RISC basada en SPARC
1990	RSC6000	IBM	Primera máquina superescalar

Figura 1-4. Algunos acontecimientos importantes en la evolución de la computadora digital moderna.

de diferencias finitas empleando polinomios. La característica más interesante de la máquina de diferencias era su método de salida: perforaba sus resultados en una placa de cobre para grabados con un troquel de acero, un precursor de posteriores medios de escritura única como las tarjetas perforadas y los CD-ROM.

Aunque la máquina de diferencias funcionaba razonablemente bien, Babbage pronto se aburrió de una máquina que sólo podía ejecutar un algoritmo. Él comenzó a dedicar cantidades cada vez más grandes de su tiempo y de la fortuna de su familia (sin mencionar 17,000 libras esterlinas de dinero del gobierno) en el diseño y construcción de una sucesora llamada **máquina analítica**. La máquina analítica tenía cuatro componentes: el almacén (memoria), el molino (unidad de cómputo), la sección de entrada (lector de tarjetas perforadas) y la

sección de salida (salidas perforadas e impresas). El almacén consistía en 1000 palabras de 50 dígitos decimales, cada una de las cuales servía para almacenar variables y resultados. El molino podía aceptar operandos del almacén, y luego sumarlos, restarlos, multiplicarlos o dividirlos, y por último devolver el resultado al almacén. Al igual que la máquina de diferencias, la máquina analítica era totalmente mecánica.

El gran avance de la máquina analítica fue su utilidad general: leía instrucciones de tarjetas perforadas y las ejecutaba. Algunas instrucciones ordenaban a la máquina obtener dos números del almacén, llevarlos al molino, efectuar operaciones sobre ellos (por ejemplo, sumarlos) y enviar el resultado de vuelta al almacén. Otras instrucciones podían probar un número y efectuar una ramificación condicional dependiendo de si era positivo o negativo. Al perforar un programa distinto en las tarjetas de entrada, era posible lograr que la máquina analítica realizara diferentes cálculos, algo que no podía hacer la máquina de diferencias.

Puesto que la máquina analítica era programable en un sencillo lenguaje ensamblador, necesitaba software. Para producir este software, Babbage contrató a una joven mujer llamada Ada Augusta Lovelace, hija del afamado poeta británico Lord Byron. Así, Ada Lovelace fue la primera programadora de computadoras del mundo. El moderno lenguaje de programación Ada® honra su memoria.

Desafortunadamente, al igual que muchos diseñadores modernos, Babbage nunca logró eliminar todos los problemas del hardware. La dificultad es que necesitaba miles y miles de levas y engranes y ruedas fabricadas con un grado de precisión que la tecnología del siglo XIX no podía alcanzar. No obstante, sus ideas se habían adelantado mucho a su época, e incluso hoy la mayor parte de las computadoras modernas tienen una estructura muy similar a la de la máquina analítica, por lo que es justo decir que Babbage fue el padre (o el abuelo) de la computadora digital moderna.

El siguiente avance importante ocurrió a fines de la década de los treinta, cuando un estudiante de ingeniería alemán llamado Konrad Zuse construyó una serie de máquinas calculadoras automáticas empleando relevadores electromagnéticos. Zuse no logró obtener financiamiento del gobierno después de iniciada la guerra porque los burócratas esperaban ganar la guerra con tal rapidez que la nueva máquina no estaría lista antes de terminar las acciones. Zuse no conocía el trabajo de Babbage, y sus máquinas fueron destruidas por el bombardeo aliado de Berlín en 1944, por lo que su trabajo no influyó en las máquinas subsiguientes. No obstante, Zuse fue uno de los pioneros en este campo.

Poco tiempo después, en Estados Unidos, dos personas diseñaron también calculadoras: John Atanasoff del Iowa State College y George Stibitz de Bell Labs. La máquina de Atanasoff era asombrosamente avanzada para su época; utilizaba aritmética binaria y tenía una memoria de condensadores, los cuales se renovaban periódicamente para evitar que la carga desapareciera, en un proceso que él llamaba “refrescar la memoria”. Los modernos chips de memoria dinámica (RAM) funcionan exactamente de la misma manera. Por desgracia, la máquina nunca logró entrar en operación. En cierto modo, Atanasoff era como Babbage: un visionario que finalmente fue derrotado por la insuficiente tecnología de hardware de su época.

La computadora de Stibitz, aunque más primitiva que la de Atanasoff, sí funcionaba. Stibitz hizo una demostración pública en una conferencia en Dartmouth College en 1940. Entre el públi-

UNIVERSIDAD DE LA PAZ
FACULTAD DE INGENIERIA
DEPARTAMENTO DE INGENIERIA
DOCUMENTACION Y BIBLIOTECA

co estaba John Mauchley, un profesor de física desconocido de la Universidad de Pennsylvania. El mundo de las computadoras oiría más acerca del profesor Mauchley posteriormente.

Mientras Zuse, Stibitz y Atanasoff diseñaban calculadoras automáticas, un joven llamado Howard Aiken realizaba tediosos cálculos numéricos a mano como parte de su tesis de doctorado en Harvard. Después de graduarse, Aiken reconoció la importancia de poder efectuar cálculos con una máquina. Él acudió a la biblioteca, descubrió los trabajos de Babbage, y decidió construir con relevadores la computadora de propósito general que Babbage no había logrado construir con ruedas dentadas.

La primera máquina de Aiken, el Mark I, se completó en Harvard en 1944. Tenía 72 palabras de 23 dígitos decimales cada una, y un tiempo de instrucción de 6 segundos. Las entradas y salidas se efectuaban mediante una cinta de papel perforada. Para cuando Aiken terminó su sucesor, el Mark II, las computadoras de relevadores eran obsoletas. Se había iniciado la era de la electrónica.

1.2.2 La primera generación – bulbos (1945-1955)

El estímulo para la computadora electrónica fue la Segunda Guerra Mundial. Durante la primera parte de la guerra, submarinos alemanes hacían estragos entre la armada británica. Los almirantes alemanes enviaban órdenes por radio hasta los submarinos, pero eran interceptadas por los ingleses. El problema es que los mensajes se codificaban mediante un aparato llamado **ENIGMA**, cuyo precursor había sido diseñado por un inventor aficionado y expresidente de Estados Unidos, Thomas Jefferson.

A principios de la guerra, los espías británicos lograron adquirir una máquina ENIGMA de los espías polacos, quienes la habían robado a los alemanes. Sin embargo, se requería un número abrumador de cálculos para decodificar un mensaje, y esto tenía que hacerse inmediatamente después de interceptarse el mensaje para que sirviera de algo. Para decodificar estos mensajes, el gobierno británico estableció un laboratorio supersecreto que construyó una computadora electrónica llamada COLOSSUS. El famoso matemático británico Alan Turing ayudó a diseñar esta máquina. COLOSSUS entró en operación en 1943, pero como el gobierno británico mantuvo prácticamente todos los aspectos del proyecto clasificados como secreto militar durante 30 años, la línea de COLOSSUS fue básicamente un callejón sin salida. Sólo vale la pena mencionarlo porque fue la primera computadora electrónica digital del mundo.

Además de destruir las máquinas de Zuse y estimular la construcción de COLOSSUS, la guerra también afectó la computación en Estados Unidos. El ejército necesitaba tablas de alcance para apuntar su artillería pesada, y producía esas tablas contratando a cientos de mujeres que las elaboraban utilizando calculadoras manuales (se pensaba que las mujeres eran más precisas que los hombres). No obstante, el proceso era muy tardado y los errores eran frecuentes.

John Mauchley, quien conocía el trabajo de Atanasoff y de Stibitz, se dio cuenta de que el ejército estaba interesado en calculadoras mecánicas. Al igual que muchos científicos de la computación que le siguieron, Mauchley elaboró una propuesta de subvención en la que pedía al ejército fondos para construir una computadora electrónica. La propuesta fue acepta-

da en 1943, y Mauchley y su estudiante de posgrado, J. Presper Eckert, procedieron a construir una computadora electrónica a la que llamaron **ENIAC** (*Electronic Numerical Integrator And Computer*). ENIAC consistía en 18,000 bulbos y 1500 relevadores, pesaba 30 toneladas y consumía 140 kilowatts de potencia. En términos de arquitectura, la máquina tenía 20 registros, cada uno capaz de almacenar un número decimal de 10 dígitos. (Un registro decimal es una memoria muy pequeña que puede almacenar un número menor que cierto número máximo de dígitos decimales, parecido al odómetro que registra la distancia recorrida por un automóvil.) ENIAC se programaba ajustando 6000 interruptores de multiposición y conectando numerosas bases con una verdadera maraña de cables interconectores.

La máquina no quedó terminada sino hasta 1946, demasiado tarde para realizar su tarea original. Sin embargo, como la guerra había terminado, se les permitió a Mauchley y Eckert organizar un curso de verano para describir el trabajo a sus colegas científicos. Ese curso de verano fue el principio del auge del interés en construir computadoras digitales grandes.

Después de ese histórico curso de verano, muchos otros investigadores se propusieron construir computadoras electrónicas. La primera que entró en operación fue EDSAC (1949), construida por Maurice Wilkes en la Universidad de Cambridge. Otras fueron JOHNIAC, de la Rand Corporation, ILLIAC de la Universidad de Illinois, MANIAC de Los Alamos Laboratory y WEIZAC del Instituto Weizmann de Israel.

Eckert y Mauchley pronto comenzaron a trabajar en un sucesor, **EDVAC** (*Electronic Discrete Variable Automatic Computer*). Sin embargo, ese proyecto quedó condenado al fracaso cuando ellos dejaron la Universidad de Pennsylvania para formar una compañía nueva, la Eckert-Mauchley Computer Corporation, en Filadelfia (todavía no se había creado Silicon Valley). Después de una serie de fusiones, esta compañía se convirtió en la moderna Unisys Corporation.

Como dato curioso, Eckert y Mauchley solicitaron una patente asegurando haber inventado la computadora digital. En retrospectiva, habría sido bueno tener una patente así. Después de años de pleitos, las cortes decidieron que la patente de Eckert y Mauchley no era válida y que John Atanasoff era el inventor de la computadora digital, aunque nunca la haya patentado.

Mientras Eckert y Mauchley estaban trabajando en la EDVAC, una de las personas que participaron en el proyecto ENIAC, John von Neumann, acudió al Institute of Advanced Studies de Princeton para construir su propia versión de EDVAC, la **máquina IAS**. Von Neuman fue un genio del calibre de Leonardo da Vinci; hablaba muchos idiomas, era experto en las ciencias físicas y en matemáticas, y recordaba perfectamente todo lo que había escuchado, visto o leído. Podía citar de memoria el texto exacto de libros que había leído hacía años. Cuando se interesó por las computadoras, ya era el matemático más eminente del mundo.

Una cosa que pronto fue obvia para él era que programar computadoras con un gran número de interruptores era lento, tedioso e inflexible. Von Neumann se dio cuenta de que el programa podía representarse en forma digital en la memoria de la computadora, junto con los datos. Él percibió también que la torpe aritmética decimal en serie utilizada por ENIAC, en la que cada dígito estaba representado por 10 bulbos (uno encendido y nueve apagados) podía ser sustituida por una aritmética binaria.

El diseño básico, que él describió por primera vez, ahora se conoce como **máquina de von Neumann**. Se usó en EDSAC, la primera computadora de programa almacenado, y sigue siendo la base de casi todas las computadoras digitales aun ahora, más de medio siglo después. Este diseño, y la máquina IAS, construida en colaboración con Herman Goldstine, ha tenido una influencia de tal magnitud que vale la pena describirla brevemente. En la figura 1-5 se presenta un bosquejo de su arquitectura.

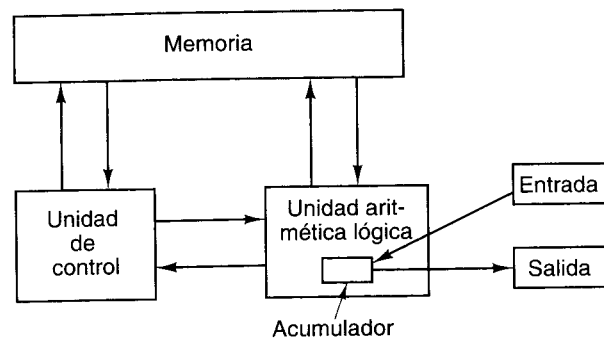


Figura 1-5. La máquina de von Neumann original.

La máquina de von Neumann tenía cinco partes básicas: la memoria, la unidad aritmética lógica, la unidad de control y el equipo de entrada y salida. La memoria constaba de 4096 palabras, cada una de las cuales contenía 40 bits que podían ser 0 o 1. Cada palabra contenía dos instrucciones de 20 bits o bien un entero con signo de 40 bits. Ocho bits de cada instrucción estaban dedicados a indicar el tipo de instrucción, y 12 bits se dedicaban a especificar una de las 4096 palabras de memoria.

Dentro de la unidad de aritmética lógica había un registro interno especial de 40 bits llamado **acumulador**. Una instrucción típica sumaba una palabra de la memoria al acumulador o almacenaba el contenido del acumulador en la memoria. La máquina no tenía aritmética de punto flotante porque von Neumann pensaba que cualquier matemático competente debería poder seguir mentalmente la pista al punto decimal (en realidad el punto binario).

Más o menos en las mismas fechas en que von Neumann estaba construyendo la máquina IAS, investigadores de MIT también estaban construyendo una computadora. A diferencia de IAS, ENIAC y otras máquinas de su clase, que empleaban palabras largas y estaban diseñadas para manipulación pesada de números, la máquina del MIT, Whirlwind I, tenía palabras de 16 bits y estaba diseñada para el control en tiempo real. Este proyecto dio pie a la invención de la memoria de núcleos magnéticos por Jay Forrester, y finalmente a la primera minicomputadora comercial.

Mientras todo esto estaba sucediendo, IBM era una compañía pequeña dedicada a fabricar perforadoras de tarjetas y máquinas clasificadoras de tarjetas. Aunque IBM había aportado parte del financiamiento de Aiken, no estaba muy interesada en las computadoras hasta que produjo la 701 en 1953, mucho después de que la compañía de Eckert y Mauchley se había convertido en la número uno del mercado comercial con su computadora UNIVAC.

La 701 tenía 2048 palabras de 36 bits, con dos instrucciones por palabra. Era la primera de una serie de máquinas científicas que llegaron a dominar la industria en el plazo de una década. Tres años después apareció la 704, que tenía 4K de memoria de núcleos, instrucciones de 36 bits y hardware de punto flotante. En 1958 IBM inició la producción de su última máquina de bulbos, la 709, que básicamente era una 704 más potente.

1.2.3 La segunda generación – transistores (1955-1965)

El transistor fue inventado en los Bell Labs en 1948 por John Bardeen, Walter Brattain y William Schockley, quienes fueron galardonados con el premio Nobel de física de 1956 por su trabajo. En un plazo de 10 años el transistor revolucionó a las computadoras, y para fines de la década de los cincuenta las computadoras de bulbos eran obsoletas. La primera computadora transistorizada se construyó en el Lincoln Laboratory del MIT, una máquina de 16 bits con un diseño parecido al de la Whirlwind I. Se le llamó **TX-0** (computadora Transistorizada eXperimental 0) y su único propósito era probar la TX-2, mucho más elegante.

La TX-2 no logró un progreso decisivo, pero uno de los ingenieros que trabajaban en el laboratorio, Kenneth Olsen, formó una compañía, Digital Equipment Corporation (DEC) en 1957 para fabricar una máquina comercial parecida a la TX-0. Tuvieron que pasar cuatro años para que apareciera esta máquina, la PDP-1, principalmente porque los inversionistas que financiaban a DEC creían firmemente que no había mercado para las computadoras. En vez de ello, DEC se dedicó primordialmente a vender pequeñas tarjetas de circuitos.

Cuando por fin la PDP-1 apareció en 1961, tenía 4K de palabras de 18 bits y un tiempo de ciclo de 5 μ s.† Este desempeño era la mitad del de la IBM 7090, la sucesora transistorizada de la 709 y la computadora más rápida del mundo en esas fechas. La PDP-1 costaba \$120,000 dólares; la 7090 costaba millones. DEC vendió docenas de PDP-1, y con esto la industria de las minicomputadoras había nacido.

Una de las primeras PDP-1 se entregó a MIT, donde rápidamente atrajo la atención de algunos de los jóvenes genios en formación tan comunes en esa institución. Una de las muchas innovaciones de la PDP-1 era una presentación visual y la capacidad de graficar puntos en cualquier lugar de su pantalla de 512 por 512. En poco tiempo, los estudiantes habían programado la PDP-1 para jugar guerras espaciales, y el mundo tuvo su primer juego de video.

Unos cuantos años después DEC introdujo la PDP-8, que era una máquina de 12 bits, pero mucho más económica que la PDP-1 (\$16,000). La PDP-8 tenía una importante innovación: un bus único, u ómnibus, que se muestra en la figura 1-6. Un **bus** es un conjunto de alambres en paralelo que sirve para conectar los componentes de una computadora. Esta arquitectura marcó una divergencia importante respecto a la máquina IAS centrada en la memoria, y ha sido adoptada por casi todas las computadoras pequeñas posteriores. DEC llegó a vender 50,000 PDP-8, lo que la estableció como líder en el negocio de las minicomputadoras.

†Prefijos métricos: mili = 10^{-3} , micro (μ) = 10^{-6} , nano = 10^{-9} ; kilo = 10^3 , mega = 10^6 , giga = 10^9 .

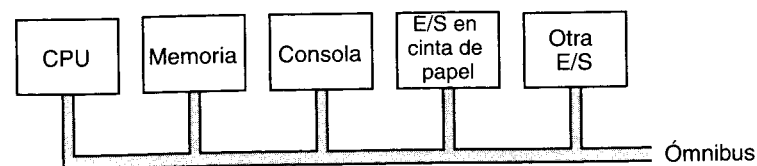


Figura 1-6. El ómnibus de la PDP-8.

Mientras tanto, la reacción de IBM ante el transistor fue construir una versión transistorizada de la 709, la 7090, como ya dijimos, y después la 7094. Ésta tenía un tiempo de ciclo de 2 μ s y una memoria de núcleos de 32 K con palabras de 36 bits. La 7090 y la 7094 marcaron el fin de las máquinas tipo ENIAC, y dominaron la computación científica gran parte de la década de los sesenta.

Al mismo tiempo que IBM se convertía en un protagonista importante en la computación científica con la 7094, estaba ganando enormes cantidades de dinero vendiendo una pequeña máquina orientada hacia los negocios: la 1401. Esta máquina podía leer y grabar cintas magnéticas, leer y perforar tarjetas, e imprimir salidas casi tan rápidamente como la 7094, a una fracción del precio. Era muy mala para la computación científica, pero era perfecta para la contabilidad comercial.

La 1401 era poco común en cuanto a que no tenía registros, y ni siquiera una longitud de palabra fija. Su memoria consistía en 4K bytes de 8 bits (4 KB[†]). Cada byte contenía un carácter de 6 bits, un bit administrativo y un bit que servía para indicar el fin de una palabra. Una instrucción MOVE (mover), por ejemplo, tenía una dirección de origen y una de destino, y comenzaba a transferir bytes del origen al destino hasta encontrar uno cuyo bit de fin de palabra valía 1.

En 1964 una compañía nueva, Control Data Corporation (CDC), introdujo la 6600, una máquina que era casi un orden de magnitud más rápida que la poderosa 7094. Para los “tritadores de números” fue amor a primera vista, y CDC estaba camino al éxito. El secreto de su velocidad, la razón por la que era mucho más rápida que la 7094, era que dentro de la CPU había una máquina con alto grado de paralelismo: contaba con varias unidades funcionales para sumar, otras para multiplicar y otras para dividir, y todas podían operar en paralelo. Aunque para aprovecharla al máximo se requería una programación cuidadosa, con un poco de esfuerzo era posible lograr la ejecución simultánea de 10 instrucciones.

Como si esto no fuera suficiente, la 6600 tenía en su interior varias computadoras pequeñas que la ayudaban, algo así como Blanca Nieves y las siete personas con disfunción vertical. Esto permitía a la CPU dedicar todo su tiempo a la trituración de números, dejando todos los detalles de control de trabajos y entrada/salida a las computadoras pequeñas. En retrospectiva, la 6600 se había adelantado décadas a su época. Muchas de las ideas fundamentales en que se basan las computadoras modernas se remontan directamente a la 6600.

El diseñador de la 6600, Seymour Cray, fue una figura legendaria, comparable con von Neumann. Dedicó su vida a construir máquinas cada vez más rápidas, ahora llamadas

[†]1K = 2¹⁰ = 1024, así que 1 KB = 1024 bytes; también 1M = 2²⁰ = 1,048,576 bytes

supercomputadoras, entre las que figuraron la 6600, la 7600 y la Cray-1. También inventó un algoritmo para comprar automóviles que se ha hecho famoso: acudir a la concesionaria más cercana al hogar, apuntar al automóvil más cercano a la puerta, y decir: “Me llevo ése.” Este algoritmo desperdicia el mínimo de tiempo en hacer cosas sin importancia (como comprar automóviles) y deja el máximo de tiempo libre para hacer cosas importantes (como diseñar supercomputadoras).

Hubo muchas otras computadoras en esta era, pero una sobresale por una razón muy distinta y vale la pena mencionarla: la Burroughs B5000. Los diseñadores de máquinas como la PDP-1, la 7094 y la 6600 se concentraban totalmente en el hardware, ya sea en bajar su costo (DEC) o en hacerlo más rápido (IBM y CDC). El software casi no venía al caso. Los diseñadores de la B5000 adoptaron una estrategia distinta: construyeron una máquina con la intención específica de programarla en Algol 60, un precursor del Pascal, e incluyeron en el hardware muchas funciones que facilitaban la tarea del compilador. Había nacido la idea de que el software también cuenta. Lo malo es que la idea cayó en el olvido casi de inmediato.

1.2.4 La tercera generación – circuitos integrados (1965-1980)

La invención del circuito integrado de silicio por Robert Noyce en 1958 hizo posible colocar docenas de transistores en un solo chip. Este empaquetamiento permitió construir computadoras más pequeñas, más rápidas y menos costosas que sus predecesores transistorizados. A continuación se describen algunas de las computadoras más importantes de esta generación.

Para 1964 IBM era el principal fabricante de computadoras y tenía un problema grave con sus dos máquinas de mayor éxito, la 7094 y la 1401: eran absolutamente incompatibles. Una era un triturador de números de alta velocidad que empleaba aritmética binaria en paralelo con registros de 36 bits, y la otra era un procesador de entrada/salida que parecía hacer más de lo que en verdad hacía y que utilizaba aritmética decimal en serie con palabras de longitud variable en la memoria. Muchos de sus clientes corporativos tenían ambas máquinas y no les gustaba la idea de mantener dos departamentos de programación distintos que nada tenían en común.

Cuando llegó la hora de sustituir estas dos series, IBM dio un paso radical: introdujo una sola línea de productos, la System/360, basada en circuitos integrados, diseñada para computación tanto científica como comercial. La System/360 contenía muchas innovaciones, siendo la más importante que era una familia de cerca de media docena de máquinas con el mismo lenguaje ensamblador y con tamaño y potencia crecientes. Una compañía podía reemplazar su 1401 por una 360 Model 30 y su 7094 por una 360 Model 75. La Model 75 era más grande y rápida (y más costosa), pero el software escrito para ella podía, en principio, ejecutarse en la Model 30. En la práctica, el software escrito para un modelo inferior se ejecutaba en un modelo superior sin problemas, pero al pasar de una máquina superior a una inferior existía la posibilidad de que el programa no cupiera en la memoria. Aun así, esto representó un avance importante respecto a la situación que prevalecía con la 7094 y la 1401. La idea de las familias de máquinas de inmediato fue bien recibida, y en unos cuantos años casi todos los fabricantes de computadoras tenían una familia de máquinas similares que cubrían una

amplia gama de precio y desempeño. En la figura 1-7 se muestran algunas características de la familia 360 inicial. Posteriormente se introdujeron otros modelos.

Propiedad	Modelo 30	Modelo 40	Modelo 50	Modelo 65
Desempeño relativo	1	3.5	10	21
Tiempo de ciclo (ns)	1000	625	500	250
Memoria máxima (KB)	64	256	256	512
Bytes obtenidos por ciclo	1	2	4	16
Número máximo de canales de datos	3	3	4	6

Figura 1-7. La oferta inicial de la línea de productos IBM 360.

Otra importante innovación de la 360 fue la **multiprogramación**: tener varios programas en la memoria a la vez de modo que mientras uno estaba esperando el término de una operación de entrada/salida otro podía realizar cálculos.

La 360 también fue la primera máquina que podía emular (simular) otras computadoras. Los modelos más pequeños podían emular la 1401, y los más grandes podían emular la 7094, para que los clientes pudieran seguir usando sus antiguos programas binarios sin modificación en tanto efectuaban la conversión a la 360. Algunos modelos ejecutaban los programas de la 1401 con tal rapidez (mucho mayor que en la 1401 misma) que muchos clientes nunca convirtieron sus programas.

La emulación era fácil en la 360 porque todos los modelos iniciales y casi todos los modelos posteriores eran microprogramados. Lo único que tenía que hacer IBM era escribir tres microprogramas: para el conjunto de instrucciones nativo de la 360, para el conjunto de instrucciones de la 1401 y para el conjunto de instrucciones de la 7094. Esta flexibilidad fue una de las razones principales por las que se introdujo la microprogramación.

La 360 resolvió el dilema de binaria paralela *versus* decimal en serie con un término medio: la máquina tenía 16 registros de 32 bits para la aritmética binaria, pero su memoria estaba organizada en bytes, como la de la 1401. Además, contaba con instrucciones seriales tipo 1401 para transferir registros de tamaño variable en la memoria.

Otra característica importante de la 360 fue un espacio de direcciones enorme (para esa época) de 2^{24} bytes (16 megabytes). Puesto que la memoria costaba varios dólares por byte en esos días, 16 megabytes parecía casi infinito. Desafortunadamente, la serie 360 fue seguida más adelante por la serie 370, la 4300, la 3080 y la 3090, todas las cuales empleaban la misma arquitectura. Para mediados de los ochenta, el límite de 16 megabytes se había convertido en un verdadero problema, e IBM tuvo que abandonar parcialmente la compatibilidad al cambiar a las direcciones de 32 bits que se necesitaban para direccionar la nueva memoria de 2^{32} bytes.

En retrospectiva, puede argumentarse que como esas máquinas tenían palabras y registros de 32 bits, probablemente debían haber tenido también direcciones de 32 bits, pero en

esa época nadie podía imaginarse una máquina de 16 megabytes. Criticar a IBM por esta falta de visión es como criticar a un moderno fabricante de computadoras personales por tener direcciones de sólo 32 bits. En unos cuantos años las computadoras personales tal vez necesiten más de 4 GB de memoria, y entonces las direcciones de 32 bits serán intolerablemente pequeñas.

El mundo de las minicomputadoras también dio un gran paso en la tercera generación cuando DEC introdujo la serie PDP-11, un sucesor de la PDP-8 de 16 bits. En muchos sentidos, la serie PDP-11 era como el hermano menor de la serie 360, igual que la PDP-1 había sido como un hermanito de la 7094. Tanto la 360 como la PDP-11 tenían registros orientados hacia palabras y memoria orientada hacia bytes, y ambas abarcaban una gama muy amplia de precio y desempeño. La PDP-11 tuvo un éxito enorme, sobre todo en las universidades, y mantuvo la ventaja de DEC sobre los otros fabricantes de minicomputadoras.

1.2.5 La cuarta generación – integración a muy grande escala (1980-?)

Para los años ochenta la **VLSI (integración a muy grande escala, Very Large Scale Integration)** había hecho posible colocar primero decenas de miles, luego centenares de miles y por último millones de transistores en un solo chip. Este avance dio pie a computadoras más pequeñas y más rápidas. Antes de la PDP-1, las computadoras eran tan grandes y costosas que las compañías y universidades necesitaban departamentos especiales llamados **centros de cómputo** para operarlas. Con la llegada de la minicomputadora, un departamento podía comprar su propia computadora. Para 1980 los precios habían bajado tanto que una persona podía tener su propia computadora. Se había iniciado la era de la computadora personal.

Las computadoras personales se usaban de forma muy diferente que las computadoras grandes; se utilizaban para procesamiento de texto, hojas de cálculo y muchas otras aplicaciones altamente interactivas que las computadoras grandes no podían manejar bien.

Las primeras computadoras personales solían venderse como “kits”. Cada kit contenía una tarjeta de circuitos impresos, un puñado de chips, que típicamente incluían un Intel 8080, varios cables, una fuente de potencia y tal vez una unidad de disco flexible de 8 pulgadas. Al comprador correspondía armar las piezas para crear una computadora. No se proporcionaba software. Si usted quería software, lo escribía. Más adelante se popularizó el sistema operativo CP/M para el 8080, escrito por Gary Kildall. Era un verdadero sistema operativo para disco (flexible), con un sistema de archivos y comandos de usuario que se introducían con el teclado.

Otra de las primeras computadoras personales fue la Apple y posteriormente la Apple II, diseñadas por Steve Jobs y Steve Wozniak en la proverbial cochera. Esta máquina se popularizó mucho entre los usuarios caseros y en las escuelas y convirtió a Apple de la noche a la mañana en protagonista.

Después de muchas deliberaciones y de observar lo que otras compañías estaban haciendo, IBM, que entonces era la fuerza dominante en la industria de la computación, por fin decidió que quería ingresar en el negocio de las computadoras personales. En lugar de diseñar una máquina desde cero, empleando únicamente componentes de IBM, lo cual habría tardado mucho tiempo, IBM hizo algo totalmente fuera de carácter: entregó a uno de sus ejecutivos, Philip Estridge, una gran bolsa de dinero y le ordenó irse a algún lugar lejos de los entrometidos burócratas de las oficinas corporativas en Armonk, N.Y., y que no volviera

antes de tener una computadora personal funcional. Estridge se estableció lejos de la oficina central, en Boca Raton, Florida, escogió el Intel 8088 como su CPU, y construyó la IBM Personal Computer a partir de componentes comerciales. La máquina se introdujo en 1981 y se convirtió de inmediato en la computadora más vendida de la historia.

IBM hizo otra cosa fuera de carácter de la que más tarde se arrepentiría. En lugar de mantener totalmente en secreto el diseño de la máquina (o al menos protegido por una barrera de patentes), como normalmente hacía, publicó los planos completos, incluidos todos los diagramas de circuitos, en un libro que vendió a 49 dólares. Lo que se buscaba es que otras compañías fabricaran tarjetas que pudieran insertarse en la IBM PC (plugins) a fin de aumentar su flexibilidad y popularidad. Por desgracia para IBM, dado que el diseño ya era totalmente público y todas las piezas se podían conseguir fácilmente de proveedores comerciales con facilidad, muchas otras compañías comenzaron a fabricar clones de la PC, casi siempre a un costo mucho menor que lo que IBM cobraba. Fue así como nació toda una industria.

Aunque otras compañías fabricaron computadoras personales utilizando unas CPU que no eran de Intel, como las Commodore, Apple, Amiga y Atari, el ímpetu de la industria de la IBM PC era tan grande que las demás fueron arrolladas. Sólo unas cuantas sobrevivieron, aunque estuvieron restringidas a ciertos nichos del mercado, como estaciones de trabajo de ingeniería o supercomputadoras.

La versión inicial de la IBM PC venía equipada con el sistema operativo MS-DOS provisto por la entonces pequeña Microsoft Corporation. Cuando Intel logró producir CPU cada vez más potentes, IBM y Microsoft pudieron producir un sucesor del MS-DOS llamado OS/2, que contaba con una interfaz gráfica con el usuario similar a la de la Apple Macintosh. Mientras tanto, Microsoft creó también su propio sistema operativo Windows que se ejecutaba encima del MS-DOS, por si acaso el OS/2 no tenía aceptación. Sin abundar en la historia, el OS/2 no experimentó una buena acogida, IBM y Microsoft tuvieron una gran pelea extremadamente pública, y Microsoft se dedicó a hacer de Windows un enorme éxito. La forma en que la diminuta Intel y la todavía más diminuta Microsoft lograron destronar a IBM, una de las corporaciones más grandes, ricas y poderosas en la historia del mundo, es una parábola que sin duda se relata con lujo de detalle en escuelas de negocios de todo el mundo.

Para mediados de los ochenta una nueva idea llamada RISC comenzó a dominar, reemplazando arquitecturas complejas (CISC) por otras mucho más sencillas pero más rápidas. En los noventa comenzaron a aparecer las CPU superescalares. Estas máquinas podían ejecutar varias instrucciones al mismo tiempo, a menudo en un orden distinto del que tenían en el programa. Presentaremos los conceptos de CISC, RISC y superescalar en el capítulo 2 y los trataremos con detalle en todo el libro.

1.3 EL ZOOLÓGICO DE LAS COMPUTADORAS

En la sección anterior presentamos una muy breve historia de los sistemas de cómputo. En ésta examinaremos el presente y miraremos hacia el futuro. Aunque las computadoras personales son las computadoras más conocidas, hay otras clases de máquinas en uso actualmente, y vale la pena dar un vistazo a esas otras computadoras.

1.3.1 Fuerzas tecnológicas y económicas

La industria de la computación está avanzando como ninguna otra. La fuerza motriz es la capacidad de los fabricantes de chips para empaquetar cada vez más un chip. Un mayor número de transistores, que son diminutos interruptores, implica memorias más grandes y procesadores más potentes.

La rapidez del progreso tecnológico puede modelarse de acuerdo con una observación llamada **ley de Moore**, descubierta por Gordon Moore, cofundador y director de Intel, en 1965. Mientras preparaba un discurso para un grupo de la industria, Moore se dio cuenta de que cada nueva generación de chips de memoria se estaba introduciendo tres años después de la anterior. Puesto que cada nueva generación tenía cuatro veces más memoria que su predecesora, Moore se percató de que el número de transistores en un chip estaba aumentando de forma constante y predijo que este crecimiento continuaría durante varias décadas. Hoy día, la ley de Moore suele expresarse como que el número de transistores se duplica cada 18 meses. Cabe señalar que esto equivale a un incremento de cerca del 60% en el número de transistores cada año. Los tamaños de los chips de memoria y sus fechas de introducción, que se muestran en la figura 1-8, confirman que la ley de Moore se sigue cumpliendo.

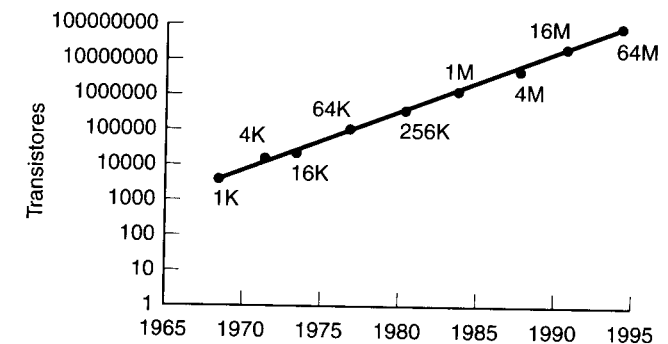


Figura 1-8. La ley de Moore predice un incremento anual de 60% en el número de transistores que se pueden colocar en un chip. Los puntos de datos dados en esta figura son tamaños de memoria, en bits.

Desde luego, la ley de Moore no es realmente una ley, sino sólo una observación empírica acerca de la rapidez con que los físicos de estado sólido y los ingenieros de proceso están empujando hacia adelante las fronteras tecnológicas, y una predicción de que seguirán haciéndolo al mismo ritmo en el futuro. Muchos observadores de la industria esperan que la ley de Moore se seguirá cumpliendo ya entrado el siglo XXI, posiblemente hasta 2020. En ese punto los transistores consistirán en un número demasiado reducido de átomos como para ser confiables, aunque los adelantos en la computación cuántica podrían hacer posible para entonces el almacenamiento de un bit utilizando el espín de un solo electrón.

La ley de Moore ha creado lo que algunos economistas llaman un **círculo virtuoso**. Los adelantos en la tecnología (transistores por chip) dan pie a mejores productos y precios más bajos. Los precios bajos dan pie a nuevas aplicaciones (nadie estaba creando juegos de video

antes de tener una computadora personal funcional. Estridge se estableció lejos de la oficina central, en Boca Raton, Florida, escogió el Intel 8088 como su CPU, y construyó la IBM Personal Computer a partir de componentes comerciales. La máquina se introdujo en 1981 y se convirtió de inmediato en la computadora más vendida de la historia.

IBM hizo otra cosa fuera de carácter de la que más tarde se arrepentiría. En lugar de mantener totalmente en secreto el diseño de la máquina (o al menos protegido por una barrera de patentes), como normalmente hacía, publicó los planos completos, incluidos todos los diagramas de circuitos, en un libro que vendió a 49 dólares. Lo que se buscaba es que otras compañías fabricaran tarjetas que pudieran insertarse en la IBM PC (plugins) a fin de aumentar su flexibilidad y popularidad. Por desgracia para IBM, dado que el diseño ya era totalmente público y todas las piezas se podían conseguir fácilmente de proveedores comerciales con facilidad, muchas otras compañías comenzaron a fabricar clones de la PC, casi siempre a un costo mucho menor que lo que IBM cobraba. Fue así como nació toda una industria.

Aunque otras compañías fabricaron computadoras personales utilizando unas CPU que no eran de Intel, como las Commodore, Apple, Amiga y Atari, el ímpetu de la industria de la IBM PC era tan grande que las demás fueron arrolladas. Sólo unas cuantas sobrevivieron, aunque estuvieron restringidas a ciertos nichos del mercado, como estaciones de trabajo de ingeniería o supercomputadoras.

La versión inicial de la IBM PC venía equipada con el sistema operativo MS-DOS provisto por la entonces pequeña Microsoft Corporation. Cuando Intel logró producir CPU cada vez más potentes, IBM y Microsoft pudieron producir un sucesor del MS-DOS llamado OS/2, que contaba con una interfaz gráfica con el usuario similar a la de la Apple Macintosh. Mientras tanto, Microsoft creó también su propio sistema operativo Windows que se ejecutaba encima del MS-DOS, por si acaso el OS/2 no tenía aceptación. Sin abundar en la historia, el OS/2 no experimentó una buena acogida, IBM y Microsoft tuvieron una gran pelea extremadamente pública, y Microsoft se dedicó a hacer de Windows un enorme éxito. La forma en que la diminuta Intel y la todavía más diminuta Microsoft lograron destronar a IBM, una de las corporaciones más grandes, ricas y poderosas en la historia del mundo, es una parábola que sin duda se relata con lujo de detalle en escuelas de negocios de todo el mundo.

Para mediados de los ochenta una nueva idea llamada RISC comenzó a dominar, reemplazando arquitecturas complejas (CISC) por otras mucho más sencillas pero más rápidas. En los noventa comenzaron a aparecer las CPU superescalares. Estas máquinas podían ejecutar varias instrucciones al mismo tiempo, a menudo en un orden distinto del que tenían en el programa. Presentaremos los conceptos de CISC, RISC y superescalar en el capítulo 2 y los trataremos con detalle en todo el libro.

1.3 EL ZOOLÓGICO DE LAS COMPUTADORAS

En la sección anterior presentamos una muy breve historia de los sistemas de cómputo. En ésta examinaremos el presente y miraremos hacia el futuro. Aunque las computadoras personales son las computadoras más conocidas, hay otras clases de máquinas en uso actualmente, y vale la pena dar un vistazo a esas otras computadoras.

1.3.1 Fuerzas tecnológicas y económicas

La industria de la computación está avanzando como ninguna otra. La fuerza impulsora primordial es la capacidad de los fabricantes de chips para empacar cada vez más transistores en un chip. Un mayor número de transistores, que son diminutos interruptores electrónicos, implica memorias más grandes y procesadores más potentes.

La rapidez del progreso tecnológico puede modelarse de acuerdo con una observación llamada **ley de Moore**, descubierta por Gordon Moore, cofundador y director de Intel, en 1965. Mientras preparaba un discurso para un grupo de la industria, Moore se dio cuenta de que cada nueva generación de chips de memoria se estaba introduciendo tres años después de la anterior. Puesto que cada nueva generación tenía cuatro veces más memoria que su predecesora, Moore se percató de que el número de transistores en un chip estaba aumentando de forma constante y predijo que este crecimiento continuaría durante varias décadas. Hoy día, la ley de Moore suele expresarse como que el número de transistores se duplica cada 18 meses. Cabe señalar que esto equivale a un incremento de cerca del 60% en el número de transistores cada año. Los tamaños de los chips de memoria y sus fechas de introducción, que se muestran en la figura 1-8, confirman que la ley de Moore se sigue cumpliendo.

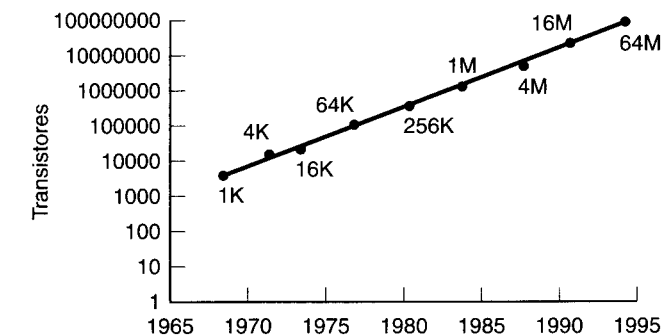


Figura 1-8. La ley de Moore predice un incremento anual de 60% en el número de transistores que se pueden colocar en un chip. Los puntos de datos dados en esta figura son tamaños de memoria, en bits.

Desde luego, la ley de Moore no es realmente una ley, sino sólo una observación empírica acerca de la rapidez con que los físicos de estado sólido y los ingenieros de proceso están empujando hacia adelante las fronteras tecnológicas, y una predicción de que seguirán haciéndolo al mismo ritmo en el futuro. Muchos observadores de la industria esperan que la ley de Moore se seguirá cumpliendo ya entrado el siglo XXI, posiblemente hasta 2020. En ese punto los transistores consistirán en un número demasiado reducido de átomos como para ser confiables, aunque los adelantos en la computación cuántica podrían hacer posible para entonces el almacenamiento de un bit utilizando el espín de un solo electrón.

La ley de Moore ha creado lo que algunos economistas llaman un **círculo virtuoso**. Los adelantos en la tecnología (transistores por chip) dan pie a mejores productos y precios más bajos. Los precios bajos dan pie a nuevas aplicaciones (nadie estaba creando juegos de video

para computadoras cuando éstas costaban 10 millones de dólares cada una). Las nuevas aplicaciones dan pie a nuevos mercados y nuevas compañías que surgen para aprovecharlos. La existencia de todas estas compañías da pie a una competencia que, a su vez, crea una demanda económica de mejores tecnologías con las cuales superar a los demás. Esto completa el círculo.

Otro factor que impulsa el mejoramiento tecnológico es la primera ley del software de Nathan (debida a Nathan Myhrvold, un alto ejecutivo de Microsoft) que dice: “El software es un gas; se expande hasta llenar el recipiente que lo contiene.” En los años ochenta el procesamiento de textos se efectuaba con programas como Troff (que se utilizó para este libro). Troff ocupa unas cuantas decenas de kilobytes de memoria. Los modernos procesadores de textos ocupan decenas de megabytes. Los programas futuros sin duda requerirán decenas de gigabytes. El software que continuamente añade funciones (como los cascos de los barcos que sin cesar adquieren bálanos) crea una demanda constante de procesadores más rápidos, memorias más grandes y mayor capacidad de entrada/salida.

Si bien el aumento en el número de transistores por chip con el paso de los años ha sido impresionante, los avances en otras tecnologías de cómputo no se han quedado muy atrás. Por ejemplo, en 1982 se introdujo la IBM PC/XT con un disco duro de 10 MB. Por regla general, los sistemas de escritorio actuales tienen discos un tanto mayores. Medir el mejoramiento de los discos no es tan sencillo, porque implica varios parámetros (capacidad, tasa de datos, precio, etc.), pero casi cualquier métrica indicará que las capacidades han aumentado desde entonces a razón de 50% anual, por lo menos.

Otras áreas que han sido testigo de avances espectaculares son las telecomunicaciones y las redes. En menos de dos décadas hemos pasado de módems de 300 bits/s a módems analógicos de 56 kbps, líneas telefónicas ISDN de 2 × 64 kbps y redes de fibra óptica de más de un gigabit/s. Los cables telefónicos trasatlánticos de fibras ópticas, como TAT-12/13, cuestan unos 700 millones de dólares, duran 10 años y pueden transportar 300,000 llamadas simultáneas, lo que da un costo de menos de un centavo de dólar por una llamada intercontinental de 10 minutos. En el laboratorio se ha demostrado la factibilidad de sistemas de comunicación ópticos que operan a 1 terabit/s (10¹² bits/s) a distancias de más de 100 km sin amplificadores. El crecimiento exponencial de Internet es tan evidente que no hace falta mencionarlo aquí.

1.3.2 La gama de las computadoras

Richard Hamming, antiguo investigador de Bell Labs, cierta vez observó que un cambio de un orden de magnitud en la cantidad causa un cambio en la calidad. Así, un automóvil que puede correr a 1000 km/h en el desierto de Nevada tiene un tipo de máquina fundamentalmente distinto de la de un automóvil normal que viaja a 100 km/h en una autopista. Así mismo, un rascacielos de 100 pisos no es sólo un edificio de apartamentos de 10 pisos a mayor escala. Y con las computadoras no estamos hablando de factores de 10, sino de factores de un millón en el curso de tres décadas.

Los aumentos predichos por la ley de Moore se pueden utilizar de varias maneras. Una de ellas es construir computadoras cada vez más potentes que no cuesten más. Otro enfoque es construir la misma computadora con menos y menos dinero cada año. La industria de la computación ha hecho ambas cosas y más, y el resultado es la amplia variedad de computadoras

que se venden actualmente. En la figura 1-9 se presenta una clasificación muy burda de las computadoras actuales.

Tipo	Precio	Ejemplo de aplicación
Computadora desechable	1	Tarjetas de felicitación
Computadora incorporada	10	Relojes, automóviles, aparatos
Computadora de juegos	100	Juegos de video caseros
Computadora personal	1 K	Computadora de escritorio o portátil
Servidor	10 K	Servidor de red
Colección de estaciones de trabajo	100 K	Minisupercomputadora
Mainframe (macrocomputadora)	1 M	Procesamiento de datos por lotes en un banco
Supercomputadora	10 M	Predicción del tiempo a largo plazo

Figura 1-9. Gama de computadoras disponibles en la actualidad. Los precios no deben tomarse muy en serio.

En el extremo inferior encontramos chips individuales pegados al interior de tarjetas de felicitación para tocar “Feliz cumpleaños”, la “Marcha nupcial” o alguna otra tonadilla igualmente abominable. El autor todavía no ha encontrado una tarjeta de condolencias que toque una marcha fúnebre, pero después de haber sacado esta idea al dominio público, se cree que no tardará en aparecer. Para cualquier persona que haya crecido con mainframes de millones de dólares, la idea de una computadora desechable es tan lógica como la de un avión desechable. No obstante, es indudable que las computadoras desechables llegaron para quedarse. (¿Qué tal bolsas para la basura parlantes que nos recuerden reciclar las latas de aluminio?)

En el siguiente peldaño tenemos las computadoras que están incorporadas en teléfonos, televisores, hornos de microondas, reproductores de CD, juguetes, muñecas y miles de aparatos más. En unos cuantos años, todo artículo eléctrico incluirá una computadora. El número de computadoras incorporadas se medirá en miles de millones, rebasando por órdenes de magnitud a todas las demás computadoras combinadas. Estas computadoras contienen un procesador, menos de un megabyte de memoria, y cierta capacidad de E/S, todo en un solo chip que cuesta unos cuantos dólares.

En el siguiente peldaño están las máquinas de videojuegos. Se trata de computadoras normales, con capacidades gráficas especiales, pero software limitado y casi ninguna extensibilidad. En este intervalo de precios también están los organizadores personales, los asistentes digitales portátiles y otros dispositivos de computación de bolsillo similares, así como las computadoras de red y las terminales de Web. Lo que estas computadoras tienen en común es que contienen un procesador, unos cuantos megabytes de memoria, algún tipo de pantalla (posiblemente un televisor) y poco más. Eso es lo que los hace tan baratos.

A continuación vienen las computadoras personales en las que la mayoría de la gente piensa cuando oye la palabra “computadora”. Éstas incluyen modelos de escritorio y portátiles. Por lo regular incluyen varios megabytes de memoria, un disco duro con unos cuantos gigabytes

de datos, una unidad de CD-ROM, un módem, una tarjeta de sonido y otros periféricos. Estas máquinas cuentan con sistemas operativos complejos, muchas opciones de expansión y un enorme surtido de software comercial. Las que tienen una CPU Intel suelen llamarse “computadoras personales”, mientras que las que tienen otro tipo de CPU a menudo se llaman “estaciones de trabajo”, aunque en lo conceptual casi no hay diferencias entre los dos tipos.

Ciertas computadoras personales o estaciones de trabajo mejoradas suelen utilizarse como servidores de red, tanto para redes de área local (casi siempre dentro de una sola compañía) como para Internet. Éstas tienen configuraciones de un solo procesador o de múltiples procesadores, tienen unos cuantos gigabytes de memoria, muchos gigabytes de espacio en disco duro, y capacidad de trabajo en red de alta velocidad. Algunas de ellas pueden manejar docenas o cientos de llamadas a la vez.

Más allá de los servidores multiprocesador pequeños encontramos sistemas conocidos como sistemas **NOW (redes de estaciones de trabajo, *Networks of Workstations*)** o **COW (cúmulos de estaciones de trabajo, *Clusters of Workstations*)**. Éstos son computadores personales o estaciones de trabajo estándar conectadas por redes de gigabits/s y que ejecutan programas especiales que permiten a todas las máquinas colaborar en la solución de un solo problema, que a menudo es científico o de ingeniería. Es fácil cambiar la escala de estos sistemas, desde un puñado de máquinas hasta miles de ellas. Gracias al bajo precio de sus componentes, los departamentos independientes pueden poseer tales máquinas, que de hecho son minisupercomputadoras.

Ahora llegamos a las mainframes, computadoras que llenan una habitación y que se remontan a los sesenta. En muchos casos, estas máquinas son descendientes directos de mainframes IBM 360 adquiridas décadas atrás. En su mayor parte, estas computadoras no son mucho más rápidas que servidores potentes, pero suelen tener mayor capacidad de E/S y están equipadas con grandes “granjas” de discos que llegan a contener un terabyte de datos o más ($1 \text{ TB} = 10^{12} \text{ bytes}$). Aunque son extremadamente costosas, estas máquinas a menudo se mantienen funcionando en vista de la enorme inversión en software, datos, procedimientos operativos y personal que representan. Muchas compañías consideran que es más económico pagar unos cuantos millones de dólares de vez en cuando por una nueva, que siquiera contemplar el esfuerzo necesario para reprogramar todas sus aplicaciones para máquinas más pequeñas.

Es esta clase de computadoras la que dio pie al problema del Año 2000 de triste fama, el cual es culpa de los programadores en COBOL de los sesenta y setenta que representaban el año como dos dígitos decimales (para ahorrar memoria). Ellos nunca imaginaron que su software duraría tres o cuatro décadas. Muchas compañías han repetido el mismo error al limitarse a agregar dos dígitos más al año. El autor aprovecha la ocasión para predecir el fin de la civilización que conocemos a la media noche del 31 de diciembre de 9999, cuando 8000 años de viejos programas en COBOL dejarán de funcionar simultáneamente.

Después de las mainframes vienen las verdaderas supercomputadoras, cuyas CPU son inconcebiblemente rápidas, tienen muchos gigabytes de memoria principal, y sus discos y sus redes muy rápidos. En años recientes, muchas de las supercomputadoras se han convertido en máquinas altamente paralelas, no muy diferentes de los diseños COW, pero con componentes más rápidos y numerosos. Las supercomputadoras se emplean para resolver problemas que requieren muchos cálculos en ciencia e ingeniería, como simular el choque de galaxias, sintetizar nuevas medicinas o modelar el flujo del aire alrededor de un ala de avión.

1.4 EJEMPLOS DE FAMILIAS DE COMPUTADORAS

En esta sección presentaremos una breve introducción a las tres computadoras que usaremos como ejemplos en el resto del libro: la Pentium II, la UltraSPARC II y la picoJava II [sic].

1.4.1 Introducción al Pentium II

En 1968 Robert Noyce, inventor del circuito integrado de silicio; Gordon Moore, cuya ley ya es famosa, y Arthur Rock, un inversionista de San Francisco, formaron la Intel Corporation para fabricar chips de memoria. En su primer año de operación, Intel sólo vendió 3000 dólares de chips, pero el negocio ha prosperado desde entonces.

A fines de los años sesenta, las calculadoras eran grandes máquinas electromecánicas del tamaño de una moderna impresora láser y con un peso de unos 20 kg. En septiembre de 1969 una compañía japonesa, Busicom, se acercó a Intel para pedirle que fabricara 12 chips a la medida para una calculadora electrónica propuesta. El ingeniero de Intel asignado a este proyecto, Ted Hoff, examinó el plan y se percató de que podía poner una CPU de 4 bits de propósito general en un solo chip, y que esto efectuaría el mismo trabajo y sería además más simple y económico. Fue así como en 1970 nació la primera CPU en un solo chip, la 4004 de 2300 transistores (Faggin *et al.*, 1996).

Vale la pena señalar que ni Intel ni Busicom tenían la menor idea de lo que acababan de hacer. Cuando Intel decidió que valdría la pena hacer el intento de usar la 4004 en otros proyectos, ofreció comprar a Busicom los derechos del nuevo chip devolviéndole los 60,000 dólares que ésta había pagado a Intel por desarrollarlo. Busicom aceptó rápidamente el ofrecimiento de Intel, y éste comenzó a trabajar en una versión de ocho bits del chip, el 8008, introducido en 1972.

Intel no esperaba mucha demanda por el 8008, así que estableció una línea de producción de bajo volumen. Para asombro de todos, el chip despertó un interés enorme, por lo que Intel se dedicó a diseñar un nuevo chip de CPU que superara la limitación de 16K de memoria del 8008 (impuesta por el número de agujas en el chip). Este diseño dio como resultado el 8080, una CPU pequeña, de propósito general, introducido en 1974. Como había hecho la PDP-8, este producto tomó a la industria por asalto y de la noche a la mañana se convirtió en un artículo de mercado masivo. Sólo que en lugar de vender millares, como había hecho DEC, Intel vendió millones.

En 1978 llegó el 8086, una verdadera CPU de 16 bits en un solo chip. El 8086 tenía un diseño similar al del 8080, pero no era totalmente compatible con éste. Al 8086 siguió el 8088, que tenía la misma arquitectura que el 8086 y ejecutaba los mismos programas, pero tenía un bus de 8 bits en lugar de uno de 16 bits, lo que lo hacía más lento pero más económico que el 8086. Cuando IBM escogió el 8088 como CPU para la IBM PC original, este chip rápidamente se convirtió en el estándar para la industria de las computadoras personales.

Ni el 8088 ni el 8086 podían direccionar más de un megabyte de memoria. Para principios de los ochenta esto se convirtió en un problema cada vez más grave, por lo que Intel diseñó el 80286, una versión del 8086 compatible hacia arriba. El conjunto de instruc-

ciones básico era casi el mismo que el del 8086 y el 8088, pero la organización de la memoria era muy diferente y un tanto torpe, debido al requisito de compatibilidad con los chips anteriores. El 80286 se usó en la IBM PC/AT y en los modelos PS/2 de mediano precio. Al igual que el 8088, este chip tuvo un éxito enorme, principalmente porque la gente lo veía como un 8088 más rápido.

El siguiente paso lógico era una verdadera CPU de 32 bits en un chip, el 80386, que salió al mercado en 1985. Al igual que el 80286, éste era más o menos compatible con todos los chips anteriores, hasta el 8080. Ser compatible con lo existente era una bendición para la gente que necesitaba ejecutar software viejo, pero un fastidio para quienes habrían preferido una arquitectura sencilla, limpia y moderna que no cargara con los errores y la tecnología del pasado.

Cuatro años después salió el 80486. Éste era básicamente una versión más rápida del 80386 que además tenía una unidad de punto flotante y 8K de memoria caché en el chip. La **memoria caché** sirve para retener las palabras de memoria más comúnmente usadas dentro o cerca de la CPU, para evitar los (lentos) accesos a la memoria principal. El 80386 también contaba con apoyo de multiprocesador integrado, que permitía a los fabricantes construir sistemas con múltiples CPU.

En este punto Intel se enteró por las malas (al perder una demanda legal de violación de marca comercial) que los números (como 80486) no pueden registrarse como marcas, así que la nueva generación recibió un nombre: Pentium (del vocablo griego penta, cinco). A diferencia del 80486, que tenía una fila de procesamiento, el Pentium tenía dos, y esto lo ayudaba a ser dos veces más rápido (trataremos las filas de procesamiento con detalle en el capítulo 2).

Cuando apareció la siguiente generación, la gente que esperaba el Sexium (*sex* es seis en latín) sufrió una decepción. El nombre Pentium ya era tan conocido que la gente de comercialización quería conservarlo, así que el nuevo chip recibió el nombre de Pentium Pro. A pesar de lo trivial del cambio en el nombre respecto al de su predecesor, este procesador representó una importante ruptura con el pasado. En lugar de tener dos o más filas de procesamiento, el Pentium Pro tenía una organización interna muy distinta y podía ejecutar hasta cinco instrucciones a la vez.

Otra innovación del Pentium Pro fue una memoria caché de dos niveles. El chip procesador mismo tenía 8 KB de memoria para retener las instrucciones de uso común, y 8 KB de memoria para retener datos de uso común. En la misma cavidad dentro del paquete Pentium Pro (pero no en el chip mismo) había una segunda memoria caché de 256 KB.

El siguiente procesador Intel fue el Pentium II, que era básicamente un Pentium Pro con la adición de extensiones multimedia especiales (llamadas **MMX**). Estas instrucciones estaban diseñadas para acelerar los cálculos requeridos para procesar audio y video, lo que hacía innecesaria la adición de coprocesadores de multimedia especiales. Estas instrucciones también estuvieron disponibles en procesadores Pentium posteriores, pero no en el Pentium Pro, de modo que el Pentium II combinaba las ventajas del Pentium Pro con multimedios.

A principios de 1998 Intel introdujo una nueva línea de productos llamada **Celeron**, que es básicamente una versión de bajo costo y bajo desempeño del Pentium II proyectada para PC del extremo inferior. Puesto que el Celeron tiene la misma arquitectura que el Pentium II, no hablaremos más de él en este libro. En junio de 1998 Intel introdujo una versión especial del Pentium II para el extremo superior del mercado. Este procesador, llamado **Xeon**, tiene

un caché más grande, un bus más rápido y mejor apoyo multiprocesador, pero por lo demás es un Pentium II normal, así que tampoco lo trataremos individualmente. La familia Intel se muestra en la figura 1-10.

Chip	Fecha	MHz	Transistores	Memoria	Notas
4004	4/1971	0.108	2,300	640	Primer microprocesador en un chip
8008	4/1972	0.108	3,500	16 KB	Primer microprocesador de 8 bits
8080	4/1974	2	6,000	64 KB	Primera CPU de propósito general en un bit
8086	6/1978	5-10	29,000	1 MB	Primera CPU de 16 bits en un chip
8088	6/1979	5-8	29,000	1 MB	Se usó en la IBM PC
80286	2/1982	8-12	134,000	16 BM	Cuenta con protección de memoria
80386	10/1985	16-33	275,000	4 GB	Primer CPU de 32 bits
80486	4/1989	25-100	1.2M	4 GB	Memoria caché de 8K integrada
Pentium	3/1993	60-233	3.1M	4 GB	Dos conductos; modelos posteriores tenían MMX
Pentium Pro	3/1995	150-200	5.5M	4 GB	Dos niveles de caché integrados
Pentium II	5/1997	233-400	7.5M	4 GB	Pentium Pro más MMX

Figura 1-10. La familia de CPU Intel. Las velocidades de reloj se miden en MHz (megahertz), donde 1 MHz es 1 millón de ciclos/s.

Todos los chips Intel son compatibles con sus predecesores hasta el 8086. En otras palabras, un Pentium II puede ejecutar programas del 8086 sin modificación. Esta compatibilidad siempre ha sido un requisito de diseño para Intel, a fin de que los usuarios puedan mantener su inversión en software existente. Desde luego, el Pentium II es 250 veces más complejo que el 8086, así que puede hacer una buena cantidad de cosas que el 8086 nunca pudo hacer. Estas extensiones incrementales han dado pie a una arquitectura menos elegante de lo que podría haber sido si alguien hubiera entregado a los arquitectos del Pentium II 7.5 millones de transistores e instrucciones para comenzar otra vez desde el principio.

Resulta interesante que si bien la ley de Moore ha estado asociada desde hace mucho al número de bits de una memoria, aplica igualmente bien a los chips de CPU. Si graficamos los números de transistores dados en la figura 1-10 contra la fecha de introducción de cada chip en una escala semilogarítmica, vemos que la ley de Moore también se cumple. La gráfica correspondiente se presenta en la figura 1-11.

1.4.2 Introducción al UltraSPARC II

En los setenta UNIX era popular en las universidades, pero ninguna computadora personal lo ejecutaba, así que los amantes de UNIX tenían que usar minicomputadoras de tiempo compar-

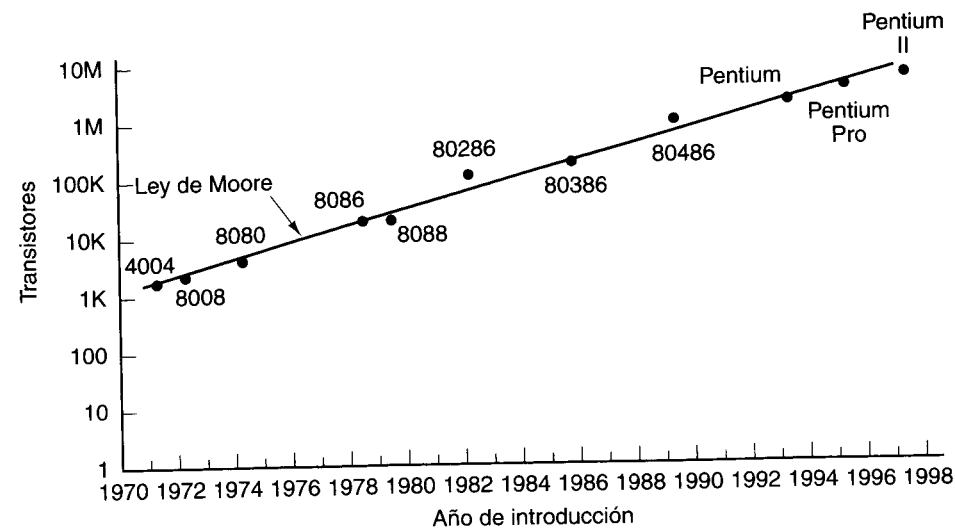


Figura 1-11. Ley de Moore para chips de CPU.

tido (a menudo sobrecargadas) como la PDP-11 y VAX. En 1981 un estudiante de posgrado alemán en Stanford, Andy Bechtolsheim, quien se sentía frustrado por tener que acudir al centro de cómputo para usar UNIX, decidió resolver el problema construyendo para su uso personal una estación de trabajo UNIX utilizando componentes comerciales, y la llamó SUN-1 (Stanford University Network).

Bechtolsheim pronto atrajo la atención de Vinod Khosla, un hindú de 27 años que tenía el deseo ferviente de retirarse como millonario antes de cumplir los 30 años. Khosla convenció a Bechtolsheim de formar una compañía para construir y vender estaciones de trabajo Sun. Luego, Khosla contrató a Scott McNealy, otro estudiante de posgrado de Stanford, para encargarse del área de producción. Para escribir el software contrataron a Bill Joy, el principal arquitecto del Berkeley UNIX. Los cuatro fundaron Sun Microsystems en 1982.

El primer producto de Sun, la Sun-1, basada en la CPU Motorola 68020, tuvo un éxito inmediato, lo mismo que las máquinas subsecuentes Sun-2 y Sun-3, que también usaban CPU Motorola. A diferencia de otras computadoras personales de la época, estas máquinas eran mucho más potentes (de ahí la designación "estación de trabajo") y estaban diseñadas desde un principio para operar en una red. Cada estación de trabajo Sun venía equipada con una conexión Ethernet y con software TCP/IP para conectarse con ARPANET, la precursora de Internet.

Para 1987 Sun, que ya estaba vendiendo 500 millones de dólares al año en sistemas, decidió diseñar su propia CPU basándola en un nuevo y revolucionario diseño de la University of California at Berkeley (el RISC II). Esta CPU, llamada **SPARC (arquitectura de procesador escalable, Scalable Processor ARChitecture)**, fue la base para la estación de trabajo Sun-4. En poco tiempo, todos los productos de Sun utilizaban la CPU SPARC.

A diferencia de muchas otras compañías de computadoras, Sun decidió no fabricar el chip de CPU SPARC ella misma. En vez de ello, otorgó licencias a varios fabricantes de semiconductores para producirlo, esperando que la competencia entre ellos hiciera subir el

desempeño y bajar los precios. Los proveedores fabricaron varios chips distintos, basados en diferentes tecnologías, que operaban a diferente velocidad de reloj, y con distintos precios. Estos chips incluyeron MicroSPARC, HyperSPARC, SuperSPARC y TurboSPARC. Aunque estas CPU diferían en aspectos menores, todas eran compatibles en lo binario y ejecutaban los mismos programas de usuario sin modificación.

Sun siempre quiso que SPARC fuera una arquitectura abierta, con muchos proveedores de componentes y sistemas, a fin de forjar una industria capaz de competir en un mundo de computadoras personales ya dominado por las CPU basadas en Intel. Para ganarse la confianza de compañías que estaban interesadas en SPARC pero no querían invertir en un producto controlado por un competidor, Sun creó un consorcio de la industria, SPARC International, para controlar el desarrollo de versiones futuras de la arquitectura SPARC. Por ello es importante distinguir entre la arquitectura SPARC, que es una especificación del conjunto de instrucciones y otras características visibles para el programador, y una implementación específica de ella. En este libro estudiaremos la arquitectura SPARC genérica y, al hablar de los chips de CPU en los capítulos 3 y 4, de un chip SPARC específico empleado en estaciones de trabajo Sun.

El SPARC inicial era una máquina completa de 32 bits que operaba a 36 MHz. La CPU, llamada **IU (unidad de enteros, Integer Unit)**, era magra y feroz, con sólo tres formatos de instrucción principales y sólo 55 instrucciones en total. Además, una unidad de punto flotante aportaba otras 14 instrucciones. Esta historia puede contrastarse con la línea Intel, que comenzó con chips de 8 y 16 bits (8088, 8086, 80286) y por fin se convirtió en un chip de 32 bits con el 80386.

La primera ruptura de SPARC con el pasado ocurrió en 1995, con el desarrollo de la versión 9 de la arquitectura SPARC, una arquitectura cabal de 64 bits, con direcciones de 64 bits y registros de 64 bits. La primera estación de trabajo Sun que implementó la arquitectura V9 (versión 9) fue la **UltraSPARC I**, introducida en 1995 (Tremblay y O'Connor, 1996). A pesar de ser una máquina de 64 bits, era totalmente compatible en lo binario con los SPARC de 32 bits existentes.

El propósito de UltraSPARC era explorar nuevos terrenos. Mientras que máquinas anteriores estaban diseñadas para manejar datos alfanuméricos, ejecutar programas como procesadores de textos y hojas de cálculo, la UltraSPARC se diseñó desde el principio para manejar imágenes, audio, video y multimedia en general. Entre otras innovaciones además de la arquitectura de 64 bits había 23 instrucciones nuevas, incluidas algunas para empacar y desempacar píxeles de palabras de 64 bits, cambiar la escala de imágenes y girarlas, transferir bloques y realizar compresión y descompresión de video en tiempo real. Estas instrucciones, llamadas **VIS (conjunto de instrucciones visuales, Visual Instruction Set)** tenían por objeto ofrecer una capacidad multimedia general, análoga a las instrucciones MMX de Intel.

El UltraSPARC estaba dirigido a aplicaciones del extremo superior, como servidores de Web multiprocesador grandes, con docenas de CPU y memorias físicas de hasta 2 TB [1 TB (terabyte) = 10^{12} bytes]. Sin embargo, versiones más pequeñas pueden usarse también en computadoras portátiles.

Los sucesores del UltraSPARC I fueron UltraSPARC II y UltraSPARC III. Estos modelos difieren primordialmente en cuanto a velocidad de reloj, pero también se añadieron algu-

nas funciones nuevas en cada iteración. En este libro, cuando tratemos la arquitectura SPARC, utilizaremos el UltraSPARC II V9 de 64 bits como ejemplo.

1.4.3 Introducción al picoJava II

El lenguaje de programación C fue inventado por Dennis Ritchie de Bell Labs para usarlo con el sistema operativo UNIX. Gracias a su diseño económico y la popularidad de UNIX, C pronto se convirtió en el lenguaje de programación dominante en el mundo para programación de sistemas. Algunos años después, Bjarne Stroustrup, también de Bell Labs, añadió a C ideas del mundo de la programación orientada a objetos para crear C++, que también se volvió muy popular.

A mediados de los noventa, investigadores de Sun Microsystems estaban explorando formas de lograr que los usuarios obtuvieran programas binarios por Internet y los ejecutaran como parte de páginas de la World Wide Web. A ellos les gustaba C++, excepto que no era seguro. En otras palabras, un programa binario en C++ recién bajado de Internet fácilmente podía espiar la máquina que lo había adquirido e interferirla de otras maneras. Su solución a este problema fue inventar un nuevo lenguaje de programación, Java, inspirado en C++ pero sin los problemas de seguridad de éste. Java es un lenguaje orientado a objetos seguro, que cada vez se usa más para muchas aplicaciones. Por ser un lenguaje popular y elegante, lo usaremos en los ejemplos de programación de este libro.

Puesto que Java no es más que un lenguaje de programación, es posible escribir compiladores de él para la arquitectura Pentium, SPARC o cualquier otra. Existen tales compiladores, pero el principal objetivo de Sun al introducir Java era poder intercambiar programas ejecutables Java entre computadoras de Internet y ejecutarlos sin modificación. Si un programa Java compilado en una máquina SPARC se enviara por Internet a una Pentium, no se ejecutaría, y no se alcanzaría la meta de poder enviar un programa binario a cualquier sitio y ejecutarlo ahí.

Para poder transportar programas binarios entre máquinas distintas, Sun definió una arquitectura de máquina virtual llamada **JVM (Java Virtual Machine)**. Esta máquina tiene una memoria formada por palabras de 32 bits, y puede ejecutar 226 instrucciones. Casi todas estas instrucciones son sencillas, pero unas cuantas son muy complejas y requieren varios ciclos de memoria.

A fin de hacer portátil a Java, Sun escribió un compilador de Java para la JVM, y también escribió un intérprete JVM para ejecutar programas binarios Java. Éste intérprete se escribió en C y por tanto puede compilarse y ejecutarse en cualquier máquina que posea un compilador de C, lo que en la práctica incluye a casi todas las máquinas del mundo. Por consiguiente, lo único que el dueño de una máquina necesita hacer para que ésta pueda ejecutar programas binarios Java es conseguir el programa binario ejecutable del intérprete de Java para esa plataforma (digamos, Pentium II y Windows 98, SPARC y UNIX, etc.) junto con ciertos programas y bibliotecas de apoyo asociados. Además, casi todos los navegadores de Internet contienen un intérprete de JVM para poder ejecutar fácilmente **applets**, que son pequeños programas binarios Java asociados a páginas de la World Wide Web. Muchos de estos applets proporcionan animación y sonido.

La interpretación de programas para JVM (o de cualesquier otros programas, para el caso) es lenta. Una estrategia alternativa a ejecutar un applet u otro programa JVM recién recibido es compilarlo primero para la máquina en cuestión y luego ejecutar el programa compilado. Esta estrategia requiere tener un compilador de JVM a lenguaje de máquina dentro del navegador y poder activarlo automáticamente cuando se necesite. Tales compiladores, llamados compiladores **JIT (justo a tiempo, Just In Time)**, existen y son comunes, pero son grandes e introducen un retraso entre la llegada del programa JVM y su ejecución mientras el programa se compila a lenguaje de máquina.

Además de las implementaciones en software de la máquina virtual Java (intérpretes de JVM y compiladores JIT), Sun y otras compañías han diseñado chips de JVM. Éstos son diseños de CPU que ejecutan directamente programas binarios JVM, sin necesidad de una capa de interpretación por software o compilación JIT. Las arquitecturas iniciales, picoJava I (O'Connor y Tremblay, 1997) y picoJava II (McGhan y O'Connor, 1998), estaban pensadas para el mercado de los sistemas incorporados. Este mercado requiere chips potentes, flexibles y sobre todo de bajo costo (menos de 50 dólares, y a veces mucho menos) que se incorporen en tarjetas inteligentes, televisores, teléfonos y otros aparatos, sobre todo los que necesitan comunicarse con el mundo exterior. Los licenciarios de Sun pueden fabricar sus propios chips utilizando el diseño picoJava, adaptándolos hasta cierto punto incluyendo o quitando la unidad de punto flotante, ajustando el tamaño de los cachés, etcétera.

El valor de un chip Java para el mercado de los sistemas incorporados es que un dispositivo puede alterar su funcionalidad mientras está operando. Por ejemplo, considere un ejecutivo de negocios que tiene un teléfono celular basado en Java y que nunca vislumbró la necesidad de leer faxes en la diminuta pantalla del teléfono, pero que repentinamente necesita hacerlo. Si llama a su proveedor celular, el ejecutivo puede bajar a su teléfono un applet para visualización de faxes y añadir esta funcionalidad al dispositivo. Las necesidades de velocidad excluyen la posibilidad de interpretar los applets Java, y la falta de memoria en el teléfono impide la compilación JIT. Ésta es una situación en la que un chip JVM es útil.

Aunque el picoJava II no es un chip concreto (no puede ir a una tienda y comprar uno), es la base de varios chips, como la CPU microJava 701 de Sun y muchos más de otros licenciarios de Sun. Utilizaremos el diseño picoJava II como uno de nuestros ejemplos constantes en todo el libro porque es muy diferente de las CPU Pentium II y UltraSPARC, y está dirigido a un área de aplicación muy diferente. Esta CPU resulta especialmente interesante para nuestros propósitos porque en el capítulo 4 presentaremos un diseño para implementar un subconjunto de JVM empleando microprogramación. Luego podremos contrastar nuestro diseño microprogramado con un verdadero diseño en hardware.

El picoJava II tiene dos unidades opcionales: un caché y una unidad de punto flotante, que cada fabricante de chips puede incluir o eliminar, a voluntad. Por sencillez, nos referiremos al picoJava II como si fuera un chip en lugar de un diseño de chip. Cuando sea importante (en contadas ocasiones), seremos específicos y hablaremos del chip microJava 701 que implementa el diseño picoJava II. Aun cuando no mencionemos el microJava 701 específicamente, el material aplicará a él y a todos los demás chips Java de otros fabricantes basados en este diseño.

Al utilizar Pentium II, UltraSPARC II y picoJava II como ejemplos, podremos estudiar tres tipos distintos de CPU. Éstos son, respectivamente, una arquitectura CISC tradicional implementada con tecnología superescalar moderna, una verdadera arquitectura RISC implementada con tecnología superescalar, y un chip Java dedicado para usarse en sistemas incorporados. Estos tres procesadores son muy diferentes entre sí, y esto nos da la oportunidad de explorar mejor el espacio de diseño y ver qué tipos de concesiones pueden hacerse para procesadores dirigidos a públicos distintos.

1.5 BOSQUEJO DEL LIBRO

Este libro trata las computadoras multinivel (término que abarca casi todas las computadoras modernas) y su organización. Examinaremos cuatro niveles con gran detalle: el nivel de lógica digital, el nivel de microarquitectura, el nivel de ISA y el nivel de sistema operativo. Entre las cuestiones básicas que examinaremos están el diseño general del nivel (y por qué se diseñó así), los tipos de instrucciones y datos con que se cuenta, la organización y direccionamiento de la memoria, y el método de implementación del nivel. El estudio de estos temas, y otros similares, se denomina organización de computadoras o arquitectura de computadoras.

Lo que más nos interesa son los conceptos, no los detalles ni las matemáticas formales. Por esa razón, algunos de los ejemplos estarán muy simplificados para hacer hincapié en las ideas centrales y no los detalles.

A fin de entender cómo se aplican en la práctica los principios presentados en este libro, utilizaremos el Pentium II, el UltraSPARC II y el picoJava II como ejemplos constantes en todo el libro. Se escogieron estos diseños por varias razones. Primera, todos se usan ampliamente y es probable que el lector tenga acceso a por lo menos uno de ellos. Segunda, cada uno tiene su propia arquitectura singular, lo que permite efectuar comparaciones y fomenta una actitud de “¿qué alternativas hay?”. Los libros que se ocupan de una sola máquina a menudo dejan al lector con la sensación de que les ha sido revelada “la máquina verdadera”, cosa que es absurda en vista de las múltiples concesiones y decisiones arbitrarias que los diseñadores se ven obligados a hacer. Recomendamos al lector estudiar éstas y todas las demás computadoras con un ojo crítico y tratar de entender por qué las cosas son como son y cómo podrían haber sido diferentes, en lugar de limitarse a aceptarlas tal cual.

Debe quedar claro desde el principio que éste no es un libro acerca de cómo programar el Pentium II, el UltraSPARC II o el picoJava II. Utilizaremos estas máquinas como ilustraciones en los puntos apropiados, pero no pretendemos presentar un tratamiento completo. Los lectores que deseen una introducción exhaustiva a cualquiera de ellas deberán consultar las publicaciones del fabricante.

El capítulo 2 es una introducción a los componentes básicos de una computadora: procesadores, memoria y equipo de entrada/salida. Su propósito es presentar un panorama general de la arquitectura de los sistemas y una introducción a los capítulos subsecuentes.

Los capítulos 3, 4, 5 y 6 se ocupan cada uno de un nivel específico de los que se muestran en la figura 1-2. Nuestro tratamiento es ascendente, porque así es como se han diseñado tradicionalmente las máquinas. El diseño del nivel k está determinado en gran medida por las propiedades del nivel $k - 1$, de modo que es difícil entender cualquier nivel si no se tiene ya

una buena comprensión del nivel subyacente que lo motivó. Además, es recomendable desde el punto de vista educativo proceder de los niveles inferiores, más sencillos, hasta los niveles superiores, más complejos, y no al revés.

El capítulo 3 se ocupa del nivel de lógica digital, el verdadero hardware de la máquina. Se explica qué son las compuertas y cómo pueden combinarse en circuitos útiles. También se presenta el álgebra booleana, una herramienta para analizar circuitos digitales. Se describen los buses de computadora, sobre todo el bus PCI tan utilizado. El capítulo examina numerosos ejemplos de la industria, incluidos los tres ejemplos constantes antes mencionados.

El capítulo 4 introduce la organización del nivel de microarquitectura y su control. Puesto que la función de este nivel es interpretar las instrucciones de nivel 2 de la capa que está arriba, nos concentraremos en este tema y lo ilustraremos con ejemplos. El capítulo también contiene descripciones del nivel de microarquitectura de algunas máquinas reales.

El capítulo 5 trata el nivel de ISA, el que la mayoría de los fabricantes de computadoras anuncia como el lenguaje de máquina. Aquí examinaremos detalladamente nuestras máquinas de ejemplo.

El capítulo 6 cubre algunas de las instrucciones, organización de memoria y mecanismos de control presentes en el nivel de sistema operativo de la máquina. Los ejemplos que se usan aquí son Windows NT (popular en sistemas servidores Pentium II del extremo superior) y UNIX, empleado en el UltraSPARC II.

El capítulo 7 habla del nivel de lenguaje ensamblador. Se cubre tanto el lenguaje ensamblador como el proceso de ensamblado. Otro tema que surge aquí es el de enlazado.

El capítulo 8 se ocupa de las computadoras paralelas, un tema cada vez más importante en nuestros días. Algunas de estas computadoras paralelas tienen múltiples CPU que comparten una misma memoria. Otras tienen varias CPU sin memoria común. Algunas son supercomputadoras; otras son COW. Todo será cubierto en este capítulo.

El capítulo 9 contiene una lista de lecturas sugeridas con anotaciones, organizada por tema, y una lista alfabética de citas bibliográficas. Éste es el capítulo más importante del libro. Úselo.

PROBLEMAS

1. Explique los siguientes términos en sus propias palabras:
 - a. Traductor.
 - b. Intérprete.
 - c. Máquina virtual.
2. ¿Qué diferencia hay entre interpretación y traducción?
3. ¿Es concebible que un compilador genere salidas para el nivel de microarquitectura en lugar del nivel ISA? Comente las ventajas y las desventajas de esta propuesta.

UNIVERSIDAD DE LA REPÚBLICA
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE
DOCUMENTACIÓN Y BIBLIOTECAS
MONTEVIDEO - URUGUAY

4. ¿Puede imaginar una computadora multinivel en la que el nivel de dispositivos y el de lógica digital no son los niveles más bajos? Explique.
5. Considere una computadora con intérpretes idénticos en los niveles 1, 2 y 3. Un intérprete requiere n instrucciones para ir a buscar, examinar y ejecutar una instrucción. Una instrucción del nivel 1 tarda k nanosegundos en ejecutarse. ¿Cuánto tarda una instrucción en los niveles 2, 3 y 4?
6. Considere una computadora multinivel en la que todos los niveles son diferentes. Cada nivel tiene instrucciones que son m veces más potentes que las del nivel inmediato inferior; es decir, una instrucción del nivel r puede efectuar el trabajo de m instrucciones del nivel $r - 1$. Si un programa del nivel 1 tarda k segundos en ejecutarse, ¿cuánto tardarían programas equivalentes en los niveles 2, 3 y 4, suponiendo que se requieren m instrucciones del nivel r para interpretar una sola instrucción del nivel $r + 1$?
7. Algunas instrucciones en el nivel de sistema operativo son idénticas a instrucciones en lenguaje ISA. El microprograma ejecuta directamente esas instrucciones, en lugar de que lo haga el sistema operativo. A la luz de su respuesta al problema anterior, ¿por qué cree que esto sea así?
8. ¿En qué sentido son equivalentes el hardware y el software? ¿En qué sentido no son equivalentes?
9. Una de las consecuencias de la idea de von Neumann de almacenar el programa en la memoria es que los programas pueden modificarse, igual que los datos. ¿Se le ocurre algún ejemplo en el que esta posibilidad podría haber sido útil? (Sugerencia: piense en realizar operaciones aritméticas con arreglos.)
10. El coeficiente de desempeño de la 360 modelo 75 es de 50 veces el de la 360 modelo 30, pero el tiempo de ciclo es sólo cinco veces más corto. ¿Cómo explica esta discrepancia?
11. En las figuras 1-5 y 1-6 se muestran dos diseños básicos de sistemas. Describa cómo podría efectuarse entrada/salida en cada sistema. ¿Cuál podría tener un mejor desempeño global?
12. En cierto momento, un transistor de un microprocesador tenía un diámetro de 1 micra. Según la ley de Moore, ¿qué tamaño tendría un transistor en el modelo del año siguiente?

2

ORGANIZACIÓN DE LOS SISTEMAS DE COMPUTADORA

Una computadora digital consiste en un sistema de procesadores interconectados, memorias y dispositivos de entrada/salida. Este capítulo es una introducción a estos tres componentes y a su interconexión, como antecedentes para el examen detallado de niveles específicos en los cinco capítulos subsecuentes. Los conceptos de procesador, memoria y entrada/salida son fundamentales y estarán presentes en todos los niveles, por lo que iniciaremos nuestro estudio de la arquitectura de las computadoras examinándolos uno por uno.

2.1 PROCESADORES

La organización de una computadora sencilla orientada hacia los buses se muestra en la figura 2-1. La **CPU (unidad central de procesamiento, Central Processing Unit)** es el “cerebro” de la computadora. Su función es ejecutar programas almacenados en la memoria principal buscando sus instrucciones y examinándolas para después ejecutarlas una tras otra. Los componentes están conectados por un **bus**, que es una colección de alambres paralelos para transmitir direcciones, datos y señales de control. Los buses pueden ser externos a la CPU, cuando la conectan a la memoria y a los dispositivos de E/S, pero también internos, como veremos en breve.

La CPU se compone de varias partes. La unidad de control se encarga de buscar instrucciones de la memoria principal y determinar su tipo. La unidad de aritmética y lógica realiza operaciones como suma y AND booleano necesarias para ejecutar las instrucciones.

La CPU también contiene una memoria pequeña y de alta velocidad que sirve para almacenar resultados temporales y cierta información de control. Esta memoria se compone de

Unidad central de procesamiento (CPU)

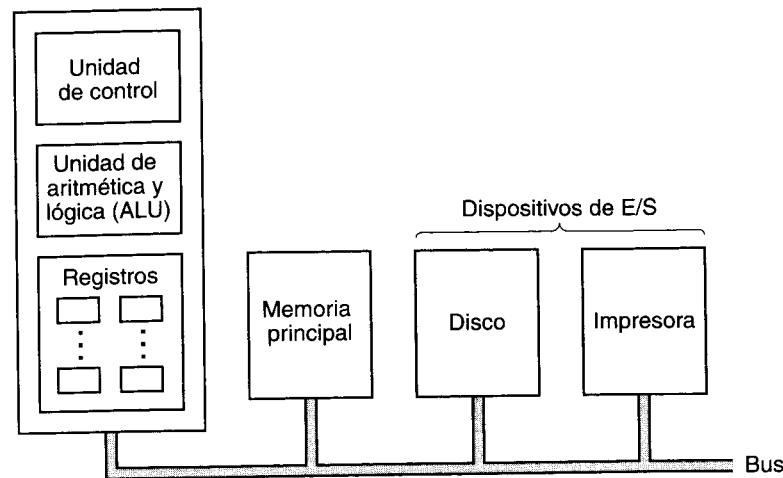


Figura 2-1. Organización de una computadora sencilla con una CPU y dos dispositivos de E/S.

varios registros, cada uno de los cuales tiene cierto tamaño y función. Por lo regular, todos los registros tienen el mismo tamaño. Cada registro puede contener un número, hasta algún máximo determinado por el tamaño del registro. Los registros pueden leerse y escribirse a alta velocidad porque están dentro de la CPU.

El registro más importante es el **contador de programa (PC, Program Counter)**, que apunta a la siguiente instrucción que debe buscarse para ejecutarse. El nombre “contador de programa” es un tanto engañoso porque no tiene nada que ver con *contar*, pero es un término de uso universal. Otro registro importante es el **registro de instrucciones (IR, Instruction Register)**, que contiene la instrucción que se está ejecutando. Casi todas las computadoras tienen varios registros más, algunos de propósito general y otros para fines específicos.

2.1.1 Organización de la CPU

En la figura 2-2 se muestra con más detalle la organización interna de una parte de una CPU von Neumann típica. Esta parte se llama **camino de datos** y consiste en los registros (generalmente del 1 al 32), la **ALU (unidad de aritmética y lógica, Arithmetic Logic Unit)** y varios buses que conectan los componentes. Los registros alimentan dos registros de entrada de la ALU, rotulados *A* y *B* en la figura. Estos registros contienen las entradas de la ALU mientras ésta está calculando. El camino de datos es muy importante en todas las máquinas y lo examinaremos con gran detalle a todo lo largo del libro.

La ALU suma, resta y realiza otras operaciones simples con sus entradas, y produce un resultado en el registro de salida. El contenido de este registro de salida se envía a un registro, que posteriormente se escribe (es decir, se guarda) en la memoria, si se

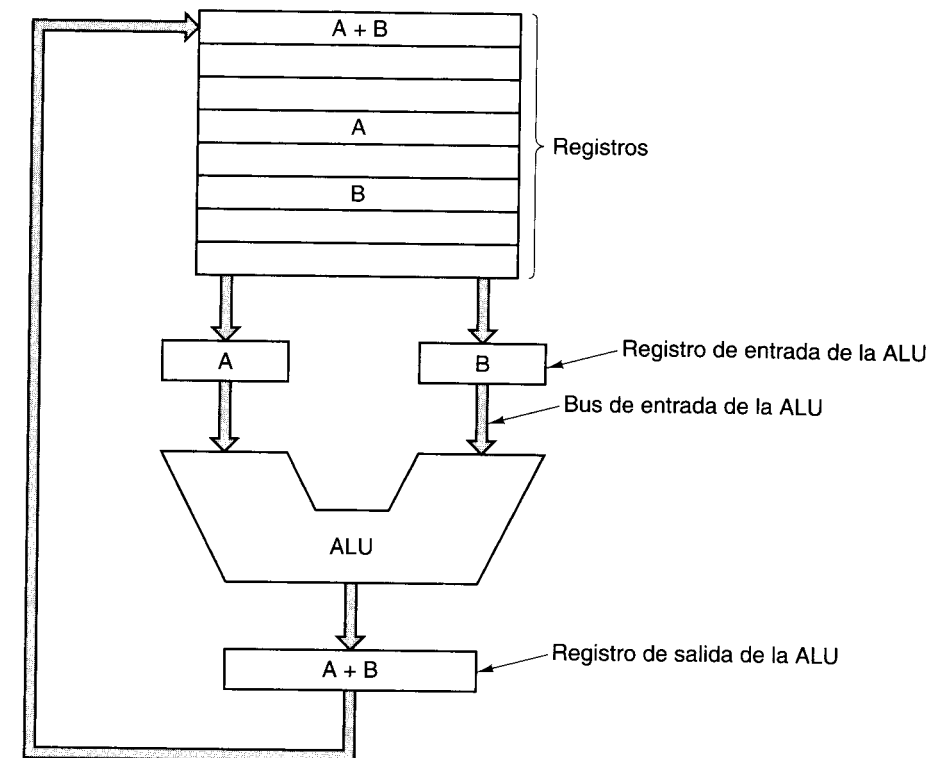


Figura 2-2. Camino de datos de una máquina von Neumann típica.

desea. No todos los diseños tienen los registros *A*, *B* y de salida. En el ejemplo se ilustra una suma.

Casi todas las instrucciones pueden dividirse en una de dos categorías: registro-memoria o registro-registro. Las instrucciones registro-memoria permiten buscar palabras de la memoria a los registros, donde pueden utilizarse como entradas de la ALU en instrucciones subsecuentes, por ejemplo. (Las “palabras” son las unidades de datos que se transfieren entre la memoria y los registros. Una palabra podría ser un entero. Trataremos la organización de la memoria más adelante en este capítulo.) Otras instrucciones registro-memoria permiten almacenar el contenido de un registro en la memoria.

La otra clase de instrucción es la de registro-registro. Una instrucción registro-registro típica busca dos operandos de los registros, los coloca en los registros de entrada de la ALU, realiza alguna operación con ellos (por ejemplo suma o AND booleano) y coloca el resultado en uno de los registros. El proceso de hacer pasar dos operandos por la ALU y almacenar el resultado se llama **ciclo del camino de datos** y es el corazón de casi todas las CPU. En gran medida, este ciclo define lo que la máquina puede hacer. Cuanto más rápido es el ciclo del camino de datos, más rápidamente opera la máquina.

2.1.2 EJECUCIÓN DE INSTRUCCIONES

La CPU ejecuta cada instrucción en una serie de pasos pequeños. A grandes rasgos, los pasos son los siguientes:

1. Buscar la siguiente instrucción de la memoria y colocarla en el registro de instrucciones.
2. Modificar el contador de programa de modo que apunte a la siguiente instrucción.
3. Determinar el tipo de la instrucción que se trajo.
4. Si la instrucción utiliza una palabra de la memoria, determinar dónde está.
5. Buscar la palabra, si es necesario, y colocarla en un registro de la CPU.
6. Ejecutar la instrucción.
7. Volver al paso 1 para comenzar a ejecutar la siguiente instrucción.

Esta sucesión de pasos se conoce como el ciclo de **búsqueda-decodificación-ejecución**, y es fundamental para el funcionamiento de todas las computadoras.

Esta descripción de cómo funciona una CPU se parece mucho a un programa escrito en español. La figura 2-3 muestra este programa informal reescrito como método Java (o sea, procedimiento) llamado *interpretar*. La máquina que se está interpretando tiene dos registros que los programas de usuario pueden ver: el contador de programa (PC), para seguir la pista a la dirección de la siguiente instrucción que debe buscarse, y el acumulador (AC), para acumular resultados aritméticos. También tiene registros internos para retener la instrucción en curso durante su ejecución (instr), el tipo de la instrucción en curso (instr_type), la dirección del operando de la instrucción (data_loc) y el operando mismo (data). Se supone que las instrucciones contienen una sola dirección de memoria. La posición de memoria direccionada contiene el operando; por ejemplo, el dato que debe sumarse al acumulador.

El simple hecho de que sea posible escribir un programa capaz de imitar la función de una CPU demuestra que un programa no tiene que ejecutarse con una CPU de “hardware” que consista en una caja llena de circuitos. En vez de ello, el programa puede ejecutarse haciendo que otro programa busque, examine y ejecute sus instrucciones. Un programa (como el de la figura 2-3) que busca, examina y ejecuta las instrucciones de otro programa se llama **intérprete**, como mencionamos en el capítulo 1.

Esta equivalencia entre procesadores en hardware e intérpretes tiene importantes implicaciones para la organización de las computadoras y el diseño de los sistemas de computación. Después de haber especificado el lenguaje de máquina, *L*, para una computadora nueva, el equipo de diseño puede decidir si quiere construir un procesador en hardware que ejecute directamente los programas en *L* o si prefieren escribir un intérprete que interprete programas en *L*. Si optan por escribir un intérprete, también deberán crear una máquina en hardware que ejecute el intérprete. También son posibles ciertas construcciones híbridas, con ejecución parcial en hardware y también algo de interpretación en software.

```
public class Interp {
    static int PC;           // contador de programa; contiene dir.de la sig. instruc.
    static int AC;           // acumulador; registro para efectuar op. aritméticas
    static int instr;        // registro para retener la instrucción en curso
    static int tipo_instr;   // el tipo de instrucción (código de operación)
    static int pos_datos;    // la dirección de los datos, o -1 si no hay datos
    static int datos;        // contiene el operando en curso
    static boolean bit_ejec = true; // bit que puede apagarse para detener la máquina

    public static void interpretar(int memoria[], int dir_inicio) {
        // Este procedimiento interpreta programas para una máquina sencilla cuyas instrucciones
        // tienen un operando en la memoria. La máquina tiene un registro AC (acumulador) que se
        // usa en las operaciones aritméticas. La instrucción ADD suma un entero de la memoria al
        // AC, por ejemplo. El intérprete continúa ejecutándose hasta que la instrucción HALT
        // apaga el bit de ejecución. El estado de un proceso que se ejecuta en esta máquina
        // consiste en la memoria, el contador de programa, el bit de ejecución y el AC. Los parámetros
        // de entrada consisten en la imagen de memoria y la dirección de inicio.

        PC = dir_inicio;
        while (bit_ejec) {
            instr = memoria[PC];           // traer siguiente instrucción a instr
            PC = PC + 1;                   // incrementar contador de programa
            tipo_instr = obt_tipo_instr(instr); // determinar tipo de instrucción
            pos_datos = encontrar_datos(instr, tipo_instr); // localizar los datos (-1 si no hay)
            if (pos_datos >= 0)             // si pos_datos es -1 no hay operando
                datos = memoria[pos_datos]; // traer los datos
            ejecutar(tipo_instr, datos);     // ejecutar la instrucción
        }

        private static int obt_tipo_instr(int direc) {...}
        private static int encontrar_datos(int instr, int tipo) {...}
        private static void ejecutar(int tipo, int datos) {...}
    }
}
```

Figura 2-3. Intérprete para una computadora simple (escrito en Java).

Un intérprete descompone las instrucciones de su máquina objetivo en pasos pequeños. Por consiguiente, la máquina en la que el intérprete se ejecuta puede ser mucho más simple y menos costosa que un procesador en hardware para la máquina objetivo. Este ahorro es más importante aún si la máquina objetivo tiene un gran número de instrucciones y éstas son complicadas, con muchas opciones. El ahorro se logra básicamente por la sustitución de hardware por software (el intérprete).

Las primeras computadoras tenían conjuntos de instrucciones pequeños y sencillos, pero la búsqueda de computadoras más potentes llevó, entre otras cosas, a instrucciones individuales más potentes. Casi desde el principio se descubrió que con instrucciones más comple-

jas a menudo era posible acelerar la ejecución de los programas, aunque las instrucciones individuales tardaran más en ejecutarse. Una instrucción de punto flotante es un ejemplo de instrucción más compleja. Otro ejemplo es el apoyo directo para acceder a los elementos de un arreglo. A veces la cosa era tan sencilla como observar que con frecuencia ocurrían las mismas dos instrucciones en sucesión, de modo que una sola instrucción podía realizar el trabajo de ambas.

Las instrucciones más complejas eran mejores porque a veces hacían posible traslapar la ejecución de operaciones individuales o ejecutarlas en paralelo empleando diferente hardware. En las costosas computadoras de alto rendimiento el costo de este hardware adicional podía justificarse fácilmente. Fue así como las computadoras de alto precio y mejor rendimiento comenzaron a tener mucho más instrucciones que las de bajo costo. Sin embargo, el creciente costo de la creación de software y las necesidades de compatibilidad de las instrucciones hicieron necesario implementar instrucciones complejas incluso en las computadoras de bajo costo en las que el precio era más importante que la velocidad.

Hacia fines de los cincuenta, IBM (que entonces era la compañía que dominaba el campo de las computadoras) se había dado cuenta de las muchas ventajas, tanto para IBM como para sus clientes, que tenía que manejar una sola familia de máquinas, todas las cuales ejecutaban las mismas instrucciones. IBM introdujo el término **arquitectura** para describir este nivel de compatibilidad. Una nueva familia de computadoras tenía una sola arquitectura pero muchas implementaciones diferentes, todas las cuales podían ejecutar el mismo programa, la única diferencia era el precio y la velocidad. Pero, ¿cómo construir una computadora de bajo costo que pudiera ejecutar todas las complejas instrucciones de las máquinas de alto costo y mayor rendimiento?

La respuesta fue la interpretación. Esta técnica, inicialmente sugerida por Wilkes (1951), permitió diseñar computadoras sencillas, de bajo costo, capaces de ejecutar un gran número de instrucciones. El resultado fue la arquitectura IBM System/360, una familia de computadoras compatibles que abarcaba casi dos órdenes de magnitud, tanto en precio como en capacidad. Sólo se usaba una implementación directa en hardware (es decir, no interpretada) en los modelos más caros.

Las computadoras sencillas con instrucciones interpretadas también tenían otros beneficios. Los más importantes eran:

1. La capacidad para corregir en el campo instrucciones mal implementadas, o incluso subsanar deficiencias de diseño en el hardware básico.
2. La oportunidad de añadir nuevas instrucciones con un costo mínimo, aun después de haber entregado la máquina.
3. Diseño estructurado que permite crear, probar y documentar instrucciones complejas de forma eficiente.

Cuando el mercado de las computadoras hizo explosión en la década de los setenta, y la capacidad de las máquinas creció rápidamente, la demanda de computadoras de bajo costo favoreció los diseños que utilizaban intérpretes. La capacidad para adaptar el hardware y el intérprete a un conjunto de instrucciones específico surgió como un diseño mucho más eco-

nómico para los procesadores. Al avanzar a pasos agigantados la tecnología subyacente de los semiconductores, las ventajas de costo sobrepasaron las oportunidades de lograr mayor rendimiento, y las arquitecturas basadas en intérpretes se convirtieron en la forma convencional de diseñar computadoras. Casi todas las computadoras nuevas diseñadas en los setenta, desde las minicomputadoras hasta las mainframes, se basaron en interpretación.

Hacia finales de esa década, el uso de procesadores simples que ejecutaban intérpretes se había extendido a todos los modelos, con excepción de los más costosos y rápidos como la Cray-1 y la serie Cyber de Control Data. El uso de un intérprete eliminaba las limitaciones de costo inherentes a las instrucciones complejas, y las arquitecturas comenzaron a explorar instrucciones más complejas, sobre todo las formas de especificar los operandos a utilizar.

La tendencia llegó a su cenit con la computadora VAX de Digital Equipment Corporation, que tenía varios cientos de instrucciones y más de 200 formas distintas de especificar los operandos a usar en cada instrucción. Desafortunadamente, la arquitectura VAX se concibió desde el principio para implementarse con un intérprete, sin pensar mucho en la implementación de un modelo de alto rendimiento. Esta filosofía llevó a la inclusión de un gran número de instrucciones de valor marginal que eran difíciles de ejecutar de manera directa. Esta omisión resultó fatal para VAX, y en última instancia también para DEC (Compaq compró a DEC en 1998).

Si bien los primeros microprocesadores de 8 bits eran máquinas muy sencillas con conjuntos de instrucciones muy simples, para fines de la década de los años setenta incluso los microprocesadores habían adoptado diseños basados en intérpretes. Durante este periodo, uno de los principales retos que los diseñadores de microprocesadores enfrentaban era controlar la creciente complejidad que los circuitos integrados hacían posible. Una ventaja importante del enfoque basado en intérpretes era la posibilidad de diseñar un procesador sencillo, dejando casi todo lo complejo para la memoria que contenía el intérprete. Así, un diseño de hardware complejo podía convertirse en un diseño de software complejo.

El éxito del Motorola 68000, que tenía un conjunto de instrucciones grande interpretado, y el fracaso concurrente del Zilog Z8000 (que tenía un conjunto de instrucciones igualmente grande, pero sin intérprete) demostró las ventajas de los intérpretes para sacar al mercado rápidamente un microprocesador nuevo. Este éxito fue más sorprendente en vista de la venta inicial de Zilog (el predecesor del Z8000, el Z80, era mucho más popular que el predecesor del 68000, el 6800). Desde luego, otros factores influyeron en este desenlace, como la larga historia de Motorola como fabricante de chips y la larga historia de Exxon (el dueño de Zilog) como compañía petrolera, no como fabricante de chips.

Otro factor que actuaba en favor de la interpretación durante esa era fue la existencia de memorias sólo de lectura rápidas, llamadas **almacenes de control**, para contener los intérpretes. Supongamos que una instrucción interpretada representativa del 68000 requiriera 10 instrucciones del intérprete, llamadas **microinstrucciones**, a 100 ns cada una, y dos referencias a la memoria principal, a 500 ns cada una. El tiempo de ejecución total era entonces de 2000 ns, sólo el doble de la rapidez máxima que podía lograrse con ejecución directa. Si no se contara con almacén de control, la instrucción habría tardado 6000 ns. Una reducción en la velocidad por un factor de 6 es mucho más difícil de aceptar que una reducción por un factor de 2.

2.1.3 RISC versus CISC

A finales de los setenta se efectuaron muchos experimentos con instrucciones muy complejas, que eran posibles gracias al intérprete. Los diseñadores trataron de salvar la “brecha semántica” entre lo que las máquinas podían hacer y lo que los lenguajes de programación de alto nivel requerían. A casi nadie se le ocurría diseñar máquinas más sencillas, como ahora casi nadie piensa en diseñar sistemas operativos, redes, procesadores de textos, etc. menos potentes (lo cual tal vez es lamentable).

Un grupo que se opuso a la tendencia y trató de incorporar algunas de las ideas de Seymour Cray en una minicomputadora de alto rendimiento fue el encabezado por John Cocke en IBM. Estos trabajos tuvieron como resultado una minicomputadora experimental llamada **801**. Aunque IBM nunca comercializó esta máquina y los resultados no se publicaron sino hasta varios años después (Radin, 1982), la información se filtró y otras personas comenzaron a investigar arquitecturas similares.

En 1980, un grupo de Berkeley dirigido por David Patterson y Carlo Séquin comenzó a diseñar chips de CPU VLSI que no utilizaban interpretación (Patterson, 1985; Patterson y Séquin, 1982). Ellos acuñaron el término **RISC** para este concepto y llamaron a su chip de CPU RISC I, el cual fue seguido en poco tiempo por el RISC II. Un poco después, en 1981, del otro lado de la Bahía de San Francisco en Stanford, John Hennessy diseñó y fabricó un chip un tanto diferente al que llamó **MIPS** (Hennessy, 1984). Estos chips llegaron a convertirse en productos importantes desde el punto de vista comercial: SPARC y MIPS, respectivamente.

Estos nuevos procesadores tenían diferencias significativas respecto a los procesadores comerciales de la época. Dado que estas nuevas CPU no tenían que ser compatibles con productos existentes, sus diseñadores estaban en libertad de escoger nuevos conjuntos de instrucciones que maximizaran el rendimiento total del sistema. Si bien el hincapié inicial fue en instrucciones simples que pudieran ejecutarse con rapidez, pronto se vio que la clave para un buen desempeño era diseñar instrucciones que pudieran **emitirse** (iniciarse) rápidamente. El tiempo real que una instrucción tardaba era menos importante que el número de instrucciones que podían iniciarse por segundo.

En la época en que se estaban diseñando por primera vez estos sencillos procesadores la característica que llamó la atención de todo mundo fue el número relativamente pequeño de instrucciones disponibles, por lo regular unas 50. Este número era mucho menor que las 200 a 300 que tenían computadoras establecidas como la DEC VAX y las grandes mainframes de IBM. De hecho, el acrónimo RISC significa **computadora de conjunto de instrucciones reducido** (*Reduced Instruction Set Computer*), lo que contrasta con CISC, que significa **computadora de conjunto de instrucciones complejo** (*Complex Instruction Set Computer*) (una referencia poco sutil al VAX, que dominaba los departamentos de ciencias de la computación universitarios en ese entonces). Hoy día, poca gente piensa que el tamaño del conjunto de instrucciones sea crucial, pero el nombre ha persistido.

Para no hacer muy larga la historia, se desencadenó una gran guerra de religión, con los partidarios de RISC atacando el orden establecido (VAX, Intel, las grandes mainframes de IBM). Ellos aseguraban que la mejor forma de diseñar una computadora era tener un número

reducido de instrucciones simples que se ejecutaran en un ciclo del camino de datos de la figura 2-2, buscar dos registros, combinarlos de alguna manera (por ejemplo, suma o AND) y almacenar el resultado en un registro nuevamente. Su argumento era que incluso si una máquina RISC requiriera cuatro o cinco instrucciones para hacer lo que una máquina CISC hace en una instrucción, si las instrucciones RISC son 10 veces más rápidas (porque no se interpretan), RISC gana. También vale la pena señalar que para esas fechas la rapidez de las memorias principales había alcanzado a la rapidez de los almacenes de control sólo de lectura, así que el castigo por interpretación se había incrementado considerablemente, lo que favorecía a las máquinas RISC.

Podríamos pensar que en vista de las ventajas de velocidad de la tecnología RISC, las máquinas RISC (como la Alpha de DEC) habrían sacado a las máquinas CISC (como la Intel Pentium) del mercado. No ha sucedido tal cosa. ¿Por qué?

En primer lugar, está la cuestión de la compatibilidad con lo existente y los miles de millones de dólares que las compañías han invertido en software para la línea Intel. Segundo, aunque parezca sorprendente, Intel ha logrado aplicar las mismas ideas incluso en una arquitectura CISC. A partir del 486, las CPU de Intel contienen un núcleo RISC que ejecuta las instrucciones más simples (y casi siempre las más comunes) en un solo ciclo del camino de datos, al tiempo que interpreta las instrucciones más complejas de la forma CISC acostumbrada. El resultado neto es que las instrucciones comunes son rápidas, y las menos comunes son lentas. Si bien este enfoque híbrido no es tan rápido como un diseño RISC puro, ofrece un desempeño global competitivo y al mismo tiempo permite la ejecución de software viejo sin modificación.

2.1.4 Principios de diseño de las computadoras modernas

Ahora, más de una década después de la introducción de las primeras máquinas RISC, ciertos principios de diseño gozan de aceptación como una buena forma de diseñar computadoras dado el estado actual de la tecnología de hardware. Si ocurre un cambio importante en la tecnología (digamos que un nuevo proceso de fabricación hace que un ciclo de memoria tarde 10 veces menos que un ciclo de CPU), cualquier cosa podría suceder. Por tanto, los diseñadores de máquinas siempre deben estar pendientes de los avances tecnológicos que podrían afectar el equilibrio entre los componentes.

Habiendo dicho eso, existe un conjunto de principios de diseño, a veces llamados **principios de diseño RISC**, que los arquitectos de las CPU de propósito general hacen lo posible por seguir. Restricciones externas, como el requisito de ser compatible con alguna arquitectura existente, a menudo requieren concesiones ocasionales, pero estos principios son metas que la mayoría de los diseñadores se fija. A continuación analizaremos los principios más importantes.

Todas las instrucciones se ejecutan directamente en hardware

El hardware ejecuta directamente todas las instrucciones comunes; éstas no se interpretan con microinstrucciones. La eliminación de un nivel de interpretación hace que la mayor parte

de las instrucciones sean rápidas. En las computadoras que implementan conjuntos de instrucciones CISC, las instrucciones más complejas pueden dividirse en partes discretas que luego pueden ejecutarse como una sucesión de microinstrucciones. Este paso adicional hace más lenta a la máquina, pero podría ser aceptable en el caso de instrucciones que ocurren con poca frecuencia.

Maximizar el ritmo con que se emiten instrucciones

Las computadoras modernas recurren a muchos trucos para maximizar su desempeño, y el principal de ellos es tratar de emitir el mayor número posible de instrucciones por segundo. Después de todo, si podemos emitir 500 millones de instrucciones por segundo, habremos construido un procesador de 500 MIPS, sin importar cuánto tarden en ejecutarse realmente las instrucciones. (MIPS significa Millones de Instrucciones Por Segundo; el procesador MIPS pretende hacer un juego de palabras con este acrónimo.) Este principio sugiere que el paralelismo puede desempeñar un papel importante en el mejoramiento del desempeño, ya que sólo es posible emitir un gran número de instrucciones lentas en un periodo corto si varias instrucciones pueden ejecutarse simultáneamente.

Aunque las instrucciones siempre se encuentran en el orden que dicta el programa, no siempre se emiten en ese orden (porque algún recurso necesario podría estar ocupado) y no tienen que terminar en el orden del programa. Desde luego, si la instrucción 1 asigna un valor a un registro y la instrucción 2 usa ese registro, debe tenerse mucho cuidado de que la instrucción 2 no lea el registro antes de que éste contenga el valor correcto. Para ello se requiere un control estricto, pero el potencial para incrementar la rapidez ejecutando varias instrucciones simultáneamente es grande.

Las instrucciones deben ser fáciles de decodificar

Un límite crítico de la rapidez con que se emiten las instrucciones es la decodificación de instrucciones individuales para determinar qué recursos necesitan. Todo lo que pueda agilizar este proceso es útil, e incluye hacer que las instrucciones tengan una longitud fija, con un número pequeño de campos. Cuanto menor sea el número de formatos de instrucciones distintos, mejor.

Sólo las operaciones de carga y almacenamiento deben hacer referencia a la memoria

Una de las formas más sencillas de desglosar las operaciones en pasos individuales es requerir que los operandos de casi todas las instrucciones provengan de —y regresen a—, registros. La operación de transferir operandos entre la memoria y los registros puede realizarse con instrucciones diferentes. Puesto que el acceso a la memoria puede tardar mucho, y el retraso es impredecible, la mejor manera de traslapar estas instrucciones con otras es cerciorarse de que no hagan más que transferir operandos entre los registros y la memoria. Esta observación implica que sólo las instrucciones LOAD y STORE deben hacer referencia a la memoria.

Incluir abundantes registros

Puesto que el acceso a la memoria es relativamente lento, es necesario contar con muchos registros (al menos 32) para que, una vez que se ha obtenido una palabra, se pueda mantener en un registro hasta que ya no se necesite. Quedarse sin registros y tener que escribirlos en la memoria sólo para volverlos a cargar después es poco recomendable y debe evitarse en la medida de lo posible. La mejor manera de lograr esto es tener suficientes registros.

2.1.5 Paralelismo en el nivel de instrucciones

Los arquitectos de computadoras se esfuerzan constantemente por mejorar el desempeño de las máquinas que diseñan. Una forma de hacerlo es aumentar la velocidad de reloj de los chips para que operen con mayor rapidez, pero para cada nuevo diseño existe un límite para lo que es posible lograr con los recursos disponibles en ese momento histórico. Por consiguiente, casi todos los arquitectos de computadoras recurren al paralelismo (hacer dos o más cosas al mismo tiempo) para sacar el mayor provecho posible a una velocidad de reloj dada.

El paralelismo adopta dos formas generales: paralelismo en el nivel de instrucciones y paralelismo en el nivel de procesador. En el primero, se aprovecha el paralelismo dentro de las instrucciones individuales para lograr que la máquina ejecute más instrucciones por segundo. En el segundo, múltiples CPU trabajan juntas en el mismo problema. Cada enfoque tiene sus propios méritos. En esta sección examinaremos el paralelismo en el nivel de instrucciones; en la que sigue veremos el paralelismo en el nivel de procesadores.

Filas de procesamiento

Se ha sabido desde hace años que la obtención de instrucciones de la memoria es un importante cuello de botella que afecta la rapidez de ejecución de las instrucciones. Para aliviar este problema, computadoras que se remontan por lo menos a la IBM Stretch (1959) han contado con la capacidad de buscar instrucciones de la memoria por adelantado, a fin de tenerlas disponibles en el momento en que se necesitan. Esas instrucciones se almacenaban en una serie de registros llamados **buffer de prebúsqueda**. Así, cuando se requería una instrucción, casi siempre podía tomarse del buffer de prebúsqueda en lugar de esperar a que terminara una lectura de la memoria.

Efectivamente, la prebúsqueda divide la ejecución de instrucciones en dos partes: búsqueda y ejecución propiamente dicha. El concepto de **fila de procesamiento (pipeline)** lleva esta estrategia mucho más lejos. En lugar de dividir la ejecución de instrucciones en sólo dos partes, a menudo se divide en muchas partes, cada una de las cuales se maneja con un componente de hardware dedicado, y todos estos componentes pueden operar en paralelo.

La figura 2-4(a) ilustra una fila de procesamiento con cinco unidades, también llamadas **etapas**. La etapa 1 busca la instrucción de la memoria y la coloca en un buffer hasta que se necesita. La etapa 2 decodifica la instrucción, determinando de qué tipo es y qué operandos necesita. La etapa 3 localiza y busca los operandos, sea de registros o de la memoria. La etapa 4 se encarga de ejecutar propiamente la instrucción, casi siempre haciendo pasar los operandos

por el camino de datos de la figura 2-2. Por último, la etapa 5 escribe el resultado en el registro apropiado.

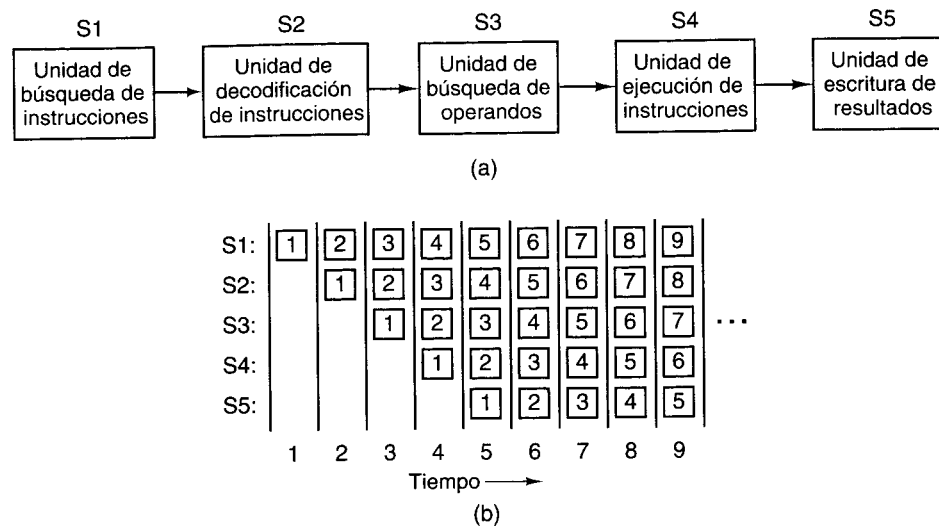


Figura 2-4. (a) Fila de procesamiento de cinco etapas. (b) Estado de cada etapa en función del tiempo. Se ilustran nueve ciclos de reloj.

En la figura 2-4(b) vemos cómo la fila de procesamiento opera como una función del tiempo. Durante el ciclo de reloj 1, la etapa S1 está trabajando en la instrucción 1, buscándola en la memoria. Durante el ciclo 2, la etapa S2 decodifica la instrucción 1, mientras la etapa S1 busca la instrucción 2. Durante el ciclo 3, la etapa S3 busca los operandos para la instrucción 1, la etapa S2 decodifica la instrucción 2 y la etapa S1 busca la tercera instrucción. Durante el ciclo 4, la etapa S4 ejecuta la instrucción 1, S3 busca los operandos para la instrucción 2, S2 decodifica la instrucción 3 y S1 busca la instrucción 4. Por último, durante el ciclo 5, S5 escribe el resultado de la instrucción 1, mientras las demás etapas trabajan con las instrucciones subsiguientes.

Consideremos una analogía para aclarar el concepto de fila de procesamiento. Imagine una fábrica de pasteles en la que se separa la preparación de los pasteles y el empaquetado de éstos para su distribución. Supongamos que el departamento de empaquetado tiene una larga banda transportadora con cinco trabajadores (unidades de procesamiento) formados junto a ella. Cada 10 segundos (el ciclo de reloj), el trabajador 1 coloca una caja de pastel vacía en la banda. La caja es transportada hasta el trabajador 2, quien coloca un pastel en ella. Poco después, la caja llega al puesto del trabajador 3, donde se le cierra y sella. Luego, la caja continúa hasta donde está el trabajador 4, quien adhiere una etiqueta a la caja. Por último, el trabajador 5 retira la caja de la banda y la coloca en un contenedor grande que después se enviará a un supermercado. Básicamente, ésta es la forma en que funcionan también las filas de procesamiento de una computadora: cada instrucción (pastel) pasa por varios pasos de procesamiento antes de salir completada por el otro extremo.

Volviendo a nuestra fila de procesamiento de la figura 2-4, supongamos que el tiempo de ciclo de esta máquina es de 2 ns. Entonces, se requieren 10 ns para que una instrucción pase por las cinco etapas de la fila de procesamiento. A primera vista, si una instrucción tarda 10 ns, podría parecer que la máquina puede operar a 100 MIPS, pero en realidad trabaja a una velocidad mucho mayor. En cada ciclo de reloj (2 ns), se completa una instrucción, así que la rapidez de procesamiento real es de 500 MIPS, no de 100 MIPS.

El uso de filas de procesamiento permite balancear la **latencia** (el tiempo que tarda en ejecutarse una instrucción) y el **ancho de banda del procesador** (cuántas MIPS puede ejecutar la CPU). Con un tiempo de ciclo de T ns, y n etapas en la fila de procesamiento, esta latencia es de nT ns y el ancho de banda es de $1000/T$ MIPS (lógicamente, dado que estamos midiendo tiempos en nanosegundos, deberíamos medir el ancho de banda de la CPU en MMIPS o GIPS, pero nadie lo hace, así que tampoco lo haremos aquí).

Arquitecturas superescalares

Si una fila de procesamiento es buena, entonces seguramente dos serán mejores. En la figura 2-5 se muestra un posible diseño para una CPU con fila de procesamiento dual basado en la figura 2-4. Aquí una sola unidad de búsqueda de instrucciones trae pares de instrucciones y coloca cada una en su propia fila de procesamiento, que cuenta con su propia ALU para poder operar en paralelo. Esto requiere que las dos instrucciones no compitan por el uso de recursos (por ejemplo, registros) y también que una no dependa del resultado de la otra. Al igual que con una sola fila de procesamiento, el compilador debe garantizar que se cumpla esta situación (lo que implica que el hardware no verifica que las instrucciones sean compatibles, y produce resultados incorrectos si no lo son), o bien los conflictos se detectan y eliminan durante la ejecución utilizando hardware adicional.

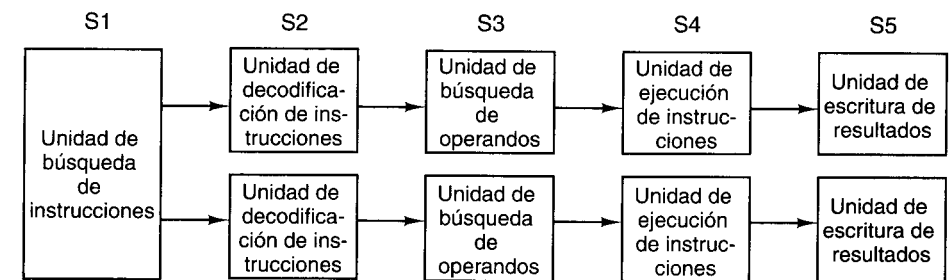


Figura 2-5. Filas de procesamiento dual de cinco etapas con unidad de búsqueda de instrucciones común.

Aunque las filas de procesamiento, sean sencillas o dobles, se usan principalmente en las máquinas RISC (el 386 y sus predecesores no los tenían), a partir del 486 Intel comenzó a introducir filas de procesamiento en sus CPU. El 486 tenía una fila de procesamiento y el Pentium tenía dos filas de procesamiento de cinco etapas más o menos como en la figura 2-5, aunque la división exacta del trabajo entre las etapas 2 y 3 (llamadas decodificar-1 y decodificar-2) era un poco diferente de nuestro ejemplo. La fila de procesamiento principal, llamada **fila de**

procesamiento u, podía ejecutar una instrucción Pentium arbitraria. La segunda fila de procesamiento, llamada **fila de procesamiento v**, sólo podía ejecutar instrucciones enteras simples (y una instrucción de punto flotante sencilla, FXCH).

Unas reglas complejas determinaban si un par de instrucciones eran compatibles y podían ejecutarse en paralelo. Si las instrucciones de un par no eran suficientemente simples o eran incompatibles, sólo se ejecutaba la primera (en la fila de procesamiento u). La segunda se retenía y se apareaba con la siguiente instrucción. Las instrucciones siempre se ejecutaban en orden. Así, compiladores específicos para Pentium que producían pares compatibles podían generar programas de más rápida ejecución que los compiladores más antiguos. Mediciones efectuadas revelaron que un Pentium que ejecutaba código optimado para él era exactamente dos veces más rápido con programas de manipulación de enteros que un 486 que operaba con la misma rapidez de reloj (Pountain, 1993). Esta ganancia podía atribuirse totalmente a la segunda fila de procesamiento.

Es concebible usar cuatro filas de procesamiento, pero ello implica duplicar demasiado hardware (los computólogos, a diferencia de los especialistas en el folklore, no creen en el número tres). En vez de ello, en las CPU de extremo superior se sigue una estrategia distinta. La idea básica es tener una sola fila de procesamiento pero proporcionarle varias unidades funcionales, como se muestra en la figura 2-6. Por ejemplo, el Pentium II tiene una estructura similar a esta figura, y la veremos en el capítulo 4. Se acuñó el término **arquitectura superescalar** en 1987 para describir este enfoque (Agerwala y Cocke, 1987). Sin embargo, sus raíces se remontan a hace más de 30 años con la computadora CDC 6600. La 6600 obtenía una instrucción cada 100 ns y la transfería a una de 10 unidades funcionales para su ejecución en paralelo mientras la CPU iba en busca de la siguiente instrucción.

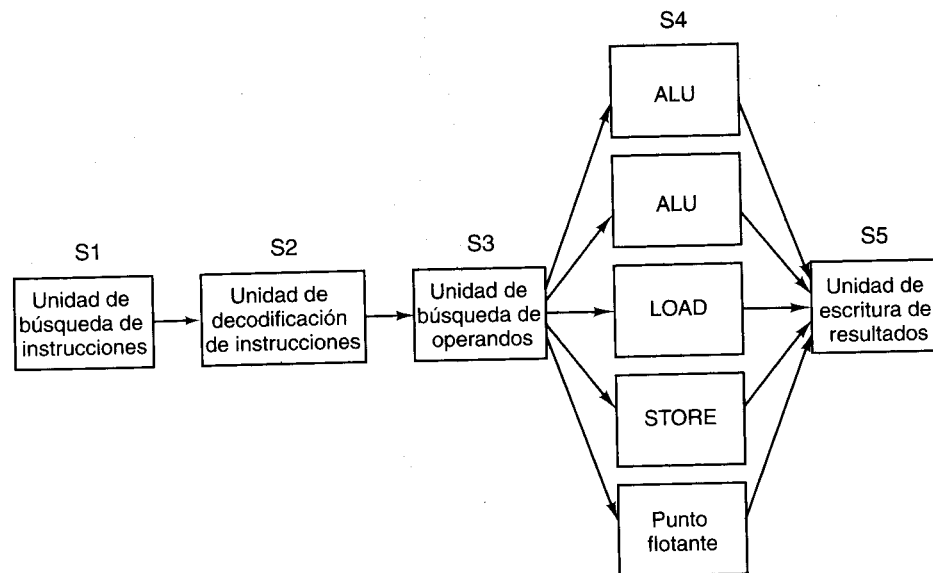


Figura 2-6. Procesador superescalar con cinco unidades funcionales.

El concepto de procesador superescalar lleva implícita la idea de que la etapa S3 puede emitir instrucciones con mucho mayor rapidez de la que la etapa S4 puede ejecutarlas. Si la etapa S3 emitiera una instrucción cada 10 ns y todas las unidades funcionales pudieran efectuar su trabajo en 10 ns, sólo una estaría ocupada en un momento dado, y la idea no representaría ninguna ventaja. En realidad, casi todas las unidades funcionales de la etapa 4 tardan mucho más que un ciclo de reloj en ejecutarse, sobre todo las que acceden a la memoria o realizan aritmética de punto flotante. Como puede verse en la figura, es posible tener varios ALU en la etapa 4.

2.1.6 Paralelismo en el nivel de procesador

La demanda de computadoras más y más rápidas parece insaciable. Los astrónomos quieren simular qué sucedió en el primer microsegundo después del *big bang*, los economistas quieren modelar la economía mundial, y los adolescentes quieren tener juegos multimedia interactivos en 3D por Internet con sus amigos virtuales. Si bien las CPU son cada vez más rápidas, llegará el momento en que se toparán con la barrera de la velocidad de la luz, que con toda seguridad no podrá rebasar los 20 cm/ns en alambre de cobre o fibra óptica, por más ingeniosos que sean los ingenieros de Intel. Los chips más rápidos también producen más calor, y su disipación es un problema.

El paralelismo en el nivel de instrucciones ayuda un poco, pero las filas de procesamiento y el funcionamiento superescalar casi nunca ganan más que un factor de 5 o 10. Para obtener ganancias de 50, 100 o más, el único camino es diseñar computadoras con múltiples CPU, por lo que ahora veremos cómo están organizadas algunas de ellas.

Computadoras de matriz

Muchos problemas de la ciencia física e ingeniería implican matrices o tienen una estructura altamente regular en algún sentido. En muchos casos los mismos cálculos se efectúan con muchos conjuntos de datos distintos al mismo tiempo. La regularidad y estructura de estos programas los convierte en candidatos idóneos para una aceleración por ejecución en paralelo. Se han utilizado dos métodos para ejecutar programas científicos grandes rápidamente. Si bien estos dos sistemas son muy similares en casi todos los sentidos, resulta irónico que uno de ellos se considere como una extensión de un solo procesador, mientras que el otro se considera como una computadora paralela.

Un **arreglo de procesadores** consiste en un gran número de procesadores idénticos que ejecutan la misma secuencia de instrucciones con diferentes conjuntos de datos. El primer arreglo de procesadores del mundo fue la computadora ILLIAC IV de la University of Illinois, que se ilustra en la figura 2-7 (Bouknight *et al.*, 1972). El plan original era construir una máquina constituida por cuatro cuadrantes, cada uno de los cuales tenía una cuadrícula 8×8 de elementos procesador/memoria. Una sola unidad de control por cuadrante transmitía instrucciones que eran ejecutadas simultáneamente por todos los procesadores, cada uno de los cuales utilizaba sus propios datos de su propia memoria (que se cargaba durante la fase de inicialización). Debido a que el presupuesto se excedió en un factor de cuatro, sólo llegó a

construirse un cuadrante, pero éste logró un desempeño de 50 megaflops (millones de operaciones de punto flotante por segundo). Se dice que si se hubiera completado la máquina original y hubiera alcanzado la meta de desempeño fijada originalmente (1 gigaflop), habría duplicado la capacidad de cómputo de todo el mundo.

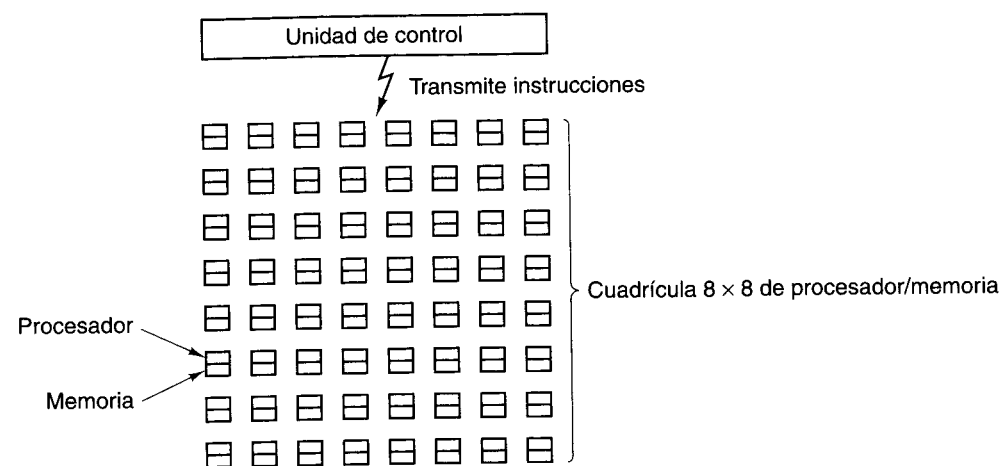


Figura 2-7. Arreglo de procesadores del tipo ILLIAC IV.

Para el programador, un **procesador vectorial** es muy parecido a un arreglo de procesadores. Al igual que éste, el procesador vectorial es muy eficiente al ejecutar una secuencia de operaciones con pares de elementos de datos. Pero, a diferencia del arreglo de procesadores, todas las operaciones de suma se efectúan en un solo sumador provisto de muchas fila de procesamientos. La compañía fundada por Seymour Cray, Cray Research, produjo muchos procesadores vectoriales, comenzando con la Cray-1 en 1974 y continuando hasta los modelos actuales (Cray Research forma parte ahora de SGI).

Tanto los arreglos de procesadores como los procesadores vectoriales trabajan con arreglos de datos. Ambos ejecutan instrucciones individuales que, por ejemplo, suman los elementos de dos vectores en pares. Pero mientras que el arreglo de procesadores lo hace teniendo tantos sumadores como elementos tiene el vector, el procesador vectorial tiene el concepto de **registro vectorial**, que consiste en un conjunto de registros convencionales que pueden cargarse desde la memoria con una sola instrucción, la cual en realidad los carga desde la memoria en serie. Luego una instrucción de suma vectorial realiza la suma por pares de los elementos de dos vectores de este tipo alimentándolos a un sumador con filas de procesamiento desde los dos registros vectoriales. El resultado del sumador es otro vector, que puede almacenarse en un registro vectorial o utilizarse directamente como operando de otra operación vectorial.

Se siguen fabricando arreglos de procesadores, pero ocupan un nicho cada vez más pequeño del mercado, ya que sólo funcionan bien con problemas que requieren realizar el mismo cálculo con muchos conjuntos de datos simultáneamente. Los arreglos de procesadores pueden realizar algunas operaciones de forma más eficiente que las computadoras vectoriales, pero requieren mucho más hardware y su programación es notoriamente difícil. Los

procesadores vectoriales, en cambio, pueden añadirse a un procesador convencional. El resultado es que las partes del programa susceptibles de **vectorizarse** pueden ejecutarse rápidamente aprovechando la unidad vectorial, mientras que el resto del programa puede ejecutarse en un procesador único convencional.

Multiprocesadores

Los elementos de procesamiento de un arreglo de procesadores no son CPU independientes, ya que todos ellos comparten la misma unidad de control. Nuestro primer sistema paralelo constituido por CPU propiamente dichas es el **multiprocesador**, un sistema con varias CPU que comparten una memoria común, como un grupo de personas en un salón que comparten un pizarrón común. Puesto que cada CPU puede leer o escribir en cualquier parte de la memoria, deben coordinarse (en software) para no estorbarse mutuamente.

Hay varios posibles esquemas de implementación. El más sencillo consiste en tener un solo bus con varias CPU y una memoria conectadas a él. En la figura 2-8(a) se muestra un diagrama de semejante multiprocesador basado en bus. Muchas compañías fabrican este tipo de sistemas.

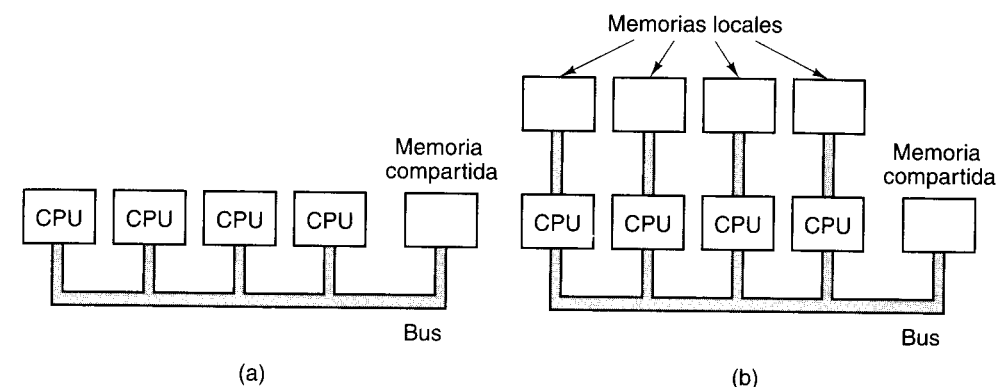


Figura 2-8. (a) Multiprocesador con un solo bus. (b) Multicomputadora con memorias locales.

No se necesita mucha imaginación para darse cuenta de que si hay un gran número de procesadores rápidos que constantemente están tratando de acceder a la memoria por el mismo bus, habrá conflictos. Los diseñadores de multiprocesadores han ideado varios esquemas para reducir esta contención y mejorar el desempeño. Un diseño, que se muestra en la figura 2-8(b), proporciona a cada procesador un poco de memoria local propia, inaccesible para los demás. Esta memoria puede utilizarse para código de programa y datos que no es necesario que se compartan. El acceso a esta memoria privada no usa el bus principal, lo que reduce considerablemente el tráfico en el bus. Hay otros posibles esquemas (por ejemplo, uso de cachés).

Los multiprocesadores tienen la ventaja, respecto a otros tipos de computadoras paralelas, de que es fácil trabajar con el modelo de programación de una sola memoria compartida. Por ejemplo, imagine un programa que busca células cancerosas en una fotografía de algún tejido

tomada a través de un microscopio. La fotografía digitalizada podría guardarse en la memoria común, y a cada procesador se le asignaría alguna región de la fotografía para su búsqueda. Puesto que cada procesador tiene acceso a toda la memoria, uno de ellos no tiene problemas para estudiar una célula que comienza en la región que se le asignó pero que rebasa la frontera con la siguiente región.

Multicomputadoras

Aunque los multiprocesadores con un número reducido de procesadores (≤ 64) son relativamente fáciles de construir, las dificultades se multiplican de forma sorprendente cuando aumenta el número de procesadores. Lo difícil es conectar todos los procesadores a la memoria. Para superar estos problemas, muchos diseñadores simplemente han abandonado la idea de tener una memoria compartida y se han limitado a construir sistemas que consisten en un gran número de computadoras interconectadas, cada una de las cuales tiene su propia memoria, sin que haya una memoria común. Estos sistemas se llaman **multicomputadoras**.

Las CPU de una multicomputadora se comunican enviándose mutuamente mensajes, parecidos al correo electrónico pero mucho más rápidos. En sistemas grandes, tener cada computadora conectada a todas las demás no resulta práctico, y se utilizan topologías como retículas bi y tridimensionales, árboles y anillos. El resultado es que los mensajes de una computadora a otra a menudo tienen que pasar por una o más computadoras intermedias o computadores para llegar del origen al destino. No obstante, es posible alcanzar sin mucha dificultad tiempos de paso de mensajes del orden de unos cuantos microsegundos. Se han construido multicomputadoras con casi 10,000 CPU, y ya están funcionando.

Puesto que los multiprocesadores son más fáciles de programar y las multicomputadoras son más fáciles de construir, se ha investigado asiduamente la posibilidad de diseñar sistemas híbridos que combinen las propiedades más atractivas de cada uno. Tales computadoras tratan de presentar la ilusión de una memoria compartida, sin hacer el gasto de construirla realmente. Estudiaremos con detalle los multiprocesadores y las multicomputadoras en el capítulo 8.

2.2 MEMORIA PRIMARIA

La **memoria** es la parte de la computadora en la que se almacenan programas y datos. Algunos especialistas en computación (sobre todo los británicos) emplean el término **almacén** (store) o **almacenamiento** (storage) en lugar de memoria (memory), aunque cada vez se extiende más el uso de "almacenamiento" para referirse al almacenamiento en disco. Sin una memoria en la cual los procesadores puedan leer y escribir información, no existirían las computadoras digitales de programa almacenado.

2.2.1 Bits

La unidad básica de memoria es el dígito binario, llamado **bit**. Un bit puede contener un 0 o un 1; es la unidad más simple posible. (Un dispositivo que sólo pudiera almacenar ceros difícilmente podría servir para construir un sistema de memoria; se requieren por lo menos dos valores.)

Suele decirse que las computadoras emplean aritmética binaria porque es "eficiente". Lo que quiere decirse (aunque pocas veces se es consciente de ello) es que es posible almacenar información digital distinguiendo entre diferentes valores de alguna cantidad física continua, como un voltaje o corriente. Cuantos más valores sea necesario distinguir, y menor separación haya entre valores adyacentes, menos confiable será la memoria. El sistema de numeración binario sólo requiere distinguir entre dos valores; por tanto, es el método más confiable para codificar información digital. Si no está familiarizado con los números binarios, vea el apéndice A.

En la publicidad de algunas computadoras, como las mainframes IBM grandes, se alardea de que tienen aritmética decimal además de binaria. Este truco se logra utilizando 4 bits para almacenar un dígito decimal empleando un código llamado **BCD (decimal codificado en binario, Binary Coded Decimal)**. Cuatro bits se pueden combinar de 16 formas, diez de las cuales se usan para los dígitos del 0 al 9; seis combinaciones no se usan. A continuación se muestra el número 1944 codificado en decimal y en binario puro, empleando 16 bits en cada ejemplo:

decimal: 0001 1001 0100 0100 binario: 0000011110011000

Dieciséis bits en el formato decimal pueden almacenar los números del 0 al 9999, lo que da sólo 10,000 combinaciones, mientras que un número binario puro de 16 bits puede almacenar 65,536 combinaciones distintas. Es por esto que decimos que el almacenamiento binario es más eficiente.

Sin embargo, considere lo que sucedería si algún joven y brillante ingeniero eléctrico inventara un dispositivo electrónico altamente confiable que pudiera almacenar directamente los dígitos del 0 al 9 dividiendo la región de 0 a 10 volts en 10 intervalos. Cuatro de estos dispositivos podrían almacenar cualquier número decimal del 0 al 9999, y podrían formar 10,000 combinaciones. También podrían usarse cuatro de esos dispositivos para almacenar números binarios, pero sólo empleando 0 y 1. En tal caso, sólo podrían almacenar 16 combinaciones. Con tales dispositivos, el sistema decimal obviamente es más eficiente.

2.2.2 Direcciones de memoria

Las memorias consisten en varias **celdas** (o **localidades**), cada una de las cuales puede almacenar un elemento de información. Cada celda tiene un número, su **dirección**, con el cual los programas pueden referirse a ella. Si una memoria tiene n celdas, tendrán las direcciones 0 a $n - 1$. Todas las celdas de una memoria contienen el mismo número de bits. Si una celda consta de k bits, podrá contener cualquiera de 2^k combinaciones de bits distintas. En la figura 2-9 se muestran tres diferentes organizaciones para una memoria de 96 bits. Observe que celdas adyacentes tienen direcciones consecutivas (por definición).

Las computadoras que emplean el sistema de numeración binario (incluidas las notaciones octal y hexadecimal para números binarios) expresan las direcciones de memoria como números binarios. Si una dirección tiene m bits, el número máximo de celdas direccionables es 2^m . Por ejemplo, una dirección empleada para referirse a la memoria de la figura 2-9(a) necesita al menos 4 bits para expresar todos los números del 0 al 11. En cambio, basta una dirección de 3 bits para las figuras 2-9(b) y (c). El número de bits de la dirección determina el número máximo de celdas direccionables directamente en la memoria y es independiente

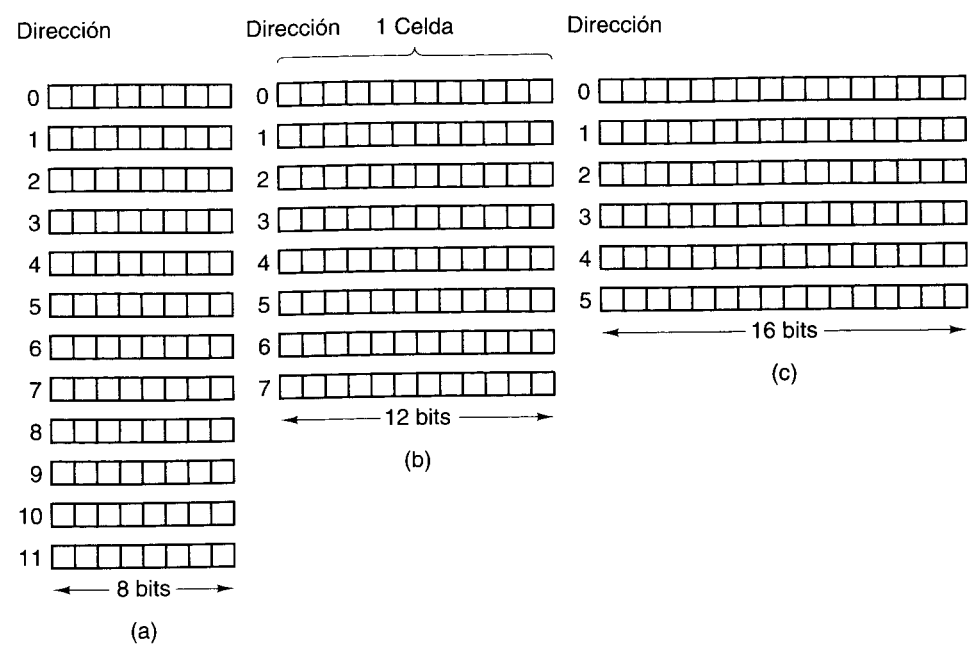


Figura 2-9. Tres formas de organizar una memoria de 96 bits.

del número de bits por celda. Una memoria que tiene 2^{12} celdas de 8 bits cada una, y una que tiene 2^{12} celdas de 64 bits cada una, necesitan ambas direcciones de 12 bits.

En la figura 2-10 se da el número de bits por celda de algunas computadoras que se han vendido comercialmente.

La importancia de la celda es que es la unidad direccionable más pequeña. En años recientes casi todos los fabricantes de computadoras han adoptado como estándar una celda de 8 bits, que recibe el nombre de **byte**. Los bytes se agrupan en **palabras**. Una computadora con palabras de 32 bits tiene 4 bytes/palabra, mientras que una con palabras de 64 bits tiene 8 bytes/palabra. La importancia de las palabras es que casi todas las instrucciones operan con palabras enteras; por ejemplo, suman dos palabras. Así, una máquina de 32 bits tiene registros de 32 bits e instrucciones para manipular palabras de 32 bits, mientras que una máquina de 64 bits tiene registros de 64 bits e instrucciones para transferir, sumar, restar y manipular de otras maneras palabras de 64 bits.

2.2.3 Ordenamiento de bytes

Los bytes de una palabra pueden numerarse de izquierda a derecha o de derecha a izquierda. A primera vista podría parecer que esta decisión carece de importancia pero, como veremos en breve, tiene implicaciones importantes. En la figura 2-11(a) se muestra parte de la memoria de una computadora de 32 bits cuyos bytes se numeran de izquierda a derecha, como la SPARC o las grandes mainframes de IBM. En la figura 2-11(b) se aprecia la representación análoga de una computadora de 32 bits que usa numeración de derecha a izquierda, como la

Computadora	Bits/celda
Burroughs B1700	1
IBM PC	8
DEC PDP-8	12
IBM 1130	16
DEC PDP-15	18
XDS 940	24
Electrologica X8	27
XDS Sigma 9	32
Honeywell 6180	36
CDC 3600	48
CDC Cyber	60

Figura 2-10. Número de bits por celda en algunas computadoras comerciales de importancia histórica.

familia Intel. El primer sistema, en el que la numeración comienza por el extremo “grande” (es decir, de orden alto), se llama computadora **big endian**, en contraste con la **little endian** de la figura 2-11(b). Estos términos provienen de la obra *Los viajes de Gulliver*, donde Jonathan Swift hacía una sátira de los políticos que declararon una guerra a causa de su disputa respecto a si los huevos debían romperse por el extremo ancho (*big end*, en la obra original en inglés) o por el extremo angosto (*little end*). Los términos se usaron por primera vez aplicados a la arquitectura de computadoras en un delicioso artículo escrito por Cohen (1981).

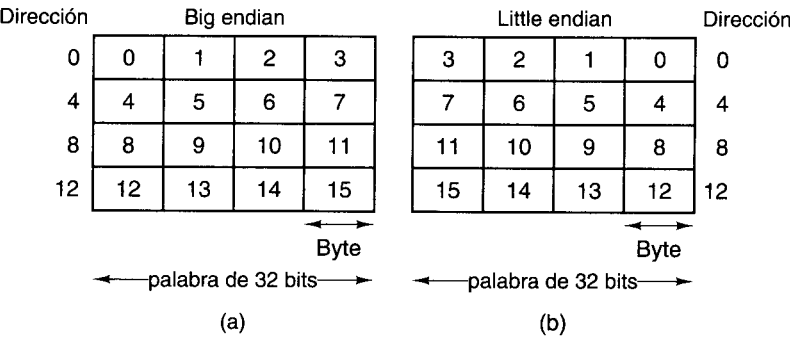


Figura 2-11. (a) Memoria big endian. (b) Memoria little endian.

Es importante entender que tanto en los sistemas big endian como en los little endian, un entero de 32 bits con el valor numérico de, digamos, 6, se representa con los bits 110 en los tres bits de la extrema derecha (de orden bajo) de una palabra, con ceros en los 29 bits de la izquierda. En el esquema big endian, los bits 110 están en el byte 3 (o 7, u 11, etc.), mientras

que en el esquema little endian están en el byte 0 (o 4, u 8, etc.). En ambos casos, la palabra que contiene este entero tiene la dirección 0.

Si las computadoras sólo almacenaran enteros, no habría problemas, pero muchas aplicaciones requieren una mezcla de enteros, cadenas de caracteres y otros tipos de datos. Considere, por ejemplo, un sencillo registro de personal que consiste en una cadena (nombre de empleado) y dos enteros (edad y número de departamento). La cadena termina con uno o más bytes 0 para rellenar una palabra. La representación big endian para Jim Smith, de 21 años, del departamento 260 ($1 \times 256 + 4 = 260$) se muestra en la figura 2.12(a); la little endian, en la figura 2-12(b).

Big endian				Little endian				Transferencia de big endian a little endian				Transferencia e intercambio			
0	J	I	M			M	I	J				J	I	M	
4	S	M	I	T		T	I	M	S			S	M	I	T
8	H	0	0	0		0	0	0	H			H	0	0	0
12	0	0	0	21		0	0	0	21			0	0	0	21
16	0	0	1	4		0	0	1	4			0	0	1	4
(a)				(b)				(c)				(d)			

Figura 2-12. (a) Registro de personal en una máquina big endian. (b) El mismo registro en una máquina little endian. (c) Resultado de transferir el registro de una big endian a una little endian. (d) Resultado de intercambiar los bytes de (c).

Ambas representaciones son excelentes y son congruentes internamente. Los problemas comienzan cuando una de las máquinas trata de enviar el registro a la otra por una red. Supongamos que la big endian envía el registro a la little endian byte por byte, comenzando con el byte 0 y terminando con el byte 19. (Seremos optimistas y supondremos que los bits de los bytes no se invierten durante la transmisión, pues ya tenemos suficientes problemas.) Así, el byte 0 de la big endian se coloca en el byte 0 de la memoria de la little endian, y así sucesivamente, como se muestra en la figura 2-12(c).

Cuando la little endian trata de imprimir el nombre, no hay problema, pero la edad aparece como 21×2^{24} , y el departamento es igualmente absurdo. Esta situación surge porque la transmisión invirtió el orden de los caracteres de una palabra, como debería, pero también invirtió los bytes de los enteros, cosa que no debió hacer.

Una solución obvia es hacer que el software invierta los bytes de una palabra después de efectuar el copiado. Esto conduce a la figura 2-12(d) y hace que los dos enteros queden bien pero convierte la cadena en "MIJTIMS" "H". Esta inversión de la cadena ocurre porque al leerla, la computadora primero lee el byte 0 (un espacio), luego el byte 1 (M), etcétera.

No existe una solución sencilla. Un remedio que funciona, pero es ineficiente, es incluir una cabecera frente a cada dato que indique el tipo de datos que sigue (cadena, entero u otro) y qué longitud tiene. Esto permite al receptor efectuar sólo las conversiones que sean necesarias. En todo caso, es evidente que la falta de un estándar para el ordenamiento de los bytes genera problemas innecesarios al intercambiar datos entre diferentes máquinas.

2.2.4 Códigos para corrección de errores

Las memorias de las computadoras pueden cometer errores ocasionales a causa de picos de voltaje en la línea de alimentación u otras causas. Para protegerse contra tales errores, algunas memorias emplean códigos para detección o corrección de errores. Cuando se usan esos códigos, se añaden bits extra de una forma especial a cada palabra de la memoria. Cuando se lee una palabra de la memoria, se verifican los bits adicionales para ver si ha ocurrido algún error.

Para entender cómo pueden manejarse los errores, es necesario examinar de cerca la naturaleza de los mismos. Supongamos que una palabra de memoria consiste en m bits de datos a los cuales añadiremos r bits redundantes, o de verificación. Sea la longitud total n (es decir, $n = m + r$). Una unidad de n bits que contiene m bits de datos y r bits de verificación se conoce como **palabra de código** de n bits.

Dadas dos palabras de código cualesquiera, digamos 10001001 y 10110001, es posible determinar cuántos bits correspondientes difieren. En este caso, tres bits difieren. Para determinar cuántos bits difieren basta calcular el OR EXCLUSIVO booleano bit por bit de las dos palabras de código, y contar el número de bits 1 en el resultado. El número de posiciones de bit en las que dos palabras de código difieren se denomina **distancia de Hamming** (Hamming, 1950). La importancia de esta cifra es que si dos palabras de código están separadas por una distancia de Hamming d , se requieren d errores de un solo bit para convertir una palabra en la otra. Por ejemplo, las palabras de código 11110001 y 00110000 están separadas por una distancia de Hamming de 3 porque se requieren tres errores de un solo bit para convertir una en la otra.

En el caso de una palabra de memoria de m bits, están permitidos los 2^m patrones de bits, pero a causa de la forma en que se calculan los bits de verificación, sólo 2^m de las 2^n palabras de código son válidas. Si al leer la memoria se obtiene una palabra de código no válida, la computadora sabrá que ocurrió un error de memoria. Si se conoce el algoritmo para calcular los bits de verificación, es posible construir una lista completa de las palabras de código permitidas, y encontrar en esta lista las dos palabras de código cuya distancia de Hamming es mínima. Esta distancia es la distancia de Hamming de todo el código.

Las propiedades de detección y corrección de errores de un código dependen de su distancia de Hamming. Para detectar d errores de un solo bit se necesita un código con una distancia de $d + 1$, porque con un código así no es posible que d errores de un solo bit conviertan una palabra de código válida en otra palabra de código válida. De forma similar, para corregir d errores de un solo bit se requiere un código con una distancia $2d + 1$ porque así las palabras de código válidas están tan lejanas unas de otras que aun con d cambios la palabra de código original está más cerca de la palabra con errores que cualquier otra palabra de código válida, y es posible determinarla de forma inequívoca.

Como ejemplo sencillo de código con detección de errores, considere un código en el que se anexa un solo **bit de paridad** a los datos. El bit de paridad se escoge de modo que el número de bits 1 en la palabra de código sea siempre par (o impar). Un código así tiene una distancia de 2, ya que cualquier error de un solo bit produce una palabra de código con la paridad equivocada. En otras palabras, se requieren dos errores de un solo bit para pasar de una a otra palabras de código válidas, y el código puede servir para detectar errores sencillos.

Siempre que se lee de la memoria una palabra con la paridad incorrecta, se indica una condición de error. El programa no puede continuar, pero al menos no se calculan resultados incorrectos.

Como ejemplo sencillo de código con corrección de errores, considere un código que sólo tiene cuatro palabras de código válidas:

0000000000, 0000011111, 1111100000 y 1111111111

Este código tiene una distancia de 5, lo que implica que puede corregir errores dobles. Si llega la palabra de código 0000000111, el receptor sabe que la palabra original tiene que haber sido 0000011111 (si no hubo más de dos errores). Por otra parte, si un triple error convierte 0000000000 en 0000000111, el error no podrá corregirse.

Imagine que quiere diseñar un código con m bits de datos y r bits de verificación que permita corregir todos los errores de un solo bit. Cada una de las 2^m palabras de memoria válidas tiene n palabras de código no válidas a una distancia de 1. Éstas se forman invirtiendo por turno cada uno de los n bits de la palabra de código de n bits que se forma a partir de la palabra de memoria válida. Así, cada una de las 2^m palabras de memoria válidas requiere $n + 1$ patrones de bits dedicados a ella (para los n posibles errores y el patrón correcto). Puesto que el número total de patrones de bits es 2^n , necesitaremos que $(n + 1)2^m \leq 2^n$. Si usamos $n = m + r$, este requisito se convierte en $(m + r + 1) \leq 2^r$. Dado m , esto fija un límite inferior para el número de bits de verificación que se requieren para corregir errores sencillos. En la figura 2-13 se muestra el número de bits de verificación necesarios para diversos tamaños de palabra de memoria.

Tamaño de palabra	Bits de	Tamaño total	Porcentaje de gasto e xtra verificación
8	4	12	50
16	5	21	31
32	6	38	19
64	7	71	11
128	8	136	6
256	9	265	4
512	10	522	2

Figura 2-13. Número de bits de verificación para un código que puede corregir un solo error.

Este límite inferior teórico puede lograrse empleando un método ideado por Richard Hamming (1950). Antes de examinar el algoritmo de Hamming, veamos una sencilla representación gráfica que ilustra claramente la idea de un código de corrección de errores para palabras de 4 bits. El diagrama de Venn de la figura 2-14(a) contiene tres círculos, A , B y C , que juntos forman siete regiones. Como ejemplo, codifiquemos la palabra de memoria de 4 bits 1100 en las regiones AB , ABC , AC y BC , un bit por región (en orden alfabético). Esta codificación se muestra en la figura 2-14(a).

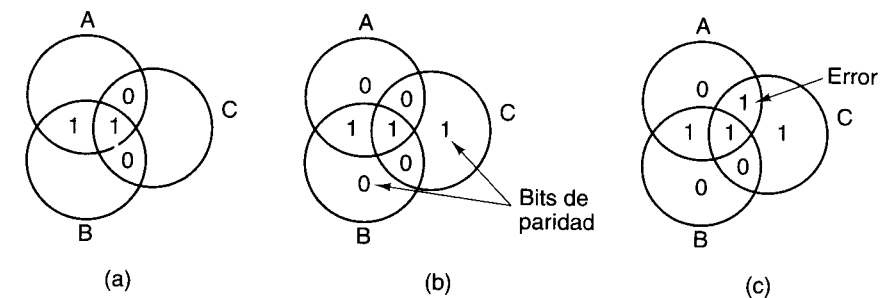


Figura 2-14. (a) Codificación de 1100. (b) Adición de paridad par. (c) Error en AC .

Ahora añadimos un bit de paridad a cada una de las tres regiones vacías a modo de producir una paridad par, como se ilustra en la figura 2-14(b). Por definición, la suma de los bits de cada uno de los tres círculos, A , B y C , es ahora un número par. En el círculo A tenemos los cuatro números 0, 0, 1 y 1, que suman 2, un número par. En el círculo B , los números son 1, 1, 0 y 0, que también suman 2, un número par. Por último, en el círculo C tenemos la misma situación. En este ejemplo da la casualidad que todos los círculos son iguales, pero en otros ejemplos podría haber también sumas de 0 y 4. Esta figura corresponde a una palabra de código con 4 bits de datos y 3 bits de paridad.

Supongamos ahora que el bit de la región AC es erróneo y cambia de 0 a 1, como se muestra en la figura 2-14(c). La computadora detecta que los círculos A y C tienen la paridad equivocada (impar). El único cambio de un solo bit que corrige la situación es restaurar AC a 0, lo que corrige el error. De este modo, la computadora puede reparar errores de memoria de un solo bit automáticamente.

Veamos ahora cómo podemos usar el algoritmo de Hamming para construir códigos con corrección de errores para palabras de memoria de cualquier tamaño. En un código de Hamming, se añaden r bits de paridad a una palabra de m bits para formar una nueva palabra de $m + r$ bits. Los bits se numeran a partir de 1, no de 0, siendo el bit 1 el de la extrema izquierda (bit de orden alto). Todos los bits cuyo número es una potencia de 2 son bits de paridad; los demás se utilizan para datos. Por ejemplo, con palabras de 16 bits se añaden cinco bits de paridad. Los bits 1, 2, 4, 8 y 16 son bits de paridad, y todos los demás son bits de datos. En total, la palabra de memoria tiene 21 bits (16 de datos, 5 de paridad). Usaremos (arbitrariamente) paridad par en este ejemplo.

Cada bit de paridad examina posiciones de bit específicas; el bit de paridad se ajusta de modo que el número total de unos en las posiciones verificadas sea par. Las posiciones de bit que verifica cada bit de paridad son

Bit 1 verifica los bits 1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21.

Bit 2 verifica los bits 2, 3, 6, 7, 10, 11, 14, 15, 18, 19.

Bit 4 verifica los bits 4, 5, 6, 7, 12, 13, 14, 15, 20, 21.

Bit 8 verifica los bits 8, 9, 10, 11, 12, 13, 14, 15.

Bit 16 verifica los bits 16, 17, 18, 19, 20, 21.

En general, el bit b es verificado por los bits $b_1, b_2, \dots, b_j$ tales que $b_1 + b_2 + \dots + b_j = b$. Por ejemplo, el bit 5 es verificado por los bits 1 y 4 porque $1 + 4 = 5$. El bit 6 es verificado por los bits 2 y 4 porque $2 + 4 = 6$, etcétera.

La figura 2-15 muestra la construcción de un código de Hamming para la palabra de memoria de 16 bits 1111000010101110. La palabra de código de 21 bits es 00101110000010101110. Para ver cómo funciona la corrección de errores, consideremos qué sucedería si el bit 5 se invirtiera por un pico en la alimentación eléctrica. La nueva palabra de código sería 00100110000010101110 en lugar de 00101110000010101110. Verifiquemos los cinco bits de paridad, con los siguientes resultados:

Paridad del bit 1 incorrecta (1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21 contienen cinco unos).

Paridad del bit 2 correcta (2, 3, 6, 7, 10, 11, 14, 15, 18, 19 contienen seis unos).

Paridad del bit 4 incorrecta (4, 5, 6, 7, 12, 13, 14, 15, 20, 21 contienen cinco unos).

Paridad del bit 8 correcta (8, 9, 10, 11, 12, 13, 14, 15 contienen dos unos).

Paridad del bit 16 correcta (16, 17, 18, 19, 20, 21 contienen cuatro unos).

El número total de unos de los bits 1, 3, 5, 7, 9, 11, 13, 15, 17, 19 y 21 debería ser par, porque estamos usando paridad par. El bit incorrecto debe ser uno de los bits verificados por el bit de paridad 1: el bit 1, 3, 5, 7, 9, 11, 13, 15, 17, 19 o 21. El bit de paridad 4 es incorrecto, lo que implica que uno de los bits 4, 5, 6, 7, 12, 13, 14, 15, 20 o 21 es incorrecto. El error debe estar en uno de los bits de ambas listas, o sea 5, 7, 13, 15 o 21. Sin embargo, el bit 2 es correcto, lo que elimina al 7 y al 15. Así mismo, el bit 8 es correcto, lo que elimina al 13. Por último, el bit 16 es correcto, lo que elimina al 21. El único bit que queda es el 5, que es el que tuvo el error. Puesto que se leyó como 1, debería ser 0. Es así como se corrigen los errores.

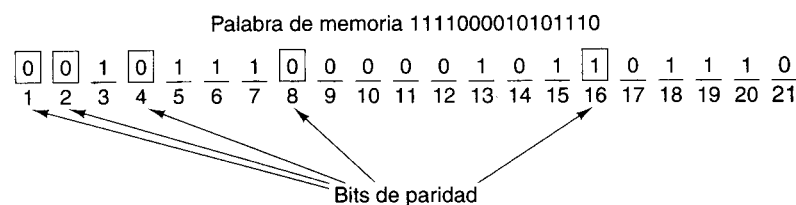


Figura 2-15. Construcción del código Hamming para la palabra de memoria 1111000010101110 añadiendo cinco bits de verificación a los 16 bits de datos.

Un método sencillo para encontrar el bit incorrecto consiste en calcular primero todos los bits de paridad. Si todos son correctos, no hubo error (o hubo más de uno). Luego se suman todos los bits de paridad incorrectos, contando 1 por el bit 1, 2 por el bit 2, 4 por el bit 4, etc. La suma que se obtiene es la posición del bit incorrecto. Por ejemplo, si los bits de paridad 1 y 4 son incorrectos pero los 2, 8 y 16 son correctos, el bit 5 ($1 + 4$) se habrá invertido.

2.2.5 Memoria caché

Históricamente, las CPU siempre han sido más rápidas que las memorias. Al mejorar las memorias, también han mejorado las CPU, y la diferencia ha persistido. De hecho, al aumentar el número de circuitos que es posible incluir en un chip, los diseñadores de CPU han aprovechado esos recursos para crear filas de procesamiento y operaciones superescalares, lo que hace a las CPU aún más rápidas. Los diseñadores de memorias generalmente han utilizado la tecnología nueva para aumentar la capacidad de sus chips, no la velocidad, así que al parecer el problema está empeorando con el paso del tiempo. Lo que esta diferencia implica en la práctica es que, después de que la CPU emite una solicitud a la memoria, pasan muchos ciclos antes de que reciba la palabra que necesita. Cuanto más lenta es la memoria, más ciclos tiene que esperar la CPU.

Como señalamos antes, hay dos formas de resolver este problema. La más sencilla es iniciar las instrucciones de lectura de memoria (READ) cuando se llega a ellas, pero continuar la ejecución y detener la CPU si una instrucción trata de usar la palabra de memoria antes de que llegue. Cuanto más lenta sea la memoria, con mayor frecuencia ocurrirá este problema y mayor será el retraso cuando ocurra. Por ejemplo, si el retraso de memoria es de 10 ciclos, es muy probable que una de las siguientes 10 instrucciones trate de usar la palabra que se leyó.

La otra solución es tener máquinas que no se detengan, y que exijan a los compiladores no generar código que use palabras antes de que hayan llegado. El problema es que no es nada fácil llevar esto a la práctica. Es común que después de un LOAD no haya nada más que hacer, y el compilador se vea obligado a insertar instrucciones NOP (no hay operación) que no hacen nada más que ocupar una ranura y desperdiciar tiempo. En realidad, esto equivale a detenerse, sólo que en software en lugar de en hardware, pero la degradación del desempeño es la misma.

De hecho, el problema no es la tecnología, sino la economía. Los ingenieros saben cómo construir memorias que sean tan rápidas como las CPU, pero para operar a su máxima velocidad tienen que estar dentro del chip de la CPU (porque ir por el bus hasta la memoria es muy lento). La colocación de una memoria grande en el chip de la CPU lo hace más grande y por tanto más costoso, pero aun si el costo no fuera importante existen límites para el tamaño que un chip de CPU puede tener. Así, la decisión se reduce a tener una cantidad pequeña de memoria rápida o una cantidad grande de memoria lenta. Lo que preferiríamos es una gran cantidad de memoria rápida a un precio bajo.

Lo interesante es que se conocen técnicas para combinar una cantidad pequeña de memoria rápida con una cantidad grande de memoria lenta para obtener la velocidad de la memoria rápida (casi) y la capacidad de la memoria grande a un precio moderado. La memoria pequeña y rápida se llama **caché** (del francés *cache*, que significa guardar o esconder). A continuación describiremos brevemente cómo se usan los cachés y como funcionan. En el capítulo 4 se dará una explicación más detallada.

La idea fundamental en que se basa la memoria caché es sencilla: las palabras de memoria de mayor uso se mantienen en el caché. Cuando la CPU necesita una palabra, primero la busca en el caché. Sólo si la palabra no está ahí recurre a la memoria principal. Si una fracción sustancial de las palabras está en el caché, el tiempo de acceso promedio puede reducirse considerablemente.

Así pues, el éxito o el fracaso dependen de qué fracción de las palabras está en el caché. Desde hace años, se sabe que los programas no acceden a la memoria de forma totalmente aleatoria. Si una referencia a la memoria dada es a la dirección A , es probable que la siguiente referencia sea a una dirección cercana a A . Un ejemplo sencillo es el programa mismo. Con excepción de las ramificaciones y las llamadas a procedimientos, las instrucciones se traen de posiciones consecutivas de la memoria. Además, casi todo el tiempo de ejecución de los programas se invierte en realizar ciclos (*loops*), en los que un número limitado de instrucciones se ejecutan una y otra vez. Así mismo, un programa que manipula matrices probablemente hará muchas referencias a la misma matriz antes de pasar a otra cosa.

La observación de que las referencias a la memoria que se hacen en cualquier periodo corto tienden a usar una fracción pequeña de la memoria total se denomina **principio de localidad** y constituye la base de todos los sistemas de caché. La idea general es que cuando se hace referencia a una palabra, ella y algunas de sus vecinas se traen de la memoria grande lenta al caché, para que la siguiente vez que se use el acceso a ella sea rápido. Un acomodo común de la CPU, el caché y la memoria principal se ilustra en la figura 2-16. Si una palabra se lee o se escribe k veces en un intervalo corto, la computadora necesitará una referencia a la memoria lenta y $k - 1$ referencias a la memoria rápida. Cuanto mayor sea k , mejor será el desempeño global.

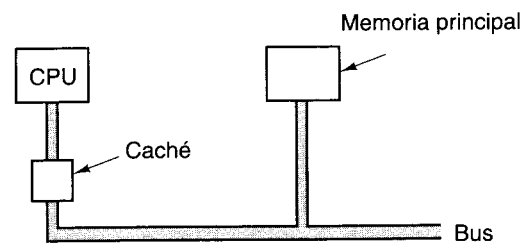


Figura 2-16. El caché se encuentra lógicamente entre la CPU y la memoria principal. Físicamente, hay varios lugares donde podría colocarse.

Podemos formalizar este cálculo introduciendo c , el tiempo de acceso al caché, m , el tiempo de acceso a la memoria principal y h , la **tasa de aciertos**, que es la fracción de todas las referencias que pueden satisfacerse con el caché. En nuestro pequeño ejemplo del párrafo anterior, $h = (k - 1)/k$. Algunos autores también definen la **tasa de fallos**, que es $1 - h$.

Con estas definiciones, podemos calcular el tiempo de acceso medio así:

$$\text{tiempo de acceso medio} = c + (1 - h)m$$

A medida que $h \rightarrow 1$, todas las referencias pueden satisfacerse con el caché, y el tiempo de acceso se aproxima a c . Por otro lado, si $h \rightarrow 0$, se requerirá una referencia a la memoria en cada ocasión, y el tiempo de acceso se aproximará a $c + m$: primero un tiempo c para examinar el caché (sin éxito) y luego un tiempo m para efectuar la referencia a la memoria. En algunos sistemas, la referencia a la memoria puede iniciarse en paralelo con la búsqueda en el caché, de modo que si no hay acierto en el caché, el ciclo de memoria ya se habrá iniciado.

Sin embargo, esta estrategia requiere que la búsqueda en la memoria pueda suspenderse si hay un acierto en el caché, y esto complica la implementación.

Con el principio de localidad como guía, las memorias principales y los cachés se dividen en bloques de tamaño fijo. Al hablar de estos bloques dentro del caché, es común usar el término **líneas de caché**. Cuando hay un fallo de caché, toda la línea de caché se carga de la memoria principal al caché, no sólo la palabra que se necesita. Por ejemplo, si las líneas son de 64 bytes, una referencia a la dirección de memoria 260 traerá la línea que consiste en los bytes 256 a 319 a una línea de caché. Con un poco de suerte, pronto se necesitarán también algunas de las otras palabras de esa línea de caché. Operar así es más eficiente que buscar palabras individuales porque es más rápido traer k palabras a la vez que traer una palabra k veces. Además, si las entradas del caché son más grandes que una palabra habrá menos de ellas, y el gasto extra para manejarlas será menor.

El diseño de cachés es un tema cada vez más importante para el diseño de las CPU de alto rendimiento. Un aspecto es el tamaño del caché. Cuanto más grande es el caché, mejor funciona, pero también cuesta más. Un segundo aspecto es el tamaño de la línea de caché. Un caché de 16 KB puede dividirse en 1 K líneas de 16 bytes, 2 K líneas de 8 bytes, y otras combinaciones. Un tercer aspecto es la organización del caché, es decir, ¿cómo hace el caché para saber qué palabras de memoria contiene en un momento dado? Examinaremos los cachés con detalle en el capítulo 4.

Un cuarto aspecto de diseño es si las instrucciones y los datos se guardan en el mismo caché o en cachés distintos. Un diseño de **caché unificado** (instrucciones y datos en el mismo caché) es más sencillo y automáticamente equilibra las obtenciones de instrucciones frente a las obtenciones de datos. No obstante, la tendencia actual es hacia los **cachés divididos**, con instrucciones en un caché y datos en otro. Este diseño también se denomina **arquitectura Harvard**, referencia que se remonta a la computadora Mark III de Howard Aiken, que tenía diferentes memorias para las instrucciones y los datos. Lo que está impulsando a los diseñadores en esta dirección es el uso tan extendido de las CPU con filas de procesamiento. La unidad de búsqueda de instrucciones necesita acceder a instrucciones al mismo tiempo que la unidad de búsqueda de operandos requiere acceder a datos. Un caché dividido permite accesos en paralelo; un caché unificado no. Además, puesto que las instrucciones normalmente no se modifican durante la ejecución, el contenido del caché de instrucciones nunca tiene que volver a escribirse en la memoria.

Por último, un quinto aspecto es el número de cachés. Hoy día no es raro tener chips con un caché primario en el chip de CPU, un caché secundario fuera del chip pero en el mismo paquete, y un tercer caché todavía más lejos.

2.2.6 Empaquetamiento y tipos de memoria

Desde los albores de las memorias de semiconductores hasta principios de los noventa, la memoria se fabricaba, se compraba y se instalaba como chips individuales. Las densidades de los chips iban desde 1K bits hasta 1M bits o más, pero cada chip era una unidad discreta. Las primeras PC a menudo tenían zócalos vacíos en los que podían insertarse chips de memoria adicionales, si el comprador llegaba a necesitarlos.

Actualmente se acostumbra usar una disposición diferente. Un grupo de chips, por lo regular 8 o 16, se monta en una diminuta tarjeta de circuitos integrados y se vende como unidad. Esta unidad se llama **SIMM** (**módulo de memoria individual en línea**, *Single Inline Memory Module*) o **DIMM** (**módulo de memoria dual en línea**, *Dual Inline Memory Module*), dependiendo de si tiene una fila de conectores en un lado o en ambos lados de la tarjeta. En la figura 2-17 se ilustra un ejemplo de SIMM.

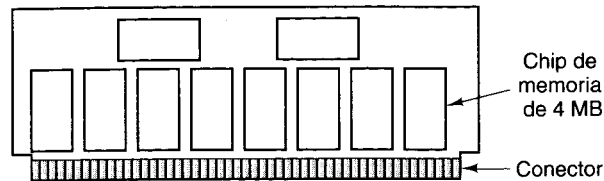


Figura 2-17. Módulo individual de memoria en línea (SIMM) que contiene 32 MB. Dos de los chips controlan el SIMM.

Una configuración de SIMM típica podría tener ocho chips con 32 megabits (4 MB) cada uno en el SIMM. Así, todo el módulo contendría 32 MB. Muchas computadoras tienen espacio para cuatro módulos, lo que da una capacidad total de 128 MB si son empleados SIMM de 32 MB. En muchos casos estos SIMM pueden sustituirse posteriormente por SIMM de 64 MB o más grandes si es necesario.

Los primeros SIMM tenían 30 conectores y alimentaban 8 bits a la vez. Los demás conectores eran para direccionamiento y control. SIMM posteriores tenían 72 conectores y alimentaban 32 bits a la vez. Para una máquina como la Pentium, que esperaba 64 bits a la vez, los SIMM de 72 conectores se apareaban, y cada uno alimentaba la mitad de los bits requeridos. Actualmente los DIMM son la forma estándar de empaquetar la memoria. Cada DIMM tiene 84 conectores chapeados con oro en cada lado, para un total de 168 conectores. Un DIMM puede alimentar 64 bits a la vez. Las capacidades típicas de los DIMM son de 64 MB o más. Un DIMM físicamente más pequeño, llamado **SO-DIMM** (**DIMM de contorno pequeño**, *Small Outline DIMM*), se usa en computadoras portátiles. Es posible añadir a los SIMM y DIMM un bit de paridad o corrección de errores, pero puesto que la tasa media de errores de un módulo es de un error cada 10 años, en casi todas las computadoras ordinarias se ha omitido la detección y corrección de errores.

2.3 MEMORIA SECUNDARIA

Por más grande que sea la memoria principal, siempre es demasiado pequeña. La gente siempre quiere almacenar más información de la que cabe en la memoria, primordialmente porque, a medida que la tecnología mejora, comienzan a pensar en almacenar cosas que antes sólo se les ocurrían a los escritores de ciencia-ficción. Por ejemplo, a medida que la disciplina presupuestaria del gobierno de Estados Unidos obliga a sus dependencias a generar sus propias ganancias, podríamos imaginar que la Biblioteca del Congreso decidiera digitalizar y vender todo su acervo como artículo de consumo ("Todo el Conocimiento Humano por sólo

49 dólares"). A muy grandes rasgos, serían unos 50 millones de libros, cada uno con 1 MB de texto y 1 MB de ilustraciones comprimidas, y necesitaríamos almacenar 10^{14} bytes = 100 terabytes. Almacenar las casi 50,000 películas que se han filmado es una tarea de magnitud comparable. Esta cantidad de información no va a caber en la memoria principal, al menos no en las próximas décadas.

2.3.1 Jerarquías de memoria

La solución tradicional para almacenar una gran cantidad de datos es una jerarquía de memoria, como se ilustra en la figura 2-18. En la cúspide están los registros de la CPU, a los que puede tenerse acceso a la velocidad máxima de la CPU. Luego viene la memoria caché, que actualmente es del orden de 32 KB a unos cuantos megabytes. Sigue la memoria principal, con tamaños que actualmente van de 16 MB para los sistemas más económicos hasta decenas de gigabytes en el extremo superior. Después vienen los discos magnéticos, el "caballito de batalla" actual para el almacenamiento permanente. Por último, tenemos la cinta magnética y los discos ópticos para el almacenamiento de archivos.

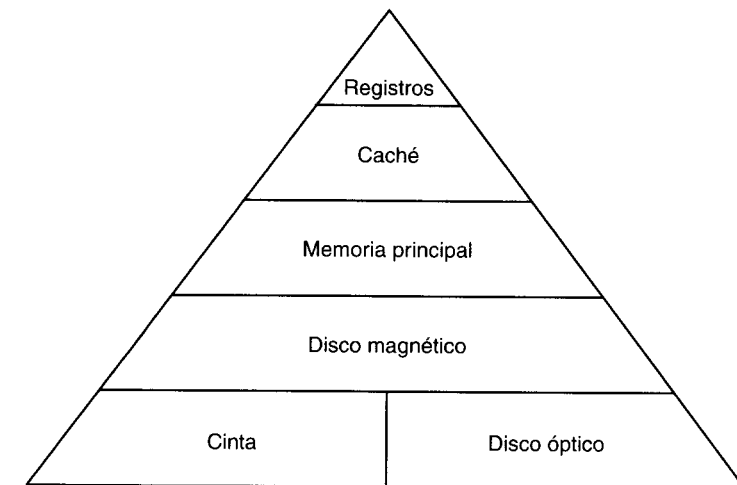


Figura 2-18. Jerarquía de memoria de cinco niveles.

Al bajar por la jerarquía, tres parámetros clave crecen. Primero, el tiempo de acceso se alarga. Los registros de la CPU pueden accederse en unos cuantos nanosegundos. Las memorias caché requieren un múltiplo pequeño del tiempo de acceso de los registros. Los accesos a la memoria principal suelen ser de unas cuantas decenas de nanosegundos. Luego viene una brecha grande, pues los tiempos de acceso a disco son de por lo menos 10 ms, y el acceso a cinta o disco óptico puede medirse en segundos si es preciso traer los medios e insertarlos en una unidad.

Segundo, la capacidad de almacenamiento aumenta al bajar por la jerarquía. Los registros de la CPU pueden contener tal vez 128 bytes; los cachés unos cuantos megabytes; las memorias principales decenas o miles de megabytes; los discos magnéticos de unos cuantos

gigabytes a decenas de gigabytes. Las cintas y los discos ópticos generalmente se guardan fuera de línea, así que su capacidad está limitada sólo por el presupuesto del propietario.

Tercero, el número de bits que se obtiene por dólar invertido aumenta al bajar por la jerarquía. Aunque los precios actuales cambian rápidamente, la memoria principal se mide en dólares/megabyte; el almacenamiento en disco magnético en centavos de dólar/megabyte, y la cinta magnética en dólares/gigabyte o menos.

Ya examinamos los registros, el caché y la memoria principal. En las secciones que siguen estudiaremos los discos magnéticos; después nos ocuparemos de los discos ópticos. No estudiaremos las cintas porque se usan casi exclusivamente para copias de seguridad, y de todos modos no hay mucho que hablar al respecto.

2.3.2 Discos magnéticos

Un disco magnético consiste en uno o más platos de aluminio con un recubrimiento magnetizable. Originalmente estos platos llegaron a tener 50 cm de diámetro, pero en la actualidad suelen ser de 3 a 12 cm, y algunos discos para computadoras portátiles tienen diámetros de menos de 3 cm y se siguen encogiendo. Una cabeza de disco que contiene una bobina de inducción flota sobre la superficie y muy cerca de ella, descansando en un colchón de aire (excepto en los discos flexibles, en los que toca la superficie). Cuando una corriente positiva o negativa pasa por la cabeza, magnetiza la superficie justo debajo de ella, alineando las partículas magnéticas hacia la izquierda o hacia la derecha, dependiendo de la polaridad de la corriente de la unidad. Cuando la cabeza pasa sobre un área magnetizada, se induce una corriente positiva o negativa en la cabeza, lo que permite leer los bits almacenados previamente. Así, el plato gira bajo la cabeza, puede escribirse una serie de bits y luego leerse. La geometría de una pista de disco se muestra en la figura 2-19.

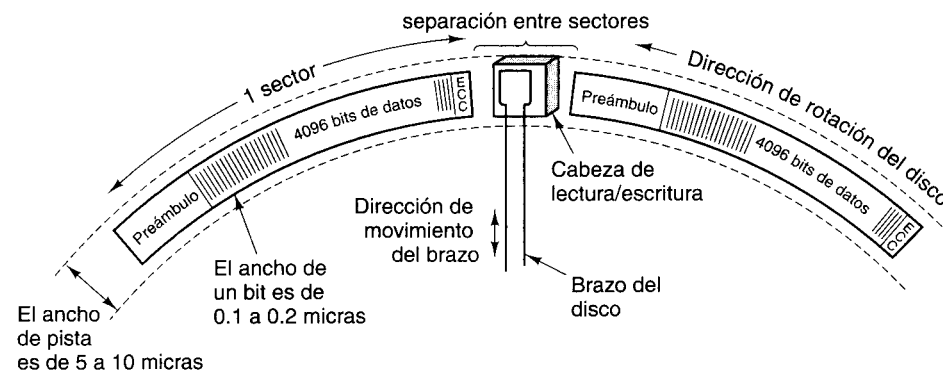


Figura 2-19. Porción de una pista de disco. Se ilustran dos sectores.

La secuencia circular de bits que se escriben cuando el disco efectúa una rotación completa se llama **pista**. Cada pista se divide en **sectores** de longitud fija, que típicamente contienen 512 bytes de datos, precedidos por un **preámbulo** que permite a la cabeza sincronizarse antes de leer o escribir. Después de los datos viene un código de corrección de errores (ECC,

Error-Correcting Code), que puede ser un código de Hamming o, más comúnmente, un código capaz de corregir múltiples errores llamado **código Reed-Solomon**. Entre sectores consecutivos hay una pequeña **separación entre sectores**. Algunos fabricantes citan la capacidad de sus discos en un estado sin formato (como si cada pista contuviera únicamente datos), pero una medición más honesta es la capacidad formateada, que no cuenta los preámbulos, ECC ni separaciones como datos. La capacidad formateada suele ser 15% menor que la capacidad sin formato.

Todos los discos tienen brazos móviles que pueden desplazarse hacia adentro y hacia afuera hasta diferentes distancias radiales del eje sobre el que gira el plato. A cada distancia radial puede escribirse una pista diferente. Así, las pistas son una serie de círculos concéntricos alrededor del eje. La anchura de una pista depende del tamaño de la cabeza y de la exactitud con que ésta puede ubicarse radialmente. Con la tecnología actual, los discos tienen entre 800 y 2000 pistas por centímetro, lo que da anchos de pista del orden de 5 a 10 micras (1 micra = 1/1000 mm). Cabe señalar que una pista no es un surco físico en la superficie, sino sólo un anillo de material magnetizado, con delgadas áreas protectoras que lo separan de las pistas que están más adentro y más afuera.

La densidad lineal de bits alrededor de la circunferencia de la pista es diferente de la radial, y en buena medida está determinada por la pureza de la superficie y la calidad del aire. Los discos actuales alcanzan densidades de 50,000 a 100,000 bits/cm. Por tanto, un bit es unas 50 veces más grande en la dirección radial que a lo largo de la circunferencia. A fin de elevar la calidad de la superficie y del aire, casi todos los discos se sellan en la fábrica para impedir la entrada de polvo. Tales unidades se llaman **discos Winchester**. Las primeras unidades (creadas por IBM) tenían 30 MB de almacenamiento fijo sellado y 30 MB de almacenamiento removible. Supuestamente, estos discos 30-30 recordaban a la gente los rifles Winchester 30-30 que tan importante papel desempeñaron en la frontera de Estados Unidos, por lo que se conservó el nombre "Winchester".

Casi todos los discos consisten en varios platos apilados verticalmente, como se muestra en la figura 2-20. Cada superficie tiene su propio brazo y cabeza. Todos los brazos están soldados juntos de modo que se mueven simultáneamente a una posición radial dada. El conjunto de todas las pistas en una posición radial dada se llama **cilindro**.

El desempeño de los discos depende de diversos factores. Para leer o escribir un sector, primero el brazo debe moverse a la posición radial correcta. Esta acción se denomina **búsqueda** (*seek*). Los tiempos de búsqueda promedio (entre pistas al azar) son del orden de 5 a 15 ms, aunque se ha logrado bajar el tiempo de búsqueda entre pistas consecutivas a menos de 1 ms. Una vez que la cabeza está ubicada radialmente, hay un retraso, llamado **latencia rotacional** en lo que el sector deseado gira hasta quedar bajo la cabeza. Casi todos los discos giran a 3600, 5400 o 7200 rpm, así que el retardo medio (media rotación) es de 4 a 8 ms. En el extremo superior hay discos que giran a 10,800 rpm (o sea, a 180 rotaciones/s). El tiempo de transferencia depende de la densidad lineal y de la velocidad de rotación. Con las tasas de transferencia típicas de 5 a 20 MB/s, un sector de 512 bytes tarda entre 25 y 100 μ s. Por consiguiente, el tiempo de búsqueda y la latencia rotacional dominan el tiempo de transferencia. La lectura de sectores al azar dispersos por todo el disco es obviamente una forma poco eficiente de operar.

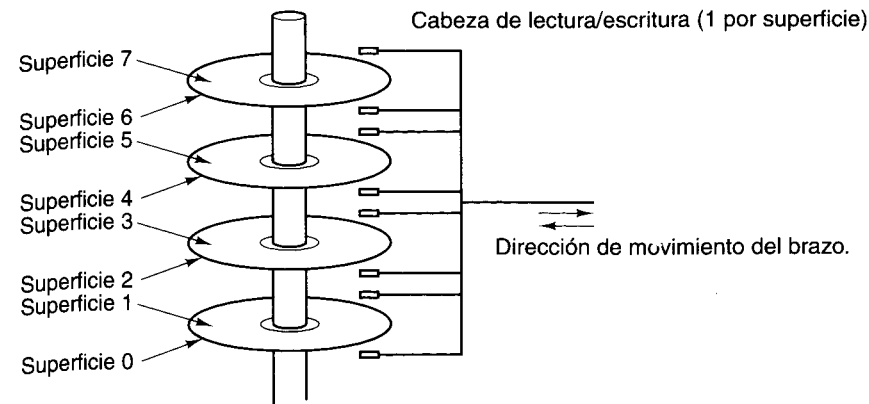


Figura 2-20. Disco con cuatro platos.

Vale la pena mencionar que, a causa de los preámbulos, los ECC, las separaciones entre sectores, los tiempos de búsqueda y las latencias rotacionales, hay una gran diferencia entre la rapidez máxima de ráfaga de una unidad y su rapidez máxima sostenida. La rapidez máxima de ráfaga es la tasa de datos una vez que la cabeza está sobre el primer bit de datos. La computadora debe poder manejar datos que llegan con esta rapidez. Sin embargo, la unidad sólo puede mantener esa rapidez durante un sector. Para algunas aplicaciones, como multimedia, lo que importa es la tasa media sostenida durante un periodo de segundos, y esto debe tener en cuenta también las búsquedas y retrasos rotacionales.

Cuando los discos giran de 60 a 120 revoluciones por segundo se calientan y expanden, lo que altera su geometría física. Algunas unidades necesitan recalibrar sus mecanismos de ubicación periódicamente para compensar esta expansión. Esto lo hacen obligando a las cabezas a salir o meterse totalmente. Tales recalibraciones pueden dar al traste con aplicaciones multimedia, que esperan un flujo más o menos constante de bits a la tasa sostenida máxima. Para manejar aplicaciones multimedia, algunas compañías fabrican **unidades de disco audiovisual** que no requieren estas recalibraciones térmicas.

Si pensamos un poco y usamos la vieja fórmula de primaria para obtener la circunferencia de un círculo, $c = 2\pi r$, veremos que las pistas exteriores comprenden una mayor distancia lineal que las interiores. Puesto que todos los discos magnéticos giran con una velocidad angular constante, sin importar dónde estén las cabezas, este hecho crea un problema. En las unidades viejas, los fabricantes utilizaban la densidad lineal máxima posible en la pista más interior, y densidades lineales de bits cada vez más bajas en las pistas más exteriores. Si un disco tenía 18 sectores por pista, por ejemplo, cada una ocupaba 20 grados de arco, sin importar en qué cilindro estuviera.

Hoy día se sigue una estrategia distinta. Los cilindros se dividen en zonas (por lo regular de 10 a 30 por unidad) y el número de sectores por pista se incrementa en cada zona a medida que ésta se aleja de la pista más interior. Este cambio dificulta seguirle la pista a la información, pero incrementa la capacidad de la unidad, lo cual se considera más importante. Todos los sectores tienen el mismo tamaño. Por fortuna, algunas cosas de la vida nunca cambian.

Cada unidad tiene asociado un **controlador de disco**, un chip que controla la unidad. Algunos controladores contienen una CPU dedicado. Las tareas del controlador incluyen aceptar comandos del software, como **READ**, **WRITE** y **FORMAT** (escribir todos los preámbulos), controlar el movimiento del brazo, detectar y corregir errores, y convertir los bytes de 8 bits que se leen de la memoria en un flujo de bits en serie y viceversa. Algunos controladores también colocan en buffers sectores enteros, colocan en caché sectores leídos que podrían usarse en el futuro, y mantienen el mapa de sectores defectuosos. Esta última función es necesaria cuando hay sectores que tienen puntos magnetizados de forma permanente. Cuando el controlador descubre un sector defectuoso, lo sustituye por uno de los sectores de reemplazo reservados para este fin dentro de cada cilindro o zona.

2.3.3 Discos flexibles

Con la llegada de la computadora personal, se requirió una forma de distribuir software. La solución fue el **disquete** o **disco flexible**, un medio pequeño y removible que recibió ese nombre porque los primeros eran físicamente flexibles. En realidad, IBM inventó el disco flexible para grabar información de mantenimiento acerca de sus mainframes para el personal de servicio, pero los fabricantes de computadoras personales pronto lo adoptaron como una forma cómoda de distribuir y vender software.

Las características generales son las mismas que las de los discos que acabamos de describir, excepto que, a diferencia de los discos duros, en los que las cabezas flotan sobre la superficie en un colchón de aire en rápido movimiento, las cabezas de las unidades de disco sí tocan los disquetes. El resultado es que tanto los medios como las cabezas se desgastan con comparativa rapidez. A fin de reducir el desgaste, las computadoras personales retraen las cabezas y detienen la rotación cuando una unidad no está leyendo o escribiendo. Por consiguiente, cuando se emite la siguiente orden de leer o escribir, hay un retraso de cerca de medio segundo mientras el motor adquiere la velocidad necesaria.

Hay dos tamaños: 5.25 pulgadas y 3.5 pulgadas. Cada uno tiene una versión de baja densidad (LD) y una de alta densidad (HD). Los disquetes de 3.5 pulgadas están protegidos con una cubierta rígida, así que no son realmente "flexibles". Puesto que los disquetes de 3.5" almacenan más datos y están mejor protegidos, prácticamente ya sustituyeron a los de 5.25", más viejos. En la figura 2-21 se muestran los parámetros más importantes de los cuatro tipos.

2.3.4 Discos IDE

Los discos para las computadoras personales modernas evolucionaron a partir del de la IBM PC XT, que era un disco Seagate de 10 MB manejado por un controlador de disco Xebec en una tarjeta insertable. El disco Seagate tenía 4 cabezas, 306 cilindros y 17 sectores/pista. El controlador podía manejar dos unidades. El sistema operativo leía de un disco y escribía en él colocando parámetros en registros de la CPU e invocando después al **BIOS (sistema básico de entrada y salida, Basic Input Output System)** situado en la memoria sólo de lectura integrada a la PC. El BIOS emitía las instrucciones de máquina necesarias para cargar los registros del controlador de disco que iniciaban las transferencias.

Parámetros	LD 5.25"	HD 5.25"	LD 3.5"	HD 3.5"
Tamaño (pulgadas)	5.25	5.25	3.5	3.5
Capacidad (bytes)	360K	1.2M	720K	1.44M
Pistas	40	80	80	80
Sectores/pista	9	15	9	18
Cabezas	2	2	2	2
Rotaciones/min	300	360	300	300
Tasa de datos (kbps)	250	500	250	500
Tipo	Flexible	Flexible	Rígido	Rígido

Figura 2-21. Características de los cuatro tipos de discos flexibles.

La tecnología evolucionó rápidamente, de tener el controlador en una tarjeta aparte a integrarlo íntimamente a las unidades de disco, comenzando por las unidades **IDE (circuitos integrados a la unidad, Integrated Drive Electronics)** a mediados de los ochenta. Sin embargo, las convenciones de invocación del BIOS no se modificaron por razones de compatibilidad con modelos anteriores. Estas convenciones direccionaban sectores dando sus números de cabeza, cilindro y sector. Las cabezas y cilindros se numeraban a partir del 0, y los sectores, a partir del 1. Esta decisión probablemente se debió a un error del programador original del BIOS, quien escribió su obra maestra en el ensamblador del 8088. Con 4 bits para la cabeza, 6 bits para el sector y 10 bits para el cilindro, la unidad de disco más grande podía tener 16 cabezas, 63 sectores y 1024 cilindros, para un total de 1,032,192 sectores. Una unidad así tiene una capacidad de 528 MB, que probablemente parecía casi infinita en ese entonces pero que ahora podría considerarse pequeña. (¿Criticaría hoy una máquina nueva que no pudiera manejar unidades de más de un terabyte?)

En relativamente poco tiempo aparecieron unidades de más de 528 MB, pero con la geometría equivocada (por ejemplo, 4 cabezas, 32 sectores, 2000 cilindros). El sistema operativo no tenía forma de direccionarlos a causa de las convenciones de invocación de BIOS estáticas. El resultado es que los controladores de disco comenzaron a mentir, fingiendo que la geometría estaba dentro de los límites de BIOS cuando en realidad estaba estableciendo una nueva correspondencia entre la geometría virtual y la geometría real. Aunque esta estrategia funcionó, hizo estragos con los sistemas operativos que colocaban cuidadosamente los datos a modo de minimizar los tiempos de búsqueda.

Con el paso del tiempo, las unidades IDE se convirtieron en unidades **EIDE (IDE extendido, Extended IDE)**, que también reconocen un segundo esquema de direccionamiento llamado **LBA (direccionamiento por bloque lógico, Logical Block Addressing)**, que simplemente numera los sectores a partir del 0 y hasta un máximo de $2^{24} - 1$. Este esquema requiere que el controlador convierta las direcciones LBA en direcciones de cabeza, sector y

cilindro, pero al menos logra superar el límite de los 528 MB. Las unidades y controladores EIDE tienen también otras mejoras, como la capacidad de controlar cuatro unidades en lugar de dos, una tasa de transferencia más alta y la capacidad para controlar unidades CD-ROM.

Los discos IDE y EIDE eran originalmente sólo para sistemas basados en Intel, ya que la interfaz es una copia exacta del bus de la IBM PC. Sin embargo, hoy día unas cuantas computadoras más los usan por su bajo precio.

2.3.5 Discos SCSI

Los discos SCSI no son diferentes de los IDE en cuanto a la organización de sus cilindros, pistas y sectores, pero tienen una interfaz diferente y tasas de transferencia mucho más altas. La historia de SCSI se remonta a Howard Shugart, el inventor del disco flexible, cuya compañía introdujo el disco SASI (Shugart Associates System Interface) en 1979. Después de algunas modificaciones y muchos alegatos, el ANSI lo estandarizó en 1986 y cambió su nombre a **SCSI (interfaz de sistema de cómputo pequeño, Small Computer System Interface)**. SCSI se pronuncia "escosi". En 1994, el ANSI emitió un estándar actualizado, SCSI-2, que en buena medida ha sustituido al SCSI-1 original. Todavía se está trabajando en el SCSI-3, aunque partes de la norma ya están terminadas. Algunos fabricantes han implementado estas partes y las usan para decir que cumplen con SCSI-3.

Puesto que los discos SCSI tienen tasas de transferencia altas, son el disco estándar en casi todas las estaciones de trabajo UNIX de Sun, HP, SGI y otros fabricantes; también son el disco estándar de las Macintosh y son comunes en las PC Intel actuales, sobre todo los servidores de red. En la figura 2-22 se dan los parámetros de algunas de las configuraciones más comunes.

Nombre	Bits de datos	Frecuencia en MHz	MB/s
SCSI-1	8	5	5
SCSI-2	8	5	5
SCSI-2 rápido	8	10	10
SCSI-2 rápido y ancho	16	10	20
Ultra SCSI	16	20	40

Figura 2-22. Algunos posibles parámetros de SCSI.

SCSI es algo más que una interfaz de disco duro; es un bus al que pueden conectarse un controlador SCSI y hasta siete dispositivos. Éstos pueden incluir uno o más discos duros SCSI, CD-ROM, grabadoras de CD, escáneres, unidades de cinta y otros periféricos SCSI. Cada dispositivo SCSI tiene un identificador único, de 0 a 7 (15 en el caso de SCSI ancho). Cada dispositivo tiene dos conectores: uno para entrada y otro para salida. La salida de un dispositivo se conecta a la entrada de otro con cables, en serie, como en una serie barata de focos navideños. El último dispositivo en la serie debe terminarse para evitar que las reflexiones de los extremos del bus SCSI interfieran con otros datos que van sobre el bus. Por lo regular el controlador está en una tarjeta insertable que es el principio de la cadena de cables, aunque la norma no exige estrictamente esta configuración.

El cable más común para SCSI de 8 bits tiene 50 hilos, 25 de los cuales son tierras apareadas uno a uno con los otros 25 hilos para proporcionar la excelente inmunidad al ruido que se necesita para el funcionamiento a alta velocidad. De los 25 hilos, ocho son para datos, uno es para paridad, nueve son para control y los demás son para la potencia o se reservan para usos futuros. Los dispositivos de 16 bits (y 32 bits) necesitan un segundo cable para las señales adicionales. Los cables pueden tener varios metros de longitud, lo que permite tener unidades de disco externas, escáneres, etcétera.

Los controladores y periféricos SCSI pueden operar sea como transmisores o bien como receptores. Por lo regular el controlador, actuando como transmisor, emite comandos a discos y otros periféricos que actúan como receptores. Estos comandos son bloques de hasta 16 bytes que le dicen al receptor qué debe hacer. Los comandos y respuestas se dan en fases, utilizando diversas señales de control para delimitar las fases y arbitrar el acceso al bus cuando varios dispositivos están tratando de usar el bus al mismo tiempo. Este arbitraje es importante porque SCSI permite a todos los dispositivos operar al mismo tiempo, lo que podría mejorar considerablemente el desempeño en un entorno en el que varios procesos están activos simultáneamente (por ejemplo, UNIX o Windows NT). IDE y EIDE sólo permiten un dispositivo activo a la vez.

2.3.6 RAID

El desempeño de las CPU se ha aumentado exponencialmente durante la década pasada, duplicándose aproximadamente cada 18 meses. No ha sucedido lo mismo con el desempeño de los discos. En los años setenta los tiempos de búsqueda promedio en discos para minicomputadora eran de 50 a 100 ms. Hoy esos tiempos son de 10 ms. En casi todas las industrias técnicas (digamos la de automóviles o de aviación) un mejoramiento en el desempeño por un factor de 5 a 10 en dos décadas sería una noticia excelente, pero en la industria de las computadoras es vergonzoso. Así, la brecha entre el desempeño de la CPU y el del disco ha aumentado con el tiempo.

Como hemos visto, el procesamiento en paralelo se está utilizando cada vez más para acelerar el funcionamiento de las CPU. A varias personas se les ha ocurrido que la E/S paralela también podría ser una buena idea. En su artículo de 1988, Patterson *et al.* sugirieron seis organizaciones de disco específicas que podrían servir para mejorar el desempeño de los discos, su confiabilidad o ambas cosas (Patterson *et al.*, 1988). La industria adoptó rápidamente estas ideas y han dado pie a una nueva clase de dispositivo de E/S llamado **RAID**. Patterson *et al.* definieron a RAID como una **formación redundante de discos de bajo costo** (*Redundant Array of Inexpensive Disks*), pero la industria redefinió la "I" de modo que significara "independientes" en lugar de "de bajo costo" (¿tal vez así podrían usar discos caros?). Puesto que también se necesitaba un villano (como en RISC *versus* CISC, también debido a Patterson), el malo aquí fue el **SLED (disco único, grande y costoso, Single Large Expensive Disk)**.

La idea en que se basa RAID es instalar una caja llena de discos junto a la computadora, que por lo regular es un servidor grande, sustituir la tarjeta del controlador de disco por un controlador de RAID, copiar los datos en el RAID, y continuar con las operaciones normales. En otras palabras, el sistema operativo veía el RAID como si fuera un SLED, sólo que el desempeño y la confiabilidad mejoraban. Puesto que los discos SCSI tienen buen desempe-

ño, precio bajo y la capacidad para conectar hasta siete unidades a un solo controlador (15 en el caso de SCSI ancho), es natural que la mayor parte de los RAID conste de un controlador RAID SCSI más una caja de discos SCSI que el sistema operativo ve como un solo disco grande. De este modo, no se requieren cambios al software para usar el RAID, un punto importante para convencer a los administradores de sistemas de comprarlo.

Además de aparentar ser un solo disco ante el software, todos los RAID tienen la propiedad de que los datos se distribuyen entre todas las unidades, a fin de permitir la operación en paralelo. Patterson y otros autores definieron varios esquemas para hacerlo y ahora se conocen como RAID nivel 0 hasta RAID nivel 5. Además, existen algunos niveles menores que no se discutirán. El término "nivel" es un tanto engañoso porque no existe una jerarquía; simplemente hay seis posibles organizaciones distintas.

RAID nivel 0 se ilustra en la figura 2-23(a), y consiste en ver el disco único virtual simulado por el RAID como si estuviera dividido en tiras (*strips*) de k sectores cada una, con los sectores 0 a $k - 1$ en la tira 0, los sectores k a $2k - 1$ en la tira 1, etc. Para $k = 1$, cada tira es un sector; para $k = 2$, una tira es dos sectores, etc. La organización RAID nivel 0 escribe tiras consecutivas en las unidades por turno circular, como se muestra en la figura 2-23(a) para un RAID con cuatro unidades de disco. La distribución de datos entre varias unidades de esta forma se llama **striping (multiplexaje de datos)**. Por ejemplo, si el software emite un comando para leer un bloque de datos que consiste en cuatro tiras consecutivas que comienzan en una frontera de tira, el controlador de RAID desglosará este comando en cuatro comandos individuales, uno para cada uno de los cuatro discos, y los hará operar en paralelo. Así tenemos E/S en paralelo sin que el software se entere de ello.

El RAID nivel 0 funciona mejor cuando la solicitud de datos es grande, mientras más grande mejor. Si una solicitud es mayor que el número de unidades multiplicado por el tamaño de una tira, algunas unidades recibirán varias solicitudes, de modo que cuando terminan de atender la primera solicitud comienzan a atender la segunda. Corresponde al controlador dividir en partes la solicitud y alimentar los comandos apropiados a los discos correctos en la secuencia debida, y luego ensamblar los resultados correctamente en la memoria. El desempeño es excelente y la implementación es sencilla.

El peor funcionamiento de RAID nivel 0 se obtiene con sistemas operativos que acostumban solicitar datos sector por sector. Los resultados son correctos, pero no hay paralelismo y por tanto no mejora el desempeño. Otra desventaja de esta organización es que la confiabilidad podría ser peor que si se tuviera un SLED. Si un RAID consta de cuatro discos, cada uno con un tiempo medio antes de fallo de 20,000 horas, una unidad fallará aproximadamente una vez cada 5000 horas y se perderán totalmente los datos. Un SLED con un tiempo medio antes de fallo de 20,000 horas sería cuatro veces más confiable. Dado que este diseño no incluye una verdadera redundancia, no es realmente un RAID.

La siguiente opción, RAID nivel 1, que se muestra en la figura 2-23(b), es un verdadero RAID. Se duplican todos los discos, de modo que hay cuatro discos primarios y cuatro discos de respaldo. En una escritura, cada tira se escribe dos veces. En una lectura, puede utilizarse cualquiera de las dos copias, lo que permite distribuir la carga entre más unidades. Por consiguiente, el desempeño de escritura no es mejor que con una sola unidad, pero el desempeño de lectura puede ser hasta dos veces mejor. La tolerancia de fallos es excelente: si una unidad se descompone, simplemente se usa la copia. La recuperación consiste en instalar una nueva unidad y copiar en ella el contenido de la unidad de respaldo.